



中华人民共和国国家标准

GB/T 20090.16—2016

信息技术 先进音视频编码 第 16 部分：广播电视视频

Information technology—Advanced coding of audio and video—
Part 16: Broadcasting video

2016-04-25 发布

2016-11-01 实施

中华人民共和国国家质量监督检验检疫总局 发布
中国国家标准化管理委员会

目 次

前言	I
引言	II
1 范围	1
2 规范性引用文件	1
3 术语和定义	1
4 缩略语	9
5 约定	10
6 编码位流的结构	17
7 位流的语法和语义	21
8 解析过程	54
9 解码过程	73
附录 A (规范性附录) 伪起始码	111
附录 B (规范性附录) 档次和级别	112
附录 C (规范性附录) 位流虚拟参考解码器	117
附录 D (规范性附录) 基本熵编码码表	122
附录 E (资料性附录) 高级熵编码解码器参考实现方法	132

前 言

GB/T 20090 在《信息技术 先进音视频编码》的总标题下,当前包括以下 14 个部分:

- 第 1 部分:系统;
- 第 2 部分:视频;
- 第 3 部分:音频;
- 第 4 部分:符合性测试;
- 第 5 部分:参考软件;
- 第 6 部分:面向数字版权管理的可信解码器与访问协议;
- 第 7 部分:面向交互应用的视频编解码;
- 第 8 部分:在 IP 网络上传输 AVS;
- 第 9 部分:AVS 文件格式;
- 第 10 部分:移动语音和音频;
- 第 11 部分:同步文本;
- 第 12 部分:综合场景;
- 第 13 部分:视频工具集;
- 第 16 部分:广播电视视频。

本部分为 GB/T 20090 的第 16 部分。

本部分按照 GB/T 1.1—2009 给出的规则起草。

请注意本文件的某些内容可能涉及专利。本文件的发布机构不承担识别这些专利的责任。

本部分由全国信息技术标准化技术委员会(SAC/TC 28)提出并归口。

本部分起草单位:北京大学、中央电视台、浙江大学、华为技术有限公司、北京博雅华录视听技术研究院有限公司、广州柯维新数码科技有限公司、深圳市海思半导体有限公司、上海国茂数字技术有限公司、清华大学、国家广播电影电视总局广播科学研究院、国家广播电影电视总局广播电视规划院、中国科学院计算技术研究所、北京大学深圳研究生院、中山大学。

本部分主要起草人:高文、丁文华、黄铁军、虞露、马思伟、郑萧桢、梁凡、何芸、张伟民、赵海武、张莉、郑建铎、曾志华、王荣刚、郭晓强、邓向冬、潘晓菲、王强、董文辉、林永兵、张贤国、王振宇。

引 言

GB/T 20090 的本部分遵循 GB/T 20090.2—2013,增加了帧级加权量化、同极性场跳过模式编码、增强场编码等技术。

本部分的发布机构提请注意如下事实,声明符合本部分时,可能涉及 8.2、8.4、9.2、9.3、9.4.2、9.4.5、9.4.6.2、9.4.6.3、9.4.9、9.4.9、9.5.1、9.5.2、9.6、9.7、9.9.2.2、9.9.2.3、9.11、附录 A 和附录 D 相关的专利的使用。

本文件的发布机构提请注意,声明符合本文件,可能涉及 65 项与数字音频编解码技术相关的专利的使用。

201210033977.4,使用非固定参考场前向预测跳过模式的编码解码方法;201110300629.4,选择参考场及获取时域运动矢量的方法;ZL200680014013.8,一种实现算术编解码的方法及装置;ZL200780000403.4,在编解码中的实现量化的方法和装置;ZL200710178532.4,图像编/解码方法、装置和系统;ZL200810102353.7,运动矢量的缩放方法和装置、编解码方法和系统;ZL03148149.3,运用于图像编码和视频编码的 8×8 整数变换方法和装置;ZL03115997.4,运用于图像编码和视频编码的整数变换方法和装置;ZL03156124.1,图像和视频编码中兼容 4×4 变换的 8×8 变换方法和装置;ZL03150939.8,视频编解码中运动矢量的预测方法和装置;ZL03150940.1,视频编解码中帧内预测的方法和装置;ZL03143431.2,视频图像亚像素插值的方法和装置;ZL03143429.0,一种视频编解码中的运动矢量预测方法和装置;ZL200310108470.1,视频编解码中变换系数块的扫描方法和装置;ZL200310109484.5,信息熵保持编码方法与装置;ZL200310109499.1,信息熵保持解码方法与装置;ZL200310108468.4, $1/4$ 像素精度插值的方法和装置;ZL200410017419.4,图像和视频编码中多尺度兼容处理装置及其方法;ZL03146301.0,一种视频图像编码方法;ZL200410012658.0,一种环路滤波方法和环路滤波器;ZL200410012857.1,视频编码的整数变换矩阵选择方法及相关的整数变换方法;ZL200410061094.X,一种复杂度可分级的帧内预测方法;ZL200510104853.0,一种基于上下文的自适应熵编/解码方法;ZL02125274.2,结合率失真优化的码率控制的方法及其装置;ZL02130833.0,用于视频编码的新型空间预测方法及其装置;ZL03157077.1,一种用于视频编码的编码端/解码端双向预测方法;ZL200310116085.1,一种参考图像缓冲区管理方法;ZL200310116087.0,最大宏块类型与编码块模式联合编/解码的方法;ZL200310116088.5,一种视频图像跳过宏块的编码方法;ZL03148029.2,用于视频编码的预测图像生成方法;ZL200410006340.1,一种帧内预测模式选取方法;ZL200410029452.9,一种 MPEG-2 到 AVS 视频码流格式转换方法及其装置;ZL200310116089.X,固定参考帧数编码方式下获取图像参考块的方法;ZL03157189.1,用于视频预测残差系数编码的熵编码方法;ZL03157078.X,一种快速定位编码图像的方法;ZL03157076.3,一种定位编码图像标识的方法;ZL03149393.9,用于编码处理的 4 输入/输出的整数变换/反变化方法;ZL03148033.0,一种用于编码处理的 8 输入/输出的整数变化/反变换方法;ZL03148032.2,基于位平面的不等长的量化/反量化方法;ZL03148031.4,多视角视频编解码预测补偿方法及装置;ZL03148030.6,用于视频编码运动矢量预测方法;ZL200310116090.2,直接编码模式下确定参考图像块的方法;ZL200310100469.4,一种用剪辑标识码来辅助视频剪辑的方法;ZL03151460.X,可随机访问数据流的封装方法及其对应的数据恢复方法;ZL200310107985.X,一种可随机访问数据流的组织方法及其对应的解码方法;ZL03159169.8,一种视频编解码方法;ZL03120952.1,一种视频图像编码方法,ZL03152502.4;一种基于多级流水线结构的实时 $1/4$ 插值方法;ZL03156678.2,一种分样本插值滤波方法;ZL200310101370.6,一种分样本插值滤波方法;ZL200310101445.0,基于流水线的帧内预测模式块编码加速方法;ZL200410015637.4,图像和视频信号的变换装置及其方法;

200510061395.7,一种用于二进制化解码得到解码数据值的方法和装置;200480016912.2,一种视频宏块模式解码方法;200680014019.5,一种二进制化方法及装置;201010135828.X,视频数据的打包、编解码方法及装置及系统;200880001326.9,多参考图像压缩码流实现随机访问的解码方法及解码器;200710126108.5,实现随机访问的方法及解码器(0989702CN 同族);201310746046.3,表示视频图像显示顺序的方法;201110240524.4,运用于图像编码和视频编码的六角变换方法和装置;201110335481.8,视频编码合并模式双向补偿方法;201310049872.2,基于颜色空间变换进行编解码误差修正的方法和系统;201310173139.1,利用 YUV 与 RGB 空间联合修正视频编解码误差的方法;201310048281.3,一种图像和视频编解码方法和系统;201210312587.0,一种快速防止出现伪起始码的方法及装置;200410015637.4,图像和视频信号的变换装置及其方法。

本部分的发布机构对于该专利的真实性、有效性和范围无任何立场。

专利持有人已向本部分的发布机构保证,他愿意同任何申请人在合理且无歧视的条款和条件下,就专利授权许可进行谈判。该专利持有人的声明已在本部分的发布机构备案。

联系人:黄铁军(数字音视频编解码技术标准工作组秘书长)

通讯地址:北京大学理科 2 号楼 2641 室

邮政编码:100871

电子邮件:tjhuang@pku.edu.cn

电 话:+10-62756172

传 真:+10-62751638

网 址:<http://www.avs.org.cn>

请注意除上述专利外,本部分的某些内容仍可能涉及专利。本部分的发布机构不承担识别这些专利的责任。

信息技术 先进音视频编码

第 16 部分:广播电视视频

1 范围

GB/T 20090 的本部分规定了多种比特率、分辨率和质量的视频压缩方法,并且规定了解码过程。本部分适用于地面电视、有线电视、卫星电视、网络等应用。

2 规范性引用文件

下列文件对于本文件的应用是必不可少的。凡是注日期的引用文件,仅注日期的版本适用于本文件。凡是不注日期的引用文件,其最新版本(包括所有的修改单)适用于本文件。

GB/T 20090.2—2013 信息技术 先进音视频编码 第 2 部分:视频

3 术语和定义

GB/T 20090.2—2013 界定的以及下列术语和定义适用于本文件。为了便于使用,以下重复列出了 GB/T 20090.2—2013 中的某些术语和定义。

3.1

保留 reserved

定义了一些特定语法元素值,这些值用于将来对本部分的扩展。

注:这些值不出现在符合本部分的位流中。

[GB/T 20090.2—2013,定义 2.1]

3.2

变长编码 variable length coding

一个可逆的熵编码过程,它将短的码字分配给出现频率较高的符号,将长的码字分配给出现频率较低的符号。

[GB/T 20090.2—2013,定义 2.2]

3.3

变换系数 transform coefficient

变换域上的一个标量。

[GB/T 20090.2—2013,定义 2.3]

3.4

编码表示 encoding presentation

数据编码后的形式。

[GB/T 20090.2—2013,定义 2.4]

3.5

编码过程 encoding process

产生符合本部分位流的过程。

注:本部分不规定该过程。

[GB/T 20090.2—2013,定义 2.5]

3.6

编码器 encoder

编码过程的实现。

[GB/T 20090.2—2013, 定义 2.6]

3.7

编码图像 coded picture

一帧图像的编码表示。

[GB/T 20090.2—2013, 定义 2.7]

3.8

标志 flag

一个二值变量。

[GB/T 20090.2—2013, 定义 2.8]

3.9

补偿 compensation

求由语法元素解码得到的样本残差与其对应的预测值之和。

[GB/T 20090.2—2013, 定义 2.9]

3.10

残差 residual

样本或数据元素的重建值与其预测值之差。

[GB/T 20090.2—2013, 定义 2.10]

3.11

参考索引 reference index

解码图像缓冲区中参考图像或其中场的编号。

[GB/T 20090.2—2013, 定义 2.12]

3.12

参考图像 reference picture

解码过程中用于后续图像帧间预测的图像。

[GB/T 20090.2—2013, 定义 2.13]

3.13

层 layer

位流中的分级结构, 高层包含低层。编码层由高到低依次为: 序列、图像、条带、宏块和块。

[GB/T 20090.2—2013, 定义 2.14]

3.14

场 field

由构成帧的 3 个样本矩阵中相间的行构成。

[GB/T 20090.2—2013, 定义 2.15]

3.15

档次 profile

本部分规定的语法、语义及算法的子集。

[GB/T 20090.2—2013, 定义 2.16]

3.16

二元符号 binary bin

组成二元符号串的符号, 包括“0”和“1”。

[GB/T 20090.2—2013, 定义 2.17]

3.17

二元符号串 binary bin string

有限位二元符号组成的有序序列,最左边符号是最高有效位(MSB),最右边符号是最低有效位(LSB)。

[GB/T 20090.2—2013,定义 2.18]

3.18

分量 component

图像的三个样值矩阵(亮度和两个色度)中的一个矩阵或矩阵中的单个样值。

[GB/T 20090.2—2013,定义 2.19]

3.19

反变换 inverse transform

将变换系数矩阵转换成空域样值矩阵的过程。

[GB/T 20090.2—2013,定义 2.20]

3.20

反量化 dequantization

对量化系数缩放后得到变换系数的过程。

[GB/T 20090.2—2013,定义 2.21]

3.21

光栅扫描 raster scan

将二维矩形光栅映射到一维光栅,一维光栅的入口从二维光栅的第一行开始,然后扫描第二行、第三行,依次类推。光栅中的行从左到右扫描。

[GB/T 20090.2—2013,定义 2.22]

3.22

宏块 macroblock

包括一个 16×16 的亮度样值块和对应的色度样值块。

[GB/T 20090.2—2013,定义 2.24]

3.23

宏块地址 macroblock address

从图像左上角的宏块开始,沿光栅扫描的顺序编号,起始号为 0。

[GB/T 20090.2—2013,定义 2.25]

3.24

宏块行 macroblock line

在编码的图像相同的垂直位置,从左边界到右边界连续的宏块,其高度是 16 个样本。

[GB/T 20090.2—2013,定义 2.26]

3.25

宏块位置 macroblock position

图像中一个宏块的二维坐标,表示为 (x, y) 。

示例:如果当前图像的两场的编码数据合并成帧,图像左上角的宏块 $(x, y) = (0, 0)$,对每个宏块列,从左到右 x 依次加 1,对每个宏块行,从上到下 y 依次加 1。如果当前图像的两场的编码数据依次出现,第一场左上角的宏块 $(x, y) = (0, 0)$,对第一场的每个宏块列,从左到右 x 依次加 1,对第一场的每个宏块行,从上到下 y 依次加 1;第二场左上角的宏块 $(x, y) = (0, (H+31) \gg 5)$, H 是图像垂直方向扫描行数,对第二场的每个宏块列,从左到右 x 依次加 1,对第二场的每个宏块行,从上到下 y 依次加 1。

[GB/T 20090.2—2013,定义 2.27]

3.26

后向预测 backward prediction

用显示顺序上将来的参考图像对当前图像进行预测。

[GB/T 20090.2—2013, 定义 2.28]

3.27

划分 partitioning

将一个集合分为子集的过程。集合中的每个元素属于且只属于某一个子集。

[GB/T 20090.2—2013, 定义 2.29]

3.28

级别 level

在某一档次下对语法元素和语法元素参数值的限定集合。

[GB/T 20090.2—2013, 定义 2.30]

3.29

交流系数 AC coefficient

AC 系数

二维变换域上索引号不全为 0 的变换系数。

[GB/T 20090.2—2013, 定义 2.31]

3.30

解码处理 decode processing

解析过程和解码过程。

[GB/T 20090.2—2013, 定义 2.32]

3.31

解码过程 decoding process

由语法元素产生解码图像的过程。

[GB/T 20090.2—2013, 定义 2.33]

3.32

解码器 decoder

完成解码处理的实体。

[GB/T 20090.2—2013, 定义 2.34]

3.33

解码顺序 decoding order

解码过程根据图像之间的预测关系,对每帧图像解码的顺序。

[GB/T 20090.2—2013, 定义 2.35]

3.34

解码图像 decoded picture

解码器根据位流重建的图像。

[GB/T 20090.2—2013, 定义 2.36]

3.35

解码图像缓冲区 decoded picture buffer

保存解码图像并用于预测、输出重排序和输出定时的缓冲区。

[GB/T 20090.2—2013, 定义 2.37]

3.36

解析过程 parse

由位流获得语法元素的过程。

[GB/T 20090.2—2013, 定义 2.38]

3.37

禁止 forbidden

定义了一些特定语法元素值, 这些值不应出现在符合本部分的位流中。

注: 禁止某些值的目的是为了在位流中出现伪起始码。

[GB/T 20090.2—2013, 定义 2.39]

3.38

块 block

一个 $M(\text{列}) \times N(\text{行})$ 的样值矩阵或者变换系数矩阵。大小为 8×8 的块又称为子块。

注: 改写 GB/T 20090.2—2013, 定义 2.40。

3.39

块扫描 block scan

量化系数的特定串行排序方式。

[GB/T 20090.2—2013, 定义 2.41]

3.40

亮度 luma**Y**

表示亮度信号的样值矩阵或单个样值。

[GB/T 20090.2—2013, 定义 2.42]

3.41

量化参数 quantization parameter

在解码过程对量化系数进行反量化的参数。

[GB/T 20090.2—2013, 定义 2.43]

3.42

量化系数 quantization coefficient

反量化前变换系数的值。

[GB/T 20090.2—2013, 定义 2.44]

3.43

X 档次解码器 x-profile decoder

能够解码符合某档次规定的位流的解码器。

[GB/T 20090.2—2013, 定义 2.45]

3.44

起始码 start code

长度为 32 位的二进制码字, 其形式在整个位流中是唯一的。

注: 起始码有多种用途, 其中之一是用来标识位流语法结构的开始。

[GB/T 20090.2—2013, 定义 2.46]

3.45

前向预测 forward prediction

用显示顺序上过去的参考图像对当前图像进行预测。

[GB/T 20090.2—2013, 定义 2.47]

3.46

前向帧间解码图像 forward inter decoded picture

P 帧

帧间预测中只使用前向预测解码的图像。

[GB/T 20090.2—2013, 定义 2.48]

3.47

色度 chroma

Cr、Cb

两种色差信号中任一种的样值矩阵或单个样值。

[GB/T 20090.2—2013, 定义 2.49]

3.48

视频序列 sequence

编码位流的最高层语法结构,包括一个或多个连续的编码图像。

[GB/T 20090.2—2013, 定义 2.50]

3.49

输出重排序延迟 output reorder delay

解码位流中一帧图像到输出该解码图像之间的延迟。这是由图像显示顺序和解码顺序不同造成的。

[GB/T 20090.2—2013, 定义 2.51]

3.50

输出处理过程 output processing

由解码图像得到输出帧或场的过程。

[GB/T 20090.2—2013, 定义 2.52]

3.51

输出顺序 output order

输出解码图像的顺序,与显示顺序相同。

[GB/T 20090.2—2013, 定义 2.53]

3.52

双向预测 bidirectional prediction

用显示顺序上过去和将来的参考图像对当前图像进行预测。

[GB/T 20090.2—2013, 定义 2.54]

3.53

双向帧间解码图像 bidirectional inter decoded picture

B 帧

帧间预测中使用双向预测解码的图像。

[GB/T 20090.2—2013, 定义 2.55]

3.54

随机访问 random access

从某一点而非位流起始点开始对位流解码并恢复出解码图像的能力。

[GB/T 20090.2—2013, 定义 2.56]

3.55

随机访问点 random access point

位流中能进行随机访问的点。

[GB/T 20090.2—2013, 定义 2.57]

3.56

填充位 stuffing bits

编码时插入位流中的位串,在解码时被丢弃。

[GB/T 20090.2—2013,定义 2.58]

3.57

条带 slice

按光栅扫描顺序排列的若干连续宏块。

[GB/T 20090.2—2013,定义 2.59]

3.58

条带头 slice header

编码的条带的一部分,是条带中宏块公用数据元素的编码表示。

[GB/T 20090.2—2013,定义 2.61]

3.59

跳过的宏块 skipped macroblock

除“跳过”指示外,无其他编码数据的宏块。

[GB/T 20090.2—2013,定义 2.62]

3.60

图像重排序 picture reordering

若解码顺序和输出顺序不同,对解码图像进行重排序的过程。

[GB/T 20090.2—2013,定义 2.63]

3.61

位串 bit string

有限位二进制位的有序序列,其最左边位是最高有效位(MSB),最右边位是最低有效位(LSB)。

[GB/T 20090.2—2013,定义 2.64]

3.62

位流 bitstream

编码图像所形成的二进制数据流。

[GB/T 20090.2—2013,定义 2.65]

3.63

位流缓冲区 bitstream buffer

存储位流的缓冲区。

[GB/T 20090.2—2013,定义 2.66]

3.64

位流顺序 bitstream order

编码图像在位流中的排列顺序,与图像解码的顺序相同。

[GB/T 20090.2—2013,定义 2.67]

3.65

显示顺序 display order

显示解码图像的顺序。

[GB/T 20090.2—2013,定义 2.68]

3.66

样本 sample

构成图像的基本元素。

[GB/T 20090.2—2013,定义 2.69]

3.67

样本宽高比 width height ratio

—帧图像中亮度样本列间的水平距离与行间的垂直距离之比。表示为：

$$h \div v$$

式中：

h ——水平宽度；

v ——垂直高度。

[GB/T 20090.2—2013, 定义 2.70]

3.68

样值 sample value

样本的幅值。

[GB/T 20090.2—2013, 定义 2.71]

3.69

游程 run

在解码过程中若干连续的相同数据元素个数。一方面指在块扫描中一个非 0 系数前(沿块扫描顺序)值为 0 的系数的个数；另一方面指跳过的宏块的数目。

[GB/T 20090.2—2013, 定义 2.72]

3.70

预测 prediction

预测过程的具体实现。

[GB/T 20090.2—2013, 定义 2.73]

3.71

预测过程 prediction process

使用预测器对当前解码样值或者数据元素进行估计。

[GB/T 20090.2—2013, 定义 2.74]

3.72

预测值 prediction value

在样值或数据元素的解码过程中,用到的先前已解码的样值或数据元素的组合。

[GB/T 20090.2—2013, 定义 2.75]

3.73

语法元素 syntax element

位流中的数据单元解析后的结果。

[GB/T 20090.2—2013, 定义 2.76]

3.74

源 source

编码前视频素材或其某些属性。

[GB/T 20090.2—2013, 定义 2.77]

3.75

运动矢量 motion vector

用于帧间预测的二维矢量,由当前图像指向参考图像,其值为当前块和参考块之间的坐标偏移量。

[GB/T 20090.2—2013, 定义 2.78]

3.76

帧 frame

视频信号空间信息的表示,由一个亮度样本矩阵(Y)和两个色度样本矩阵(Cb 和 Cr)构成。

[GB/T 20090.2—2013,定义 2.80]

3.77

帧间编码 inter coding

使用帧间预测对宏块或图像进行编码。

[GB/T 20090.2—2013,定义 2.81]

3.78

帧间预测 inter prediction

使用先前解码图像(或场)生成当前图像(或场)样本预测值的过程。

[GB/T 20090.2—2013,定义 2.82]

3.79

帧内编码 intra coding

使用帧内预测对宏块或图像进行编码。

[GB/T 20090.2—2013,定义 2.83]

3.80

帧内解码图像 intra decoded picture**I 帧**

只使用帧内预测解码的图像。

注: 如果 I 帧采用场编码,则第一场只使用帧内预测编码。

[GB/T 20090.2—2013,定义 2.84]

3.81

帧内预测 intra prediction

在相同解码图像(或场)中使用先前解码的样值生成当前样本预测值的过程。

[GB/T 20090.2—2013,定义 2.86]

3.82

直流系数 DC coefficient**DC 系数**

二维变换域上索引号全为 0 的变换系数。

[GB/T 20090.2—2013,定义 2.79]

3.83

字节 byte

8 位的位串。

[GB/T 20090.2—2013,定义 2.87]

3.84

字节对齐 byte alignment

从位流的第一个二进制位开始,某二进制位的位置是 8 的整数倍。

[GB/T 20090.2—2013,定义 2.88]

4 缩略语

下列缩略语适用于本文件。

BBV	位流参考解码器(Bitstream Buffer Verifier)
CBR	恒定比特率(Constant Bit Rate)
LSB	最低有效位(Least Significant Bit)
MB	宏块(Macroblock)
MSB	最高有效位(Most Significant Bit)
VLC	变长编码(Variable Length Coding)
SAR	样本宽高比(Sample Aspect Ratio)
DAR	显示宽高比(Display Aspect Ratio)

5 约定

5.1 概述

本部分中使用的数学运算符和优先级与 C 语言使用的类似。但对整型除法和算术移位操作进行了特定定义。除特别说明外,约定编号和计数从 0 开始。

5.2 算术运算符

算术运算符定义见表 1。

表 1 算术运算符定义

算术运算符	定 义
+	加法运算
—	减法运算(二元运算符)或取反(一元前缀运算符)
×	乘法运算
a^b	幂运算,表示 a 的 b 次幂。也可表示上标
/	整除运算,沿向 0 的取值方向截断。例如, $7/4$ 和 $-7/-4$ 截断至 1, $-7/4$ 和 $7/-4$ 截断至 -1
÷	除法运算,不做截断或四舍五入
$\frac{a}{b}$	除法运算,不做截断或四舍五入
$\sum_{i=a}^b f(i)$	自变量 i 取由 a 到 b (含 b)的所有整数值时,函数 $f(i)$ 的累加和
$a \% b$	模运算, a 除以 b 的余数,其中 a 与 b 都是正整数

5.3 逻辑运算符

逻辑运算符定义见表 2。

表 2 逻辑运算符定义

逻辑运算符	定义
$a \ \&\&\ b$	a 和 b 之间的与逻辑运算
$a \ \ b$	a 和 b 之间的或逻辑运算
!	逻辑非运算

5.4 关系运算符

关系运算符定义见表 3。

表 3 关系运算符定义

关系运算符	定义
$>$	大于
$>=$	大于或等于
$<$	小于
$<=$	小于或等于
$==$	等于
$!=$	不等于

5.5 位运算符

位运算符定义见表 4。

表 4 位运算符定义

位运算符	定义
$\&$	与运算
$ $	或运算
$\sim$	取反运算
$a \gg b$	将 a 以 2 的补码整数表示的形式向右移 b 位。仅当 b 取正数时定义此运算
$a \ll b$	将 a 以 2 的补码整数表示的形式向左移 b 位。仅当 b 取正数时定义此运算

5.6 赋值运算

赋值运算定义见表 5。

表 5 赋值运算定义

赋值运算	定义
$=$	赋值运算符
$++$	递增, $x++$ 相当于 $x=x+1$ 。当用于数组下标时, 在自加运算前先求变量值
$--$	递减, $x--$ 相当于 $x=x-1$ 。当用于数组下标时, 在自减运算前先求变量值
$+=$	自加指定值, 例如 $x+=3$ 相当于 $x=x+3$, $x+=(-3)$ 相当于 $x=x+(-3)$
$-=$	自减指定值, 例如 $x-=3$ 相当于 $x=x-3$, $x-=(-3)$ 相当于 $x=x-(-3)$

5.7 数学函数

数学函数定义见式(1)~式(8)。

$$\text{Abs}(x) = \begin{cases} x & ; \quad x \geq 0 \\ -x & ; \quad x < 0 \end{cases} \dots\dots\dots(1)$$

式中:

x ——自变量。

$$\text{Ceil}(x) = \lceil x \rceil \dots\dots\dots(2)$$

式中:

x ——自变量。

$$\text{Clip1}(x) = \text{Clip3}(0, 2^n - 1, x) \dots\dots\dots(3)$$

式中:

x ——自变量;

n ——样本点精度。

$$\text{Clip3}(i, j, x) = \begin{cases} i & ; \quad x < i \\ j & ; \quad x > j \\ x & ; \quad \text{其他} \end{cases} \dots\dots\dots(4)$$

式中:

x ——自变量;

i ——下界;

j ——上界。

$$\text{Median}(x, y, z) = x + y + z - \text{Min}(x, \text{Min}(y, z)) - \text{Max}(x, \text{Max}(y, z)) \dots\dots\dots(5)$$

式中:

x, y, z ——自变量。

$$\text{Min}(x, y) = \begin{cases} x & ; \quad x \leq y \\ y & ; \quad x > y \end{cases} \dots\dots\dots(6)$$

式中:

x, y ——自变量。

$$\text{Max}(x, y) = \begin{cases} x & ; \quad x \geq y \\ y & ; \quad x < y \end{cases} \dots\dots\dots(7)$$

式中:

x, y ——自变量。

$$\text{Sign}(x) = \begin{cases} 1 & ; \quad x \geq 0 \\ -1 & ; \quad x < 0 \end{cases} \dots\dots\dots(8)$$

式中:

x ——自变量。

5.8 结构关系符

结构关系符定义见表6。

表 6 结构关系符定义

结构关系符	定义
—>	例如:a—>b 表示 a 是一个结构,b 是 a 的一个成员变量

5.9 位流语法、解析过程和解码过程的描述方法

5.9.1 位流语法的描述方法

位流语法描述方法类似 C 语言。位流的语法元素使用粗体字表示,每个语法元素通过名字(用下划线分割的英文字母组,所有字母都是小写)、语法和语义来描述。语法表和正文中语法元素的值用常规字体表示。

某些情况下,可在语法表中应用从语法元素导出的其他变量值,这样的变量在语法表或正文中用不带下划线的小写字母和大写字母混合命名。大写字母开头的变量用于解码当前以及相关的语法结构,也可用于解码后续的语法结构。小写字母开头的变量只在它们所在的小节内使用。

语法元素值的助记符和变量值的助记符与它们的值之间的关系在正文中说明。在某些情况下,二者等同使用。助记符由一个或多个使用下划线分隔的字母组表示,每个字母组以大写字母开始,也可包括多个大写字母。

位串的长度是 4 的整数倍时,可使用十六进制符号表示。十六进制的前缀是“0x”,例如“0x1a”表示位串“0001 1010”。

条件语句中 0 表示 FALSE,非 0 表示 TRUE。

语法表描述了所有符合本部分的位流语法的超集,附加的语法限制在相关条中说明。

表 7 给出了描述语法的伪代码例子。当语法元素出现时,表示从位流中读一个数据单元。

表 7 语法描述的伪代码

伪代码	描述符
/* 语句是一个语法元素的描述符,或者说明语法元素的存在、类型和数值,下面给出两个例子。 */	
syntax_element	ue(v)
conditioning statement	
/* 花括号括起来的语句组是复合语句,在功能上视作单个语句。 */	
{	
statement	
...	
}	
/* “while”语句测试 condition 是否为 TRUE,如果为 TRUE,则重复执行循环体,直到 condition 不为 TRUE。 */	
while (condition)	
statement	

表 7 (续)

伪代码	描述符
/* “do ... while”语句先执行循环体一次,然后测试 condition 是否为 TRUE,如果为 TRUE,则重复执行循环体,直到 condition 不为 TRUE。 */	
Do	
statement	
while (condition)	
/* “if ... else”语句首先测试 condition,如果为 TRUE,则执行 primary 语句,否则执行 alternative 语句。如果 alternative 语句不需要执行,结构的“else”部分和相关的 alternative 语句可忽略。 */	
if (condition)	
primary statement	
else	
alternative statement	
/* “for”语句首先执行 initial 语句,然后测试 condition,如果 condition 为 TRUE,则重复执行 primary 语句和 subsequent 语句直到 condition 不为 TRUE。 */	
for (initial statement; condition; subsequent statement)	
primary statement	

解析过程和解码过程用文字和类似 C 语言的伪代码描述。

5.9.2 函数

5.9.2.1 说明

以下函数用于位流语法描述。假定解码器中存在一个位流指针,这个指针指向位流中要读取的下一个二进制位的位置。函数由函数名及左右圆括号内的参数构成。函数也可没有参数。

5.9.2.2 byte_aligned()

如果位流的当前位置是字节对齐的,返回 TRUE,否则返回 FALSE。

5.9.2.3 next_bits(*n*)

返回位流的随后 *n* 个二进制位,MSB 在前,不改变位流指针。如果剩余的二进制位少于 *n*,则返回 0。

5.9.2.4 byte_aligned_next_bits(*n*)

如果位流当前位置不是字节对齐的,返回位流当前位置的下一个字节开始的 *n* 个二进制位,MSB 在前,不改变位流指针;如果位流当前位置是字节对齐的,返回位流随后的 *n* 个二进制位,MSB 在前,不改变位流指针。如果剩余的二进制位少于 *n*,则返回 0。

5.9.2.5 next_start_code()

在位流中寻找下一个起始码,将位流指针指向起始码前缀的第一个二进制位。函数定义见表 8。

表 8 next_start_code 函数的定义

函数定义	描述符
next_start_code() {	
stuffing_bit	'1'
while (! byte_aligned())	
stuffing_bit	'0'
while (next_bits(24) != '0000 0000 0000 0000 0000 0001')	
stuffing_byte	'00000000'
}	

stuffing_byte 应出现图像头之后和第一个条带起始码之前。

5.9.2.6 is_end_of_slice()

在位流中检测是否已达到条带的结尾,如果已到达条带的结尾,返回 TRUE,否则返回 FALSE。此函数不修改位流指针。函数定义见表 9。

表 9 is_end_of_slice 函数的定义

函数定义	描述符
is_end_of_slice () {	
if (byte_aligned ()) {	
if (next_bits(32) == 0x80000001)	
return TRUE; // 条带结束	
}	
else {	
if ((byte_aligned_next_bits(24) == 0x000001) && is_stuffing_pattern())	
return TRUE; // 条带结束	
}	
return FALSE;	
}	

5.9.2.7 is_stuffing_pattern()

在位流中检测当前字节中剩下的位或在字节对齐时下一个字节是否是条带结尾填充的二进制位,如果是,则返回 TRUE,否则返回 FALSE。此函数不修改位流指针。函数定义见表 10。

表 10 is_stuffing_pattern 函数的定义

函数定义	描述符
is_stuffing_pattern () {	
if (next_bits(8-n) == (1<<(7-n))) // n:0~7,为位流指针在当前字节的位置偏移,n 为 0 时位流指针指向当前字节最高位	
return TRUE;	
else	
return FALSE;	
}	

5.9.2.8 read_bits(*n*)

返回位流的随后 *n* 个二进制位,MSB 在前,同时位流指针前移 *n* 个二进制位。如果 *n* 等于“0”,则返回 0,位流指针不前移。

函数也用于解析过程和解码过程的描述。

5.9.3 描述符

描述符表示不同语法元素的解析过程,见表 11。

表 11 描述符

描述符	说 明
ae(<i>v</i>)	高级熵编码的语法元素。解析过程在 8.4 中定义
b(8)	一个任意取值的字节。解析过程由函数 read_bits(8)的返回值规定
ce(<i>v</i>)	变长编码的语法元素。解析过程在 8.3 中定义
f(<i>n</i>)	取特定值的连续 <i>n</i> 个二进制位。解析过程由函数 read_bits(<i>n</i>)的返回值规定
i(<i>n</i>)	<i>n</i> 位整数。在语法表中,如果 <i>n</i> 是“ <i>v</i> ”,其位数由其他语法元素值确定。解析过程由函数 read_bits(<i>n</i>)的返回值规定,该返回值用高位在前的 2 的补码表示
me(<i>v</i>)	用指数哥伦布码编码的语法元素。解析过程在 8.2 中定义
r(<i>n</i>)	连续 <i>n</i> 个“0”。解析过程由函数 read_bits(<i>n</i>)的返回值规定
se(<i>v</i>)	有符号整数语法元素,用指数哥伦布码编码。解析过程在 8.2 中定义
u(<i>n</i>)	<i>n</i> 位无符号整数。在语法表中,如果 <i>n</i> 是“ <i>v</i> ”,其位数由其他语法元素值确定。解析过程由函数 read_bits(<i>n</i>)的返回值规定,该返回值用高位在前的二进制表示
ue(<i>v</i>)	无符号整数语法元素,用指数哥伦布码编码。解析过程在 8.2 中定义

5.9.4 保留、禁止和标记

本部分定义的位流语法中,某些语法元素的值被标注为“保留”(reserved)或“禁止”(forbidden)。

“保留”定义了一些特定语法元素值用于将来对本部分的扩展。这些值不应出现在符合本部分的位流中。

“禁止”定义了一些特定语法元素值,这些值不应出现在符合本部分的位流中。

“标记”(marker_bit)指该位的值应为“1”。

位流中的“保留位”(reserved_bits)表明保留了一些语法单元用于将来对本部分的扩展,解码处理应忽略这些位。“保留位”不应出现从任意字节对齐位置开始的 21 个以上连续的“0”。

6 编码位流的结构

6.1 视频序列

6.1.1 概述

视频序列是位流的最高层语法结构。视频序列由序列头开始,后面跟着一个或多个编码图像,每帧图像之前应有图像头。编码图像在位流中按位流顺序排列,位流顺序应与解码顺序相同。解码顺序可与显示顺序不相同。序列结束码表明了一个视频序列的结束。

6.1.2 逐行和隔行视频序列

本部分支持两种序列:逐行序列和隔行序列。

帧由三个样本矩阵构成,包括一个亮度样本矩阵(Y)和两个色度样本矩阵(Cb 和 Cr)。样本矩阵元素的值为整数。Y、Cb 和 Cr 三个分量与原始的(模拟)红、绿和蓝色信号之间的关系,包括原始信号的色度和转移特性等可在位流中定义,这些信息不影响解码过程。

场由构成帧的三个样本矩阵中相间的行构成,即帧样本矩阵的第一行、第三行、第五行等奇数行构成一个场,称为顶场;第二行、第四行、第六行等偶数行构成另一个场,称为底场。

解码器的输出是一系列帧或场,两帧之间存在着一个帧时间间隔。对隔行序列而言,每帧图像的两场之间存在着一个场时间间隔。对逐行序列而言,每帧图像的两场之间时间间隔为 0。

6.1.3 序列头

视频序列头由视频序列起始码开始,后面跟着一串编码图像数据。

序列头可在位流中重复出现,称为重复序列头。使用重复序列头的主要目的是支持对视频序列的随机访问。

序列头后的第一个编码图像应是 I 帧。序列头后出现的 P 帧只能参考该序列头后出现的图像。在对位流进行编辑或随机访问的情况下,重复序列头之前的全部数据可被丢弃,这样得到的一个新的位流仍应符合本部分。

6.2 图像

6.2.1 概述

一幅图像是一帧,其编码数据由图像起始码开始,到序列起始码、序列结束码或下一个图像起始码结束。

在位流中,隔行扫描图像的两场的编码数据可依次出现,也可合并成帧。两场数据的解码和显示顺序在图像头中规定。

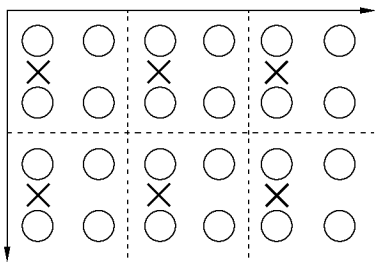
图像的解码处理包括解析过程和解码过程。

6.2.2 图像格式

6.2.2.1 4:2:0 格式

对于 4:2:0 格式,Cb 和 Cr 矩阵水平和垂直方向的尺寸都只有 Y 矩阵的一半。Y 矩阵的行数和

每行样本数都应是偶数。另外,如果图像两场的编码数据依次出现,则 Y 矩阵的行数还应能被 4 整除。
亮度和色度样本位置见图 1。



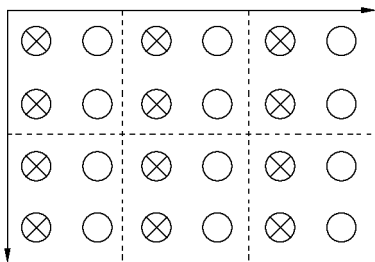
说明:

- 代表亮度样本;
- × 代表色度样本。

图 1 4 : 2 : 0 格式下亮度和色度样本位置

6.2.2.2 4 : 2 : 2 格式

对于 4 : 2 : 2 格式,Cb 和 Cr 矩阵在水平方向的尺寸只有 Y 矩阵的一半,在垂直方向的尺寸和 Y 相同。Y 矩阵的每行样本数应是偶数。如果图像两场的编码数据依次出现,则 Y 矩阵的行数也应是偶数。
亮度和色度样本位置见图 2。



说明:

- 代表亮度样本;
- × 代表色度样本。

图 2 4 : 2 : 2 格式下亮度和色度样本位置

6.2.3 图像类型

- 本部分定义了 3 种解码图像:
- 帧内解码图像(I 帧);
 - 前向帧间解码图像(P 帧);
 - 双向帧间解码图像(B 帧)。

6.2.4 图像间的顺序

如果视频序列中没有 B 帧,解码顺序与显示顺序相同。如果视频序列中包含 B 帧,解码顺序与显示顺序不同,解码图像输出显示前应进行图像重排序。图像重排序规则如下:

- 当前解码图像是 B 帧,输出由此 B 帧解码的图像;

- 当前解码图像是 I 帧或 P 帧,如果存在前一个 I 帧或 P 帧的解码图像,输出前一个 I 帧或 P 帧的解码图像。如果不存在前一个 I 帧或 P 帧的解码图像,不输出任何解码图像;
- 位流解码完成后,如果缓冲区中还有未输出的解码图像,则输出该图像。

下面举例说明图像重排序:I 帧和 P 帧之间有两个 B 帧,两个连续的 P 帧之间也有两个 B 帧。用图像 1I 预测图像 4P,用图像 4P 和 1I 预测图像 2B 和 3B。解码顺序是 1I,4P,2B,3B;显示顺序是 1I,2B,3B,4P。

示例:

编码器输入顺序:

1	2	3	4	5	6	7	8	9	10	11	12	13
I	B	B	P	B	B	P	B	B	I	B	B	P

解码顺序:

1	4	2	3	7	5	6	10	8	9	13	11	12
I	P	B	B	P	B	B	I	B	B	P	B	B

解码器输出,即显示顺序:

1	2	3	4	5	6	7	8	9	10	11	12	13
I	B	B	P	B	B	P	B	B	I	B	B	P

6.2.5 参考图像

P 帧或 B 帧最多可有两帧参考图像。P 帧可参考前向的两帧。除 B 帧外,在一帧中,后解码的场还可参考当前帧的另外一场。B 帧可参考一前一后的两帧。

运动矢量所指的参考像素可超出参考图像的边界,在这种情况下对超出参考图像边界的整数样本应使用距离该整数参考样本所指位置最近的图像内的整数样本进行边界扩展。对亮度样本矩阵,参考块的像素在水平和垂直方向均不应超出参考图像边界外 16 个像素。对色度样本矩阵:

- 如果图像格式是 4 : 2 : 0,参考块的像素在水平和垂直方向均不应超出参考图像边界外 8 个像素。
- 如果图像格式是 4 : 2 : 2,参考块的像素在水平方向不应超出参考图像边界外 8 个像素,在垂直方向不应超出参考图像边界外 16 个像素。

场边界扩展方法和参考图像边界扩展方法相同。

6.3 条带

条带是按光栅扫描顺序连续排列的若干宏块行,条带内的宏块不应重叠,条带之间也不应重叠。如果当前图像的两场数据合并成帧,条带内宏块的解码处理不应使用本图像其他条带的数据。如果当前图像的两场数据依次出现,这两场数据应属于不同的条带,条带内宏块的解码处理不应使用同一场其他条带的数据。

条带结构见图 3。条带边界扩展方法和参考图像边界扩展方法相同。

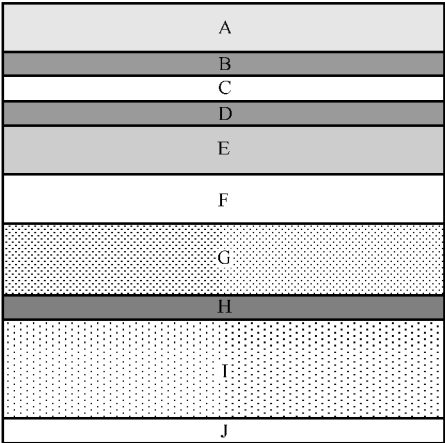


图 3 条带结构

6.4 宏块

图像划分为宏块,宏块左上角的点不应超出图像边界。在位流中,当隔行扫描图像的两场编码数据依次出现时,任一宏块的像素应来自同一场。

宏块的划分见图 4,这种划分用于运动补偿。图 4 中矩形里的数字表示宏块划分后运动矢量和参考索引在位流中的顺序。

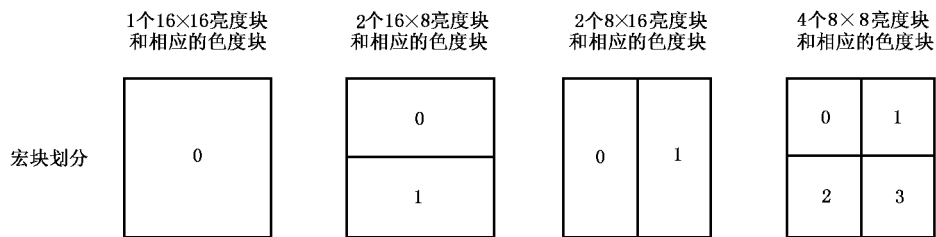


图 4 宏块的划分

6.5 8×8 块

在 4 : 2 : 0 格式下,一个宏块包括 4 个 8×8 亮度块(Y)和 2 个 8×8 色度块(1 个 Cb,1 个 Cr),见图 5。图中数字为宏块中 8×8 块的顺序号。

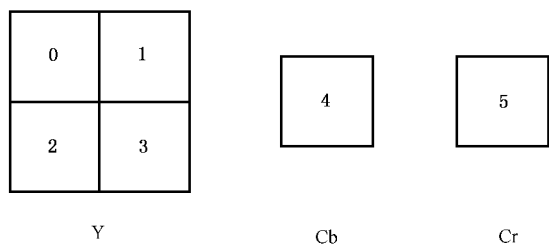


图 5 宏块划分为 8×8 块(4 : 2 : 0 格式)

在 4 : 2 : 2 格式下,一个宏块包括 4 个 8×8 亮度块(Y)和 4 个 8×8 色度块(2 个 Cb,2 个 Cr),见图 6。图中数字为宏块中 8×8 块的顺序号。

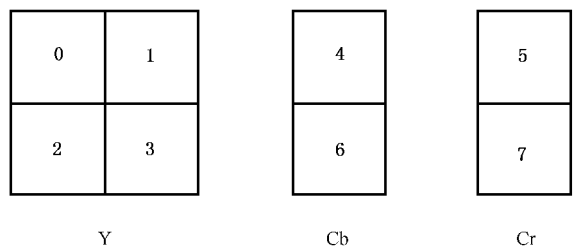


图 6 宏块划分为 8×8 块(4 : 2 : 2 格式)

宏块中的各个 8×8 块在位流中出现的顺序由图 5 到图 6 中的数字规定。

7 位流的语法和语义

7.1 语法描述

7.1.1 起始码

起始码是一组特定的位串。在符合本部分的位流中,除起始码外的任何情况下都不应出现这些位串。

起始码由起始码前缀和起始码值构成。起始码前缀是位串“0000 0000 0000 0000 0000 0001”。所有的起始码都应字节对齐。

起始码值是一个 8 位整数,用来表示起始码的类型,见表 12。

表 12 起始码值

起始码类型	起始码值(十六进制)
条带起始码(slice_start_code)	00~AF
视频序列起始码(video_sequence_start_code)	B0
视频序列结束码(video_sequence_end_code)	B1
用户数据起始码(user_data_start_code)	B2
I 图像起始码(i_picture_start_code)	B3
保留	B4
视频扩展起始码(extension_start_code)	B5
PB 图像起始码(pb_picture_start_code)	B6
视频编辑码(video_edit_code)	B7
保留	B8
系统起始码	B9~FF

部分语法元素取特定值时可得到与起始码前缀相同的位串,称为伪起始码。符合本部分的编码器和解码器应使用附录 A 定义的方法处理伪起始码问题。

7.1.2 视频序列语法

7.1.2.1 视频序列定义

视频序列定义见表 13。

表 13 视频序列定义

视频序列定义	描述符
video_sequence() {	
do {	

表 13 (续)

视频序列定义	描述符
sequence_header()	
extension_and_user_data(0)	
do {	
if (next_bits(32) == i_picture_start_code)	
i_picture_header()	
Else	
pb_picture_header()	
extension_and_user_data(1)	
picture_data()	
} while ((next_bits(32) == pb_picture_start_code) (next_bits(32) == i_picture_start_code))	
} while (next_bits(32) != video_sequence_end_code && next_bits(32) != video_edit_code)	
if (next_bits(32) == video_sequence_end_code)	
video_sequence_end_code	f(32)
if (next_bits(32) == video_edit_code)	
video_edit_code	f(32)
}	

7.1.2.2 序列头定义

序列头定义见表 14。

表 14 序列头定义

序列头定义	描述符
sequence_header() {	
video_sequence_start_code	f(32)
profile_id	u(8)
level_id	u(8)
progressive_sequence	u(1)
horizontal_size	u(14)
vertical_size	u(14)
chroma_format	u(2)
sample_precision	u(3)
aspect_ratio	u(4)
frame_rate_code	u(4)
bit_rate_lower	u(18)

表 14（续）

序列头定义	描述符
marker_bit	f(1)
bit_rate_upper	u(12)
low_delay	u(1)
marker_bit	f(1)
bbv_buffer_size	u(18)
reserved_bits	r(3)
next_start_code()	
}	

7.1.2.3 扩展和用户数据定义

扩展和用户数据定义见表 15。

表 15 扩展和用户数据定义

扩展和用户数据定义	描述符
extension_and_user_data(i) {	
while ((next_bits(32) == extension_start_code) (next_bits(32) == user_data_start_code)) {	
if (next_bits(32) == extension_start_code)	
extension_data(i)	
if (next_bits(32) == user_data_start_code)	
user_data()	
}	
}	

扩展数据定义见表 16，用户数据定义见表 17。

表 16 扩展数据定义

扩展数据定义	描述符
extension_data(i) {	
while (next_bits(32) == extension_start_code) {	
extension_start_code	f(32)
if (i == 0) { /* 序列头之后 */	
if (next_bits(4) == '0010') /* 序列显示扩展 */	
sequence_display_extension()	
else if (next_bits(4) == '0100') /* 版权扩展 */	
copyright_extension()	

表 16（续）

扩展数据定义	描述符
else if (next_bits(4) == '1011')	/* 摄像机参数扩展 */
camera_parameters_extension()	
else	
while (next_bits(24) != '0000 0000 0000 0000 0000 0001')	
reserved_extension_data_byte	u(8)
}	
}	
else { /* 图像头之后 */	
if (next_bits(4) == '0100')	/* 版权扩展 */
copyright_extension()	
else if (next_bits(4) == '0111')	/* 图像显示扩展 */
picture_display_extension()	
else if (next_bits(4) == '1011')	/* 摄像机参数扩展 */
camera_parameters_extension()	
else {	
while (next_bits(24) != '0000 0000 0000 0000 0000 0001')	
reserved_extension_data_byte	u(8)
}	
}	
}	
}	

表 17 用户数据定义

用户数据定义	描述符
user_data() {	
user_data_start_code	f(32)
while (next_bits(24) != '0000 0000 0000 0000 0000 0001') {	
user_data	b(8)
}	
}	

7.1.2.4 序列显示扩展定义

序列显示扩展定义见表 18。

表 18 序列显示扩展定义

序列显示扩展定义	描述符
sequence_display_extension() {	
extension_id	f(4)
video_format	u(3)
sample_range	u(1)
colour_description	u(1)
if (colour_description) {	
colour_primaries	u(8)
transfer_characteristics	u(8)
matrix_coefficients	u(8)
}	
display_horizontal_size	u(14)
marker_bit	f(1)
display_vertical_size	u(14)
stereo_packing_mode	u(2)
next_start_code()	
}	

7.1.2.5 版权扩展定义

版权扩展定义见表 19。

表 19 版权扩展定义

版权扩展定义	描述符
copyright_extension() {	
extension_id	f(4)
copyright_flag	u(1)
copyright_id	u(8)
original_or_copy	u(1)
reserved_bits	r(7)
marker_bit	f(1)
copyright_number_1	u(20)
marker_bit	f(1)
copyright_number_2	u(22)
marker_bit	f(1)
copyright_number_3	u(22)
next_start_code()	
}	

7.1.2.6 摄像机参数扩展定义

摄像机参数扩展定义见表 20。

表 20 摄像机参数扩展定义

摄像机参数扩展定义	描述符
camera_parameters_extension() {	
extension_id	f(4)
reserved_bits	r(1)
camera_id	u(7)
marker_bit	f(1)
height_of_image_device	u(22)
marker_bit	f(1)
focal_length	u(22)
marker_bit	f(1)
f_number	u(22)
marker_bit	f(1)
vertical_angle_of_view	u(22)
marker_bit	f(1)
camera_position_x_upper	i(16)
marker_bit	f(1)
camera_position_x_lower	i(16)
marker_bit	f(1)
camera_position_y_upper	i(16)
marker_bit	f(1)
camera_position_y_lower	i(16)
marker_bit	f(1)
camera_position_z_upper	i(16)
marker_bit	f(1)
camera_position_z_lower	i(16)
marker_bit	f(1)
camera_direction_x	i(22)
marker_bit	f(1)
camera_direction_y	i(22)
marker_bit	f(1)
camera_direction_z	i(22)
marker_bit	f(1)
image_plane_vertical_x	i(22)

表 20 (续)

摄像机参数扩展定义	描述符
marker_bit	f(1)
image_plane_vertical_y	i(22)
marker_bit	f(1)
image_plane_vertical_z	i(22)
marker_bit	f(1)
reserved_bits	r(32)

7.1.3 图像定义

7.1.3.1 I 图像头定义

I 图像头定义见表 21。

表 21 I 图像头定义

I 图像头定义	描述符
i_picture_header() {	
i_picture_start_code	f(32)
bbv_delay	u(16)
if (profile_id == 0x48) {	
marker_bit	f(1)
bbv_delay_extension	u(7)
}	
time_code_flag	u(1)
if (time_code_flag == '1')	
time_code	u(24)
marker_bit	f(1)
picture_distance	u(8)
if (low_delay == '1')	
bbv_check_times	ue(v)
progressive_frame	u(1)
if (progressive_frame == '0')	
picture_structure	u(1)
top_field_first	u(1)
repeat_first_field	u(1)
fixed_picture_qp	u(1)
picture_qp	u(6)

表 21 (续)

I 图像头定义	描述符
if (progressive_frame == '0' && picture_structure == '0')	
skip_mode_flag	u(1)
}	
reserved_bits	r(4)
loop_filter_disable	u(1)
if (! loop_filter_disable) {	
loop_filter_parameter_flag	u(1)
if (loop_filter_parameter_flag) {	
alpha_c_offset	se(v)
beta_offset	se(v)
}	
}	
if (profile_id == 0x48) {	
weighting_quant_flag	u(1)
if (weighting_quant_flag == '1') {	
reserved_bits	r(1)
chroma_quant_param_disable	u(1)
if (chroma_quant_param_disable == '0') {	
chroma_quant_param_delta_cb	se(v)
chroma_quant_param_delta_cr	se(v)
}	
weighting_quant_param_index	u(2)
weighting_quant_model	u(2)
if (weighting_quant_param_index == '01') {	
for (i=0; i<6; i++)	
weighting_quant_param_delta1[i]	se(v)
}	
if (weighting_quant_param_index == '10') {	
for (i=0; i<6; i++)	
weighting_quant_param_delta2[i]	se(v)
}	
}	
if (profile_id == 0x48) {	
aec_enable	u(1)
}	
next_start_code()	
}	

7.1.3.2 PB 图像头定义

PB 图像头定义见表 22。

表 22 PB 图像头定义

PB 图像头定义	描述符
pb_picture_header() {	
pb_picture_start_code	f(32)
bbv_delay	u(16)
if (profile_id == 0x48) {	
marker_bit	f(1)
bbv_delay_extension	u(7)
}	
picture_coding_type	u(2)
picture_distance	u(8)
if (low_delay == '1')	
bbv_check_times	ue(v)
progressive_frame	u(1)
if (progressive_frame == '0') {	
picture_structure	u(1)
if (picture_structure == '0')	
advanced_pred_mode_disable	u(1)
}	
top_field_first	u(1)
repeat_first_field	u(1)
fixed_picture_qp	u(1)
picture_qp	u(6)
if (!(picture_coding_type == '10' && PictureStructure == 1))	
picture_reference_flag	u(1)
no_forward_reference_flag	u(1)
pb_field_enhanced_flag	u(1)
reserved_bits	r(2)
skip_mode_flag	u(1)
loop_filter_disable	u(1)
if (! loop_filter_disable) {	
loop_filter_parameter_flag	u(1)
if (loop_filter_parameter_flag) {	
alpha_c_offset	se(v)

表 22 (续)

PB 图像头定义	描述符
beta_offset	se(v)
}	
}	
if (profile_id==0x48) {	
weighting_quant_flag	u(1)
if (weighting_quant_flag=='1') {	
reserved_bits	r(1)
chroma_quant_param_disable	u(1)
if (chroma_quant_param_disable=='0') {	
chroma_quant_param_delta_cb	se(v)
chroma_quant_param_delta_cr	se(v)
}	
weighting_quant_param_index	u(2)
weighting_quant_model	u(2)
if (weighting_quant_param_index=='01') {	
for (i=0;i<6;i++)	
weighting_quant_param_delta1[i]	se(v)
}	
if (weighting_quant_param_index=='10') {	
for (i=0;i<6;i++)	
weighting_quant_param_delta2[i]	se(v)
}	
}	
}	
if (profile_id==0x48) {	
aec_enable	u(1)
}	
next_start_code()	
}	

7.1.3.3 图像显示扩展定义

图像显示扩展定义见表 23。

表 23 图像显示扩展定义

图像显示扩展定义	描述符
picture_display_extension() {	
extension_id	f(4)
for (i=0; i < NumberOfFrameCentreOffsets; i++) {	
frame_centre_horizontal_offset	i(16)
marker_bit	f(1)
frame_centre_vertical_offset	i(16)
marker_bit	f(1)
}	
next_start_code()	
}	

7.1.3.4 图像数据定义

图像数据定义见表 24。

表 24 图像数据定义

图像数据定义	描述符
picture_data() {	
do {	
slice()	
} while (next_bits(32) == slice_start_code)	
}	

7.1.3.5 条带定义

条带定义见表 25。

表 25 条带定义

条带定义	描述符
slice() {	
slice_start_code	f(32)
if (vertical_size > 2800)	
slice_vertical_position_extension	u(3)
if (fixed_picture_qp == '0') {	
fixed_slice_qp	u(1)
slice_qp	u(6)

表 25 (续)

条带定义	描述符
}	
if (PictureType != 0 (PictureStructure == 0 && MbIndex >= MbWidth × MbHeight / 2)) {	
slice_weighting_flag	u(1)
if (slice_weighting_flag == '1') {	
for (i=0; i<NumberOfReference; i++) {	
luma_scale	u(8)
luma_shift	i(8)
marker_bit	f(1)
chroma_scale	u(8)
chroma_shift	i(8)
marker_bit	f(1)
}	
mb_weighting_flag	u(1)
}	
}	
if (aec_enable == '1') {	
while (! byte_aligned())	
aec_byte_alignment_bit	f(1)
}	
do {	
if (PictureType != 0 (PictureStructure == 0 && MbIndex >= MbWidth × MbHeight / 2)) {	
if (skip_mode_flag == '1')	
mb_skip_run	ue(v) ae(v)
if (aec_enable == '1' && SkipMbCount != 0)	
aec_mb_stuffing_bit	ae(v)
}	
if (! is_end_of_slice()) {	
macroblock()	
if (aec_enable == '1')	
aec_mb_stuffing_bit	ae(v)
}	
} while (! is_end_of_slice())	
next_start_code()	
}	

7.1.3.6 宏块定义

宏块定义见表 26。

表 26 宏块定义

宏块定义	描述符
macroblock() {	
if (PictureType != 0 (PictureStructure == 0 && MbIndex >= MbWidth × MbHeight/2))	
mb_type	ue(v) ae(v)
if (MbType != 'P_Skip' && MbType != 'B_Skip') {	
if (MbType == 'B_8x8') {	
for (i=0; i<4; i++)	
mb_part_type	u(2) ae(v)
}	
if (MbType == 'I_8x8') {	
for (i=0; i<4; i++) {	
if (aec_enable == '0')	
pred_mode_flag	u(1)
if (! pred_mode_flag aec_enable == '1')	
intra_luma_pred_mode	u(2) ae(v)
}	
if (chroma_format != '00')	
intra_chroma_pred_mode	ue(v) ae(v)
if (chroma_format == '10')	
intra_chroma_pred_mode_422	ue(v) ae(v)
}	
if ((PictureType == 1 (PictureType == 2 && PictureStructure == 0)) && picture_reference_flag == '0') {	
for (i=0; i < MvNum; i++)	
mb_reference_index	u(1)/u(2) ae(v)
}	
for (i=0; i < MvNum; i++) {	
mv_diff_x	se(v) ae(v)
mv_diff_y	se(v) ae(v)
}	
if (MbWeightingFlag == 1 && MbType != 'I_8x8')	
weighting_prediction	u(1) ae(v)

表 26 (续)

宏块定义	描述符
if (aec_enable== '1')	
cbp	ae(v)
else if (!(MbTypeIndex>=24 && PictureType==2) (MbTypeIndex>=5 && (PictureType==0 PictureType==1)))	
cbp	me(v)
if (chroma_format== '10')	
cbp_422	me(v) ae(v)
if ((MbCBP > 0 (MbCBP422 > 0 && chroma_format== '10')) && ! FixedQP)	
mb_qp_delta	se(v) ae(v)
for (i=0;i < 6;i++)	
block(i)	
if (chroma_format== '10') {	
for (i=6;i < 8;i++)	
block(i)	
}	
}	
}	

7.1.3.7 块定义

块定义见表 27。

表 27 块定义

块定义	描述符
block(i) {	
if ((i < 6 && (MbCBP & (1 << i))) (i>=6 && (MbCBP422 & (1 << (i-6))))) {	
do {	
trans_coefficient	ce(v) ae(v)
if (trans_coefficient >=59 && aec_enable== '0')	
escape_level_diff	ce(v)
} while (trans_coefficient != 'EOB')	
}	
}	

7.2 语义描述

7.2.1 视频扩展

本条定义了若干视频扩展,在语法的不同位置,可出现的视频扩展是不同的。每一种视频扩展都有一个唯一的视频扩展标号,见表 28。

表 28 视频扩展标号

视频扩展标号	含义
0000	保留
0001	保留
0010	序列显示扩展
0011	保留
0100	版权扩展
0101	保留
0110	保留
0111	图像显示扩展
1000 ~ 1010	保留
1011	摄像机参数扩展
1100 ~ 1111	保留

7.2.2 视频序列语义描述

7.2.2.1 视频序列

7.2.2.1.1 视频编辑码 **video_edit_code**

位串“0x000001B7”。说明紧跟 video_edit_code 的第一个 I 帧后续的 B 帧可能因缺少参考帧,不能正确解码。

7.2.2.1.2 视频序列结束码 **video_sequence_end_code**

位串“0x000001B1”。标识视频序列的结束。

7.2.2.2 序列头

7.2.2.2.1 视频序列起始码 **video_sequence_start_code**

位串“0x000001B0”。标识视频序列的开始。

7.2.2.2.2 档次标号 **profile_id**

8 位无符号整数。表示位流符合的档次。

7.2.2.2.3 级别标号 **level_id**

8 位无符号整数。表示位流符合的级别。

档次和级别见附录 B。

7.2.2.2.4 逐行序列 progressive_sequence

标志。规定视频序列的扫描格式。值为“1”表示编码视频序列只包含逐行扫描的帧图像；值为“0”表示编码视频序列可包含逐行扫描和隔行扫描帧图像。

7.2.2.2.5 水平尺寸 horizontal_size

14 位无符号整数。规定图像亮度分量可显示区域(该区域与图像的左侧边缘对齐)的宽度,即水平方向样本数。

以宏块为单位的 MbWidth 计算如下

$$\text{MbWidth} = (\text{horizontal_size} + 15) / 16$$

horizontal_size 不应为“0”。horizontal_size 的单位应是图像每行样本数。可显示区域的左上角样本应与解码图像左上角样本对齐。

7.2.2.2.6 垂直尺寸 vertical_size

14 位无符号整数。规定图像亮度分量可显示区域(该区域与图像的顶部边缘对齐)的高度,即垂直方向扫描行数。

在视频序列位流中,可能存在隔行扫描图像的两场编码数据依次出现时(即 progressive_sequence 的值为“0”时),以宏块为单位计算的 MbHeight 计算如下:

$$\text{MbHeight} = 2 \times ((\text{vertical_size} + 31) / 32)$$

在其他情况下,以宏块为单位计算的 MbHeight 计算如下:

$$\text{MbHeight} = (\text{vertical_size} + 15) / 16$$

vertical_size 不应为 0。vertical_size 的单位应是图像样本的行数。

horizontal_size、vertical_size 与图像边界的关系见图 7。图 7 中,实线表示图像可显示区域边界,其宽度和高度分别由 horizontal_size 和 vertical_size 决定;虚线表示图像边界,其宽度和高度分别由 MbWidth 和 MbHeight 决定。例如 horizontal_size 的值为 1 920,vertical_size 的值为 1 080,则 MbWidth×16 等于 1 920,MbHeight×16 等于 1 088。

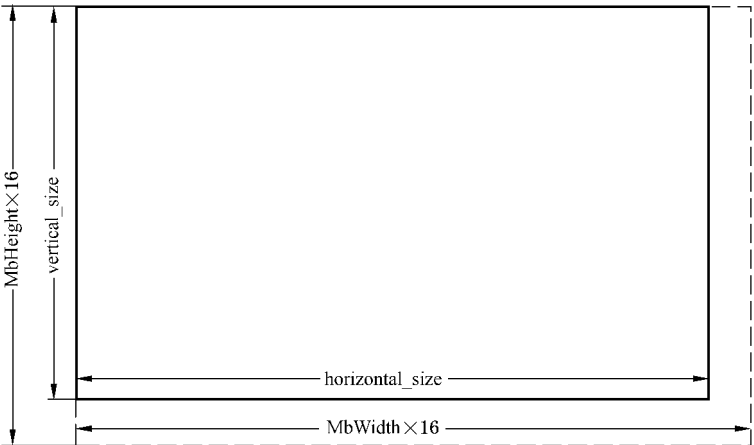


图 7 图像边界示意图

7.2.2.2.7 色度格式 chroma_format

2 位无符号整数。规定色度分量的格式,见表 29。

表 29 色度格式

chroma_format 的值	含义
00	保留
01	4 : 2 : 0
10	4 : 2 : 2
11	保留

7.2.2.2.8 样本精度 sample_precision

3 位无符号整数。规定亮度和色度样本的精度,见表 30。

表 30 样本精度

sample_precision 的值	含义
000	禁止
001	亮度和色度均为 8 位精度
010~111	保留

7.2.2.2.9 宽高比 aspect_ratio

4 位无符号整数。规定重建图像的 SAR 或 DAR,见表 31。

表 31 宽高比

aspect_ratio 的值	SAR	DAR
0000	禁止	禁止
0001	1.0	—
0010	—	4 : 3
0011	—	16 : 9
0100	—	2.21 : 1
0101~1111	—	保留

如果位流中没有序列显示扩展,那么整个重建图像将要映射到整个活动显示区域。样本宽高比按下式计算:

$$SAR = DAR \times vertical_size \div horizontal_size$$

注: 在这种情况下, horizontal_size 和 vertical_size 受源图像的样本宽高比和选定的显示宽高比限制。

如果位流中有序列显示扩展出现,样本宽高比按下式计算:

$$SAR = DAR \times display_vertical_size \div display_horizontal_size$$

7.2.2.2.10 帧率代码 **frame_rate_code**

4 位无符号整数。规定帧率,见表 32。

表 32 帧率代码

frame_rate_code 的值	帧率
0000	禁止
0001	$24\ 000 \div 1\ 001(23.976\dots)$
0010	24
0011	25
0100	$30\ 000 \div 1\ 001(29.97\dots)$
0101	30
0110	50
0111	$60\ 000 \div 1\ 001(59.94\dots)$
1000	60
1001~1111	保留

连续两帧之间的时间间隔是帧率的倒数。隔行扫描帧中两场之间的时间间隔是帧率的倒数的 1/2。

7.2.2.2.11 比特率低位 **bit_rate_lower**

BitRate 的低 18 位。

7.2.2.2.12 比特率高位 **bit_rate_upper**

BitRate 的高 12 位。

$\text{BitRate} = (\text{bit_rate_upper} \ll 18) + \text{bit_rate_lower}$

BitRate 以 400 bit/s 为单位计算视频位流的比特率,并向上取整。BitRate 不应为 0。

7.2.2.2.13 低延迟 **low_delay**

标志。值为“1”说明视频序列不包含 B 帧,不存在图像重排序延时,位流中可能包含所谓“大图像”(见附录 C);值为“0”说明视频序列可包含 B 帧,存在图像重排序延时,位流中不包含所谓“大图像”(见附录 C)。

7.2.2.2.14 位流缓冲区大小 **bbv_buffer_size**

18 位无符号整数。规定了位流参考解码器对视频序列解码的位流缓冲区大小(见附录 C)。BBS 是位流参考解码器对视频序列解码所需的位流缓冲区最小值(按位计算),其定义如下:

$$\text{BBS} = 16 \times 1\ 024 \times \text{bbv_buffer_size}$$

7.2.2.3 扩展和用户数据

7.2.2.3.1 扩展数据

7.2.2.3.1.1 视频扩展起始码 **extension_start_code**

位串“0x000001B5”。标识视频扩展数据的开始。

7.2.2.3.1.2 视频扩展数据保留字节 reserved_extension_data_byte

8 位无符号整数。保留位。解码器应丢弃这些数据。视频扩展数据保留字节中不应出现从任意字节对齐位置开始的 21 个以上连续的“0”。

7.2.2.3.2 用户数据

7.2.2.3.2.1 用户数据起始码 user_data_start_code

位串“0x000001B2”。标识用户数据的开始。用户数据连续存放,直到下一个起始码。

7.2.2.3.2.2 用户数据字节 user_data

8 位整数。用户数据的含义由用户自行定义。用户数据中不应出现从任意字节对齐位置开始的 21 个以上连续的“0”。

7.2.2.4 序列显示扩展

7.2.2.4.1 视频扩展标号 extension_id

位串“0010”。标识序列显示扩展。

7.2.2.4.2 视频格式 video_format

3 位无符号整数。说明视频在按本部分进行编码之前的格式,见表 33。如果位流中没有出现序列显示扩展,可假设视频格式为“未作规定的视频格式”。

表 33 视频格式

video_format 的值	含义
000	分量信号
001	PAL
010	NTSC
011	SECAM
100	MAC
101	未作规定的视频格式
110	保留
111	保留

7.2.2.4.3 样值范围 sample_range

标志。说明亮度和色度信号样值的范围。如果位流中没有出现序列显示扩展,设 sample_range 为“0”。

7.2.2.4.4 彩色信息描述 colour_description

标志。值为“1”说明位流中有 colour_primaries、transfer_characteristics 和 matrix_coefficients;值为“0”说明位流中没有 colour_primaries、transfer_characteristics 和 matrix_coefficients。

7.2.2.4.5 彩色三基色 colour_primaries

8 位无符号整数。说明源图像三基色的色度坐标,见表 34。

表 34 彩色三基色

colour_primaries 的值	彩色三基色		
0	禁止		
1	基色	x	y
	绿	0.300	0.600
	蓝	0.150	0.060
	红	0.640	0.330
	白 D65	0.312 7	0.329 0
2	未作规定的视频		
3	保留		
4	基色	x	y
	绿	0.21	0.71
	蓝	0.14	0.08
	红	0.67	0.33
	白 C	0.310	0.316
5	基色	x	y
	绿	0.29	0.60
	蓝	0.15	0.06
	红	0.64	0.33
	白 D65	0.313	0.329
6	基色	x	y
	绿	0.310	0.595
	蓝	0.155	0.070
	红	0.630	0.340
	白 D65	0.312 7	0.329 0
7	基色	x	y
	绿	0.310	0.595
	蓝	0.155	0.070
	红	0.630	0.340
	白 D65	0.312 7	0.329 1
8	普通胶片(彩色滤光镜,C 光源)		
	基色	x	y
	绿	0.243	0.692 (Wratten 58)
	蓝	0.145	0.049 (Wratten 47)
	红	0.681	0.319 (Wratten 25)。
9~255	保留		

如果位流中没有序列显示扩展,或者 colour_description 的值是“0”,假设色度已由应用本身隐含定义。

7.2.2.4.6 光电转移特性 transfer_characteristics

8 位无符号整数。说明源图像的光电转移特性,见表 35。

表 35 光电转移特性

transfer_characteristics 的值	光电转移特性
0	禁止
1	$E' = 1.099 L^{0.45} - 0.099, 1.33 \geq L \geq 0.018$ $E' = 4.500 L, 0.018 > L \geq -0.004 5$ $E' = -\{1.099(-4L)^{0.45} - 0.099\}/4, -0.004 5 > L \geq -0.25$
2	未作规定的视频
3	保留
4	假设显示伽玛值为2.2
5	假设显示伽玛值为2.8
6	$V = 1.099 L_c^{0.45} - 0.099, 1 \geq L_c \geq 0.018$ $V = 4.500 L_c, 0.018 > L_c \geq 0$
7	$V = 1.111 5 L_c^{0.45} - 0.111 5, L_c \geq 0.022 8$ $V = 4.0 L_c, 0.022 8 > L_c$
8	线性转移特性 即 $V = L_c$
9	对数转移特性（范围 100 : 1） $V = 1.0 - (\log_{10}(L_c))/2, 1 \geq L_c \geq 0.01$ $V = 0.0, 0.01 > L_c$
10	对数转移特性（范围 316.227 77 : 1） $V = 1.0 - (\log_{10}(L_c))/2.5, 1 \geq L_c \geq 0.003 162 277 7$ $V = 0.0, 0.003 162 277 7 > L_c$
11~255	保留

7.2.2.4.7 彩色信号转换矩阵 matrix_coefficients

8 位无符号整数。说明从红绿蓝三基色转换为亮度和色度信号时采用的转换矩阵,见表 36。

表 36 彩色信号转换矩阵

matrix_coefficients 的值	彩色信号转换矩阵
0	禁止
1	$E'_Y = 0.212 6 E'_R + 0.715 2 E'_G + 0.072 2 E'_B$ $E'_{CB} = (E'_B - E'_Y)/1.855 6$ $E'_{CR} = (E'_R - E'_Y)/1.574 8$
2	未作规定的视频 图像特性未知
3	保留
4	$E'_Y = 0.59 E'_G + 0.11 E'_B + 0.30 E'_R$ $E'_{PB} = -0.331 E'_G + 0.500 E'_B - 0.169 E'_R$ $E'_{PR} = -0.421 E'_G - 0.079 E'_B + 0.500 E'_R$

表 36 (续)

matrix_coefficients 的值	彩色信号转换矩阵
5	$E'_Y = 0.587 E'_G + 0.114 E'_B + 0.299 E'_R$ $E'_{PB} = -0.331 E'_G + 0.500 E'_B - 0.169 E'_R$ $E'_{PR} = -0.419 E'_G - 0.081 E'_B + 0.500 E'_R$
6	$E'_Y = 0.587 E'_G + 0.114 E'_B + 0.299 E'_R$ $E'_{PB} = -0.331 E'_G + 0.500 E'_B - 0.169 E'_R$ $E'_{PR} = -0.419 E'_G - 0.081 E'_B + 0.500 E'_R$
7	$E'_Y = 0.701 E'_G + 0.087 E'_B + 0.212 E'_R$ $E'_{PB} = -0.384 E'_G + 0.500 E'_B - 0.116 E'_R$ $E'_{PR} = -0.445 E'_G - 0.055 E'_B + 0.500 E'_R$
8~255	保留

在表 36 中：

- E'_Y 是值在“0”和“1”之间的模拟量；
- E'_{PB} 和 E'_{PR} 是值在“−0.5”和“0.5”之间的模拟量；
- E'_R 、 E'_G 和 E'_B 是值在“0”和“1”之间的模拟量；
- Y 、 Cb 和 Cr 与 E'_Y 、 E'_{PB} 和 E'_{PR} 的关系如下：

如果 sample_range 为“0”：

$$Y = (219 \times 2^{n-8} \times E'_Y) + 2^{n-4}$$
$$Cb = (224 \times 2^{n-8} \times E'_{PB}) + 2^{n-1}$$
$$Cr = (224 \times 2^{n-8} \times E'_{PR}) + 2^{n-1}$$

如果 sample_range 为“1”：

$$Y = ((2^n - 1) \times E'_Y)$$
$$Cb = ((2^n - 1) \times E'_{PB}) + 2^{n-1}$$
$$Cr = ((2^n - 1) \times E'_{PR}) + 2^{n-1}$$

其中 n 是样本点精度。例如：

$n=8$, sample_range 为“0”时：

$$Y = (219 \times E'_Y) + 16$$
$$Cb = (224 \times E'_{PB}) + 128$$
$$Cr = (224 \times E'_{PR}) + 128$$

Y 的取值范围是 16~235, Cb 和 Cr 的取值范围是 16~240。

$n=8$, sample_range 为“1”时：

$$Y = (255 \times E'_Y)$$
$$Cb = (255 \times E'_{PB}) + 128$$
$$Cr = (255 \times E'_{PR}) + 128$$

Y 、 Cb 和 Cr 的取值范围都是 0~255。

注 1：本部分规定的解码过程将输出的 Y 、 Cb 和 Cr 的样值范围限制在 $0 \sim 2^n - 1$ 。如果位流中没有出现序列显示扩展，或者 colour_description 的值是“0”，假设转换矩阵已由应用本身隐含定义。

注 2：某些应用可能有多个不同的视频信号，而不同的视频信号又可能具有不同的彩色三基色、转移特性和/或转换矩阵。在这种情况下建议应用首先将这些不同的参数集转换到一个统一的参数集。

7.2.2.4.8 水平显示尺寸 **display_horizontal_size**

7.2.2.4.9 垂直显示尺寸 **display_vertical_size**

display_horizontal_size 和 **display_vertical_size** 都是 14 位无符号整数。它们共同定义了一个矩形,如果该矩形的尺寸比编码图像的尺寸小,宜只显示编码图像的一部分;如果该矩形的尺寸比编码图像的尺寸大,宜只在显示设备的一部分上显示重建图像。

display_horizontal_size 的单位应是编码图像每行样本数。**display_vertical_size** 的单位应是编码图像的行数。

display_horizontal_size 和 **display_vertical_size** 对解码过程没有影响。它们可被显示过程使用。本部分不定义显示过程。

7.2.2.4.10 双视点拼接模式 **stereo_packing_mode**

2 位无符号整数。规定双目立体视频的拼接方式,见表 37。

表 37 双视点拼接模式

stereo_packing_mode 的值	含义
00	无拼接
01	左右拼接
10	上下拼接
11	保留

表 37 中无拼接表示当前视频即为通常的单目视频;左右拼接表示当前视频序列为左右拼接双目立体视频,在解码图像中,左视点的图像的像素均在右视点的图像的像素的左侧;上下拼接表示当前视频序列为上下拼接双目立体视频,在解码图像中,左视点的图像的像素均在右视点的图像的像素的上侧。

7.2.2.5 版权扩展

7.2.2.5.1 视频扩展标号 **extension_id**

位串“0100”。标识版权扩展。

7.2.2.5.2 版权标志 **copyright_flag**

标志。值为“1”说明该版权扩展定义的版权信息有效期直到下一个版权扩展或视频序列结束码;值为“0”说明该版权扩展没有定义版权信息。

版权信息由 **copyright_id** 和 **CopyrightNumber** 进一步说明。

7.2.2.5.3 版权标号 **copyright_id**

8 位无符号整数。版权所有者的代码,由版权注册机构统一分配。如果为“0”,说明没有相关版权信息。

如果 **copyright_id** 的值为“0”,**CopyrightNumber** 应为“0”。

如果 **copyright_flag** 的值为“0”,**copyright_id** 应为“0”。

7.2.2.5.4 原创或拷贝 **original_or_copy**

标志。值为“1”说明源视频的内容是原创的;值为“0”说明源视频的内容是翻录的。

7.2.2.5.5 版权号 1 **copyright_number_1**

20 位无符号整数。CopyrightNumber 的第 44 位~第 63 位。

7.2.2.5.6 版权号 2 **copyright_number_2**

22 位无符号整数。CopyrightNumber 的第 22 位~第 43 位。

7.2.2.5.7 版权号 3 **copyright_number_3**

22 位无符号整数。CopyrightNumber 的第 0 位~第 21 位。

CopyrightNumber 是 64 位无符号整数,定义如下:

$$\text{CopyrightNumber} = (\text{copyright_number_1} \ll 44) + (\text{copyright_number_2} \ll 22) + \text{copyright_number_3}$$

如果 copyright_flag 的值是“1”, CopyrightNumber 和该版权扩展说明的源视频内容一一对应, CopyrightNumber 为“0”说明没有相关信息;如果 copyright_flag 的值是“0”, CopyrightNumber 也应为“0”。

7.2.2.6 摄像机参数扩展

7.2.2.6.1 视频扩展标号 **extension_id**

位串“1011”。标识摄像机参数扩展。

7.2.2.6.2 摄像机标号 **camera_id**

7 位无符号整数。摄像机标识符。

7.2.2.6.3 图像设备高度 **height_of_image_device**

22 位无符号整数。给出图像设备的高度,以 0.001 mm 为单位,范围为 0~4 194.303 mm。

7.2.2.6.4 焦距 **focal_length**

22 位无符号整数。给出摄像机的焦距,以 0.001 mm 为单位,范围为 0~4 194.303 mm。

7.2.2.6.5 光圈 **f_number**

22 位无符号整数。给出摄像机的光圈(光圈=焦距÷(镜头的有效孔径),以 0.001 为单位,范围为 0~4 194.303)。

7.2.2.6.6 垂直视角 **vertical_angle_of_view**

22 位无符号整数。给出由图像设备顶端和底端决定的垂直视角,以 0.000 1°为单位,范围为 0°~180°。

7.2.2.6.7 摄像机坐标 X 高位,摄像机坐标 Y 高位,摄像机坐标 Z 高位 **camera_position_x_upper, camera_position_y_upper, camera_position_z_upper**

分别表示 CameraPositionX, CameraPositionY 和 CameraPositionZ 的高 16 位。

7.2.2.6.8 摄像机坐标 X 低位,摄像机坐标 Y 低位,摄像机坐标 Z 低位 camera_position_x_lower,camera_position_y_lower, camera_position_z_lower

分别表示 CameraPositionX, CameraPositionY 和 CameraPositionZ 的低 16 位。
CameraPositionX, CameraPositionY 和 CameraPositionZ 是一组 32 位整数,用 2 的补码表示。说明摄像机光学原点在由用户定义的全局坐标系中的坐标值,每个坐标值都以 0.001 mm 为单位,范围从 -2 147 483.648 mm~2 147 483.647 mm。

7.2.2.6.9 摄像机方向矢量 X,摄像机方向矢量 Y,摄像机方向矢量 Z camera_direction_x,camera_direction_y,camera_direction_z

一组 22 位整数,用 2 的补码表示。说明摄像机的方向,每个值范围从 -2 097 152~2 097 151。摄像机的方向用从摄像机光学原点到摄像机前面位于摄像机光轴上的某点的矢量表示。

7.2.2.6.10 图像平面垂直矢量 X,图像平面垂直矢量 Y,图像平面垂直矢量 Z image_plane_vertical_x, image_plane_vertical_y, image_plane_vertical_z

一组 22 位整数,用 2 的补码表示。说明摄像机向上的方向,每个值的范围从 -2 097 152~2 097 151。摄像机向上的方向用平行于设备的边缘,方向从底到顶的矢量表示。

摄像机原理如图 8 所示,摄像机坐标系如图 9 所示。

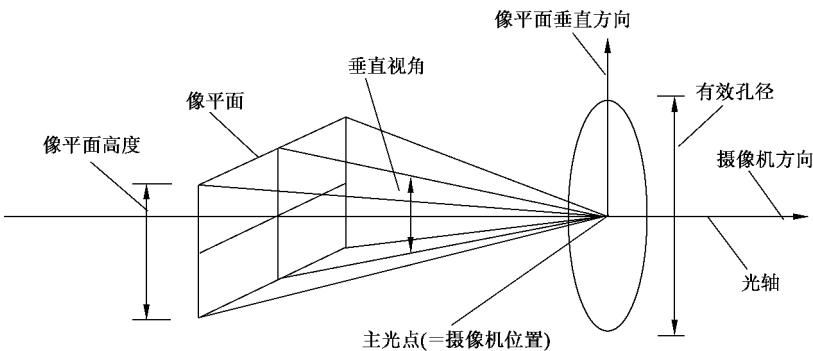


图 8 摄像机原理示意图

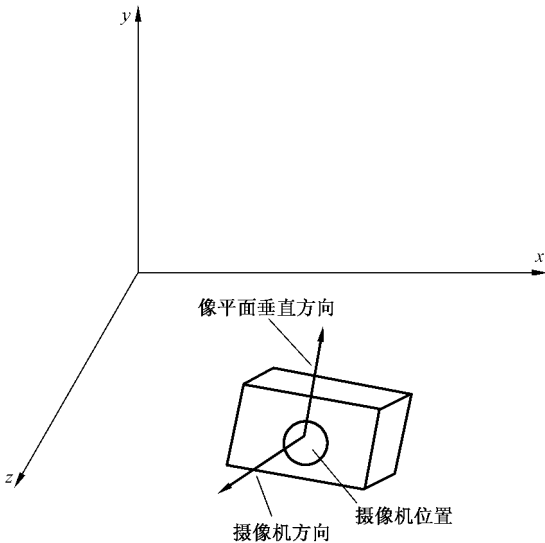


图 9 摄像机坐标系示意图

7.2.3 图像

7.2.3.1 I 图像头

7.2.3.1.1 I 图像起始码 `i_picture_start_code`

位串“0x000001B3”。标识 I 帧或 G 帧的开始。

7.2.3.1.2 BBV 延时 `bbv_delay`

16 位无符号整数。

7.2.3.1.3 BBV 延迟扩展 `bbv_delay_extension`

7 位无符号整数。

如果 `profile_id` 的值等于 0x20, `BbvDelay` 的值等于 `bbv_delay` 的值;否则, `BbvDelay` 的值等于 $(\text{bbv_delay} < 7) + \text{bbv_delay_extension}$ 。

如果 `profile_id` 的值等于 0x20, 则 `BbvDelayMax` 的值等于 0xFFFF;否则, `BbvDelayMax` 的值等于 0x7FFFFFFF。

如果 `BbvDelay` 不等于 `BbvDelayMax`, 它规定了 BBV 从收到图像起始码的最后一个字节到开始解码图像之间要等待的时间。这个时间用从 27 MHz 系统时钟导出的 90 kHz 时钟周期数来表示。如果视频序列中某一帧图像的 `BbvDelay` 等于 `BbvDelayMax`, 那么整个视频序列中所有图像的 `BbvDelay` 均应等于 `BbvDelayMax`。见附录 C。

7.2.3.1.4 时间编码标志 `time_code_flag`

标志。值为“1”表示位流中包含 `time_code`;值为“0”表示位流中没有 `time_code`。

7.2.3.1.5 时间编码 `time_code`

24 位位串, 包括以下字段: `DropFrameFlag`, `TimeCodeHours`, `TimeCodeMinutes`, `TimeCodeSeconds` 和 `TimeCodePictures`, 见表 38。表中 `TimeCodeHours`, `TimeCodeMinutes`, `TimeCodeSeconds` 和 `TimeCodePictures` 字段是无符号整数。这些参数与 IEC 60461 中定义的参数相对应。`time_code` 描述从当前帧开始(含当前帧)的位流中第一帧(显示顺序)的显示时间。

表 38 时间编码(`time_code`)

<code>time_code</code> 的字段	取值	描述符
<code>DropFrameFlag</code>		u(1)
<code>TimeCodeHours</code>	0~23	u(5)
<code>TimeCodeMinutes</code>	0~59	u(6)
<code>TimeCodeSeconds</code>	0~59	u(6)
<code>TimeCodePictures</code>	0~59	u(6)

7.2.3.1.6 图像间距 `picture_distance`

8 位无符号整数。`picture_distance` 等于前一编码帧(显示顺序)的 `picture_distance` 加 1, 再加上当前帧和前一编码帧之间被跳过的图像帧数(视频序列起始码与随后第一个视频序列结束码或视频序列

起始码与随后第一个视频编辑码之间,按显示顺序每两个连续编码帧之间被跳过的帧数均应小于 32,按显示顺序每两个相邻的非双向帧间解码图像之间被跳过的帧数和 B 帧数之和均应小于 127),最后模 256。

7.2.3.1.7 BBV 检测次数 `bbv_check_times`

如果 `low_delay` 的值为“0”,位流中不应出现 `bbv_check_times`,此时 `BbvCheckTimes` 等于“0”。如果位流中出现 `bbv_check_times`,由 `bbv_check_times` 解析得到 `BbvCheckTimes`,解析过程见 8.2。`bbv_check_times` 的值应小于 $2^{16} - 1$ 。

检测 BBV 缓冲区的次数为 `BbvCheckTimes` 加 1 次,`BbvCheckTimes` 大于“0”说明当前图像是一个大图像(见附录 C)。

7.2.3.1.8 逐行帧 `progressive_frame`

标志。值为“0”表示该帧的两场是隔行场,这两场之间存在一个场时间间隔,在这种情况下,`repeat_first_field` 的值应是“0”。

如果 `progressive_frame` 的值是“1”,表示该帧的两场实际上来自同一时刻,在这种情况下,`PictureStructure` 的值应是“1”。

7.2.3.1.9 图像编码结构 `picture_structure`

标志。`picture_structure` 的值为“0”表示当前图像的两场的编码数据依次出现;值为“1”表示当前图像的两场的编码数据合并成帧。如果 `progressive_sequence` 的值为“1”,则 `picture_structure` 的值也应为“1”。`PictureStructure` 的值等于 `picture_structure`。

7.2.3.1.10 顶场在先 `top_field_first`

标志。其含义由 `progressive_sequence`、`progressive_frame`、`picture_structure` 和 `repeat_first_field` 决定。

- a) 如果 `progressive_sequence` 的值是“0”,`top_field_first` 说明解码场的输出显示顺序。
 - 1) 如果 `PictureStructure` 的值是“0”,`top_field_first` 的值是“1”,说明顶场的编码数据首先出现在位流中,顶场在底场之前输出显示;`top_field_first` 的值是“0”,说明底场的编码数据首先出现在位流中,底场在顶场之前输出显示;
 - 2) 如果 `PictureStructure` 的值是“1”,解码处理首先解码整帧。如果 `top_field_first` 的值是“1”则顶场在底场之前输出,`top_field_first` 的值是“0”则底场在顶场之前输出。
- b) 如果 `progressive_sequence` 的值是“1”,`top_field_first` 和 `repeat_first_field` 一起说明解码处理输出帧的次数(1 次、2 次或 3 次)。
 - 1) 如果 `repeat_first_field` 的值是“0”,`top_field_first` 的值也应是“0”,解码处理输出一个帧;
 - 2) 如果 `top_field_first` 的值是“0”,`repeat_first_field` 的值是“1”,解码处理输出两个完全一样的帧;
 - 3) 如果 `top_field_first` 和 `repeat_first_field` 的值都是“1”,解码处理输出三个完全一样的帧。

如果 `profile_id` 的值是 0x20,`progressive_sequence` 值是“0”,并且 `PictureStructure` 的值是“0”,则 `top_field_first` 的值应是“1”。

如果 `profile_id` 的值不是 0x20,`progressive_sequence` 的值是“0”,则视频序列中所有图像的 `top_field_first` 的值应相同。

7.2.3.1.11 重复首场 `repeat_first_field`

标志。只在 `progressive_frame` 的值为“1”时起作用,否则 `repeat_first_field` 的值应为“0”。

- a) 如果 progressive_sequence 和 progressive_frame 的值都是“0”,repeat_first_field 的值也应是“0”,解码处理输出两个场,第一场(由 top_field_first 决定是顶场还是底场),后面跟着第二场。
- b) 如果 progressive_sequence 的值是“0”,progressive_frame 的值是“1”,那么:
 - 1) 如果 repeat_first_field 的值是“0”,解码处理输出两个场,第一场(由 top_field_first 决定是顶场还是底场),后面跟着第二场;
 - 2) 如果 repeat_first_field 的值是“1”,解码处理输出三个场,第一场(由 top_field_first 决定是顶场还是底场),后面跟着第二场,最后重复输出第一场。
- c) 如果 progressive_sequence 的值是“1”,那么:
 - 1) 如果 repeat_first_field 的值是“0”,解码处理输出一帧;
 - 2) 如果 repeat_first_field 的值是“1”,解码处理输出两或三帧,由 top_field_first 决定。

7.2.3.1.12 固定图像量化因子 fixed_picture_qp

标志。值为“1”说明在该帧图像内量化因子不变;“0”说明在该帧图像内量化因子可变。

7.2.3.1.13 图像量化因子 picture_qp

6 位无符号整数。给出图像的量化因子。量化因子取值范围是 0~63。

7.2.3.1.14 宏块跳过模式标志 skip_mode_flag

标志。值为“1”表示宏块跳过模式使用游程编码;值为“0”表示宏块跳过模式的编码由 mb_type 决定,见 9.4.1。

7.2.3.1.15 环路滤波禁用 loop_filter_disable

标志。值为“1”表示不应使用环路滤波;值为“0”表示应使用环路滤波。

7.2.3.1.16 环路滤波参数标志 loop_filter_parameter_flag

标志。值为“1”表示位流中包含 alpha_c_offset 和 beta_offset;值为“0”表示位流中没有 alpha_c_offset 和 beta_offset。

7.2.3.1.17 α 和 C 索引的偏移 alpha_c_offset

当前图像环路滤波 α 和 C 索引的偏移,alpha_c_offset 取值范围是-8~8,环路滤波参数 AlphaCOffset 等于 alpha_c_offset。如果位流中没有 alpha_c_offset,AlphaCOffset 等于“0”。

7.2.3.1.18 β 索引的偏移 beta_offset

当前图像环路滤波 β 索引的偏移,beta_offset 取值范围是-8~8,环路滤波参数 BetaOffset 等于 beta_offset。如果位流中没有 beta_offset,BetaOffset 等于“0”。

7.2.3.1.19 加权量化标志 weighting_quant_flag

标志。值为“1”表示应使用加权量化;值为“0”表示不应使用加权量化。如果当前图像的图像头不存在 weighting_quant_flag,则 weighting_quant_flag 的值为“0”。

7.2.3.1.20 色度量化参数禁用 chroma_quant_param_disable

标志。值为“1”表示当前图像的图像头中不存在 chroma_quant_param_delta_cb 和 chroma_quant_param_delta_cr;值为“0”表示当前图像的图像头中存在 chroma_quant_param_delta_cb 和 chroma_

quant_param_delta_cr。

7.2.3.1.21 色度量化参数增量 Cb chroma_quant_param_delta_cb, 色度量化参数增量 Cr chroma_quant_param_delta_cr

色度块量化参数相对于 CurrentQP 的增量,取值范围-16~16。解析过程见 8.2,解码过程见 9.6.1。如果当前图像的图像头中不存在 chroma_quant_param_delta_cb 和 chroma_quant_param_delta_cr,则 chroma_quant_param_delta_cb 和 chroma_quant_param_delta_cr 的值均为“0”。

7.2.3.1.22 加权量化参数索引 weighting_quant_param_index

2 位无符号整数。当前图像的加权量化参数索引。值为“11”保留。

7.2.3.1.23 加权量化矩阵模型 weighting_quant_model

2 位无符号整数,规定加权量化参数的分布模型。值为“11”保留。

7.2.3.1.24 加权量化参数增量 1 weighting_quant_param_delta1[i], 加权量化参数增量 2 weighting_quant_param_delta2[i]

当前图像的加权量化参数的增量,取值范围是-128~127。解析过程见 8.2。解码过程见 9.2。如果当前图像的图像头中不存在 weighting_quant_param_delta1[i]或 weighting_quant_param_delta2[i],则 weighting_quant_param_delta1[i]或 weighting_quant_param_delta2[i]的值为 0。

7.2.3.1.25 高级熵编码使能 aec_enable

标志。值为“0”表示语法元素使用基本熵编码;值为“1”表示语法元素使用高级熵编码。如果当前图像的图像头中没有 aec_enable,则 aec_enable 的值为“0”。

7.2.3.2 PB 图像头

7.2.3.2.1 PB 图像起始码 pb_picture_start_code

位串“0x000001B6”。标识 P 帧或 B 帧的开始。

7.2.3.2.2 图像编码方式 picture_coding_type

2 位无符号整数。规定图像的编码方式,见表 39。

表 39 图像编码方式

picture_coding_type 的值	编码方式
00	禁止
01	前向预测编码(P)
10	双向预测编码(B)
11	保留

7.2.3.2.3 高级预测模式禁用 advanced_pred_mode_disable

标志。值应为“1”,说明禁止使用高级预测模式。值为“0”保留。

7.2.3.2.4 图像参考标志 picture_reference_flag

标志。值为“1”表示所有宏块都使用缺省参考图像;值为“0”表示由每个宏块在宏块层自行确定参

考图像。确定参考图像的方法见 9.4.5。如果当前图像的图像头中没有 `picture_reference_flag`, 则 `picture_reference_flag` 的值为“1”。

7.2.3.2.5 无前向参考标志 `no_forward_reference_flag`

标志。值为“1”表示当前图像不应参考前向参考图像;值为“0”表示当前图像可参考前向参考图像。

7.2.3.2.6 PB 帧场编码预测标志 `pb_field_enhanced_flag`

标志。如果 `profile_id` 的值等于 0x48, 则 `PFieldSkip` 和 `BFieldEnhanced` 的值等于 `pb_field_enhanced_flag` 的值;否则, `PFieldSkip` 和 `BFieldEnhanced` 的值均应为 0。

7.2.3.3 图像显示扩展

7.2.3.3.1 视频扩展标号 `extension_id`

位串“0111”。标识图像显示扩展。

7.2.3.3.2 帧中心水平偏移 `frame_centre_horizontal_offset`

16 位整数。以 1/16 样本为单位给出水平偏移。正值表示重建图像的中心位置在显示矩形中心的右侧。

7.2.3.3.3 帧中心垂直偏移 `frame_centre_vertical_offset`

16 位整数。以 1/16 样本为单位给出垂直偏移。正值表示重建图像的中心位置在显示矩形中心的下方。

显示矩形区域的尺寸在序列显示扩展中定义。编码图像内区域的坐标由图像显示扩展定义。

重建图像的中心指由 `horizontal_size` 和 `vertical_size` 定义的矩形的中心。

在隔行序列中,一幅编码图像可能与一个、两个或三个解码场有关,因此图像显示扩展最多可以定义三组偏移量。

7.1.3.3 中 `NumberOfFrameCentreOffsets` 的值按以下方式定义:

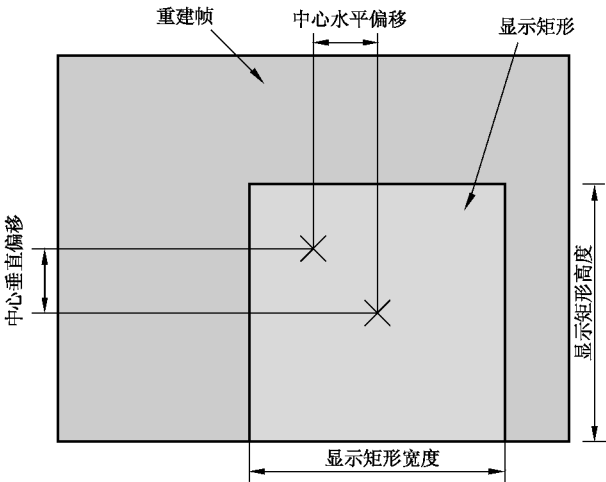
```

if(progressive_sequence=='1'){
if(repeat_first_field=='1'){
    if(top_field_first=='1')
        NumberOfFrameCentreOffsets=3
    else
        NumberOfFrameCentreOffsets=2
} else {
    NumberOfFrameCentreOffsets=1
}
} else {
    if(picture_structure=='0'){
        NumberOfFrameCentreOffsets=1
    } else {
        if(repeat_first_field=='1')
            NumberOfFrameCentreOffsets=3
        else

```

```
NumberOfFrameCentreOffsets=2
}
}
```

如果前面的序列头之后没有序列显示扩展,那么位流中不应出现图像显示扩展。
如果一帧图像没有图像显示扩展,使用最近的解码图像的中心偏移量。
注意: 丢失帧的中心偏移量的值与前一非丢失帧的值相同。在序列头后,所有图像的中心偏移量置为 0,直到出现图像显示扩展。
利用图像中心偏移量可定义一个矩形区域,这个区域在整个重建图像范围内移动来实现全景扫描。图像中心偏移量参数见图 10。



注: 显示矩形的尺寸可能大于重建图像。

图 10 帧中心偏移量参数

7.2.4 条带

7.2.4.1 条带起始码 slice_start_code

32 位位串,前 24 位是 0x000001。后 8 位为 slice_vertical_position,其范围是 0x00~0xAF。

7.2.4.2 条带垂直位置 slice_vertical_position

8 位无符号整数,给出条带的第一个宏块在图像中的垂直位置 MbRow,以宏块为单位。slice_vertical_position 的值的范围应为 0x00~0xAF。

如果图像的 vertical_size 大于 2 800,MbRow 由 slice_vertical_position 和 slice_vertical_position_extension 决定。

7.2.4.3 条带垂直位置扩展 slice_vertical_position_extension

3 位无符号整数,如果图像的 vertical_size 小于或等于 2 800,位流中不应出现 slice_vertical_position_extension。

MbRow 按下面的方法计算:

if(vertical_size>2 800)

MbRow=(slice_vertical_position_extension<<7)+slice_vertical_position

else

$MbRow = slice_vertical_position$

7.2.4.4 固定条带量化因子 **fixed_slice_qp**

标志。值为“1”说明在该条带内量化因子不变;值为“0”说明在该条带内量化因子可变。

7.2.4.5 条带量化因子 **slice_qp**

6 位无符号整数。给出条带的量化因子。量化因子取值范围是 0~63。

7.2.4.6 条带加权预测标志 **slice_weighting_flag**

标志。值为“1”表示宏块的运动补偿可使用加权预测;值为“0”表示宏块的运动补偿不应使用加权预测。

7.2.4.7 亮度缩放参数 **luma_scale**

8 位无符号整数,表示预测亮度时的缩放参数。

7.2.4.8 亮度平移参数 **luma_shift**

8 位有符号整数,表示预测亮度时的平移参数。

7.2.4.9 色度缩放参数 **chroma_scale**

8 位无符号整数,表示预测色度时的缩放参数。

7.2.4.10 色度平移参数 **chroma_shift**

8 位有符号整数,表示预测色度时的平移参数。

7.2.4.11 宏块加权预测标志 **mb_weighting_flag**

标志。**mb_weighting_flag** 的值为“0”表示所有非帧内预测宏块都应采用加权运动补偿;值为“1”表示每个非帧内预测宏块由 **WeightingPrediction** 决定是否采用加权预测。

7.2.4.12 高级熵编码字节对齐填充位 **aec_byte_alignment_bit**

填充位。值应为“1”。

7.2.4.13 跳过宏块计数 **mb_skip_run**

跳过宏块计数。如果 **aec_enable** 的值为“0”,解析过程见 8.2;如果 **aec_enable** 的值为“1”,解析过程见 8.4。解码过程见 9.3。

7.2.4.14 高级熵编码宏块填充位 **aec_mb_stuffing_bit**

填充位。条带的最后一个宏块的 **aec_mb_stuffing_bit** 的值应为“1”,解析过程见 8.4。

7.2.5 宏块

7.2.5.1 宏块类型 **mb_type**

宏块的类型,其语义由图像类型、**PictureStructure** 和 **skip_mode_flag** 决定。如果 **aec_enable** 的值

为“0”,解析过程见 8.2;如果 aec_enable 的值为“1”,解析过程见 8.4。解码过程见 9.4.2。

7.2.5.2 宏块子类型 mb_part_type

2 位无符号整数。宏块的子类型。如果 aec_enable 的值为“1”,解析过程见 8.4。解码过程见 9.4.2。

7.2.5.3 预测模式标志 pred_mode_flag

标志。值为“1”表示根据预测模式的预测值确定帧内亮度预测模式;值为“0”表示根据 intra_luma_pred_mode 确定帧内亮度预测模式。

7.2.5.4 帧内亮度预测模式 intra_luma_pred_mode

2 位无符号整数。用于确定亮度块的帧内预测模式。如果 aec_enable 的值为“1”,解析过程见 8.4。解码过程见 9.4.4。

7.2.5.5 帧内色度预测模式 intra_chroma_pred_mode

用于确定 4:2:0 和 4:2:2 格式下宏块中顺序号为 4、5 的两个色度块的帧内预测模式。如果 aec_enable 的值为“0”,解析过程见 8.2;如果 aec_enable 的值为“1”,解析过程见 8.4。解码过程见 9.4.4。

7.2.5.6 4:2:2 帧内色度预测模式 intra_chroma_pred_mode_422

用于确定 4:2:2 格式下宏块中顺序号为 6、7 的两个色度块的帧内预测模式。解析过程见 8.2。

7.2.5.7 宏块参考索引 mb_reference_index

如果 aec_enable 的值为“1”,解析过程见 8.4。

如果 PictureStructure 的值为 0 并且 PictureType 的值为 1,mb_reference_index 是 2 位无符号整数;否则,mb_reference_index 是 1 位无符号整数。

解码过程见 9.4.5。先解码全部前向参考索引,然后解码全部后向参考索引。

7.2.5.8 运动矢量水平分量差值 mv_diff_x,运动矢量垂直分量差值 mv_diff_y

运动矢量差值。单位为 1/4 样本,取值范围为-4 096~4 095(按亮度像素样本为-1 024~1 023.75)。先解码全部前向运动矢量,然后解码全部后向运动矢量。如果 aec_enable 的值为“0”,解析过程见 8.2;如果 aec_enable 的值为“1”,解析过程见 8.4。解码过程见 9.4.6。

7.2.5.9 加权预测 weighting_prediction

标志。如果 aec_enable 的值为“1”,解析过程见 8.4。

7.2.5.10 宏块编码模板 cbp

cbp 确定宏块中顺序号为 0 到 5 的 8×8 亮度块和色度块是否包含编码数据。由 cbp 得到一个 6 位无符号整数 MbCBP。

如果 aec_enable 的值为“0”,cbp 的解析过程见 8.2;如果 aec_enable 的值为“1”,cbp 的解析过程见 8.4。MbCBP 的解码过程见 9.4.7。

7.2.5.11 4:2:2 宏块编码模板 cbp_422

cbp_422 确定 4:2:2 格式下宏块中顺序号为 6、7 的两个 8×8 色度块是否包含编码数据。由 cbp_422 得到一个 2 位无符号整数 MbCBP422。如果 aec_enable 的值为“0”,cbp_422 的解析过程见 8.2;如果

aec_enable 的值为“1”，cbp_422 的解析过程见 8.4。MbCBP422 的解码过程见 9.4.7。

7.2.5.12 宏块量化参数增量 **mb_qp_delta**

给出当前宏块的量化参数相对预测量化参数的增量。如果 aec_enable 的值为“0”，解析过程见 8.2；如果 aec_enable 的值为“1”，解析过程见 8.4，解码过程见 9.4.8。

7.2.6 块

7.2.6.1 变换系数 **trans_coefficient**

如果 aec_enable 的值为“0”，用于确定游程长度和非零量化系数值的联合索引，或用于确定转逸游程长度和转逸系数值的符号，解析过程见 8.3，解码过程见 9.5.1。

如果 aec_enable 的值为“1”，用于确定游程长度和非零量化系数值，解析过程见 8.4，解码过程见 9.5.2。

7.2.6.2 转逸系数差值 **escape_level_diff**

escape_level_diff 用于确定转逸系数的绝对值，解析过程见 8.3，解码过程见 9.5.1。

8 解析过程

8.1 k 阶指数哥伦布码

解析 k 阶指数哥伦布码时，首先从位流的当前位置开始寻找第一个非零位，并将找到的零位个数记为 leadingZeroBits，然后根据 leadingZeroBits 计算 CodeNum。用伪代码描述如下：

```
leadingZeroBits = -1
for(b=0; ! b; leadingZeroBits++)
    b = read_bits(1)
CodeNum = 2leadingZeroBits+k - 2k + read_bits(leadingZeroBits+k)
```

表 40 给出了 0 阶、1 阶、2 阶和 3 阶指数哥伦布码的结构。指数哥伦布码的位串分为“前缀”和“后缀”两部分。前缀由 leadingZeroBits 个连续的“0”和一个“1”构成。后缀由 leadingZeroBits+k 个位构成，即表中的 x_i 串， x_i 的值为“0”或“1”。

表 40 k 阶指数哥伦布码表

阶数	码字结构	CodeNum 取值范围
k=0	1	0
	0 1 x_0	1~2
	0 0 1 $x_1 x_0$	3~6
	0 0 0 1 $x_2 x_1 x_0$	7~14
	...	...
k=1	1 x_0	0~1
	0 1 $x_1 x_0$	2~5
	0 0 1 $x_2 x_1 x_0$	6~13
	0 0 0 1 $x_3 x_2 x_1 x_0$	14~29
	...	...

表 40 (续)

阶数	码字结构	CodeNum 取值范围
k=2	$1\ x_1\ x_0$	0~3
	$0\ 1\ x_2\ x_1\ x_0$	4~11
	$0\ 0\ 1\ x_3\ x_2\ x_1\ x_0$	12~27
	$0\ 0\ 0\ 1\ x_4\ x_3\ x_2\ x_1\ x_0$	28~59
	...	...
k=3	$1\ x_2\ x_1\ x_0$	0~7
	$0\ 1\ x_3\ x_2\ x_1\ x_0$	8~23
	$0\ 0\ 1\ x_4\ x_3\ x_2\ x_1\ x_0$	24~55
	$0\ 0\ 0\ 1\ x_5\ x_4\ x_3\ x_2\ x_1\ x_0$	56~119
	...	...

8.2 ue(v)、se(v)和 me(v)的解析过程

- ue(v)、se(v)和 me(v)描述的语法元素使用 0 阶指数哥伦布码,它们的解析过程为:
- ue(v):语法元素的值等于 CodeNum;
 - se(v):根据表 41 给出的有符号指数哥伦布码的映射关系求语法元素的值;
 - me(v):分别根据表 42 和表 43 求 MbCBP 和 MbCBP422 的值(见 9.4.1 和 9.4.7);

表 41 se(v)与 CodeNum 的映射关系

CodeNum 值	语法元素值
0	0
1	1
2	-1
3	2
4	-2
5	3
6	-3
k	$(-1)^{k+1} \times \text{Ceil}(k \div 2)$

表 42 MbCBP 与 CodeNum 的映射关系

CodeNum 值	MbCBP 值 xxxxxxx (543210)	
	宏块帧内编码模式	宏块帧间编码模式
0	63	0
1	15	15
2	31	63
3	47	31
4	0	16
5	14	32
6	13	47
7	11	13
8	7	14
9	5	11
10	10	12
11	8	5
12	12	10
13	61	7
14	4	48
15	55	3
16	1	2
17	2	8
18	59	4
19	3	1
20	62	61
21	9	55
22	6	59
23	29	62
24	45	29
25	51	27
26	23	23
27	39	19
28	27	30
29	46	28
30	53	9
31	30	6

表 42（续）

CodeNum 值	MbCBP 值 xxxxxxx (543210)	
	宏块帧内编码模式	宏块帧间编码模式
32	43	60
33	37	21
34	60	44
35	16	26
36	21	51
37	28	35
38	19	18
39	35	20
40	42	24
41	26	53
42	44	17
43	32	37
44	58	39
45	24	45
46	20	58
47	17	43
48	18	42
49	48	46
50	22	36
51	33	33
52	25	34
53	49	40
54	40	52
55	36	49
56	34	50
57	50	56
58	52	25
59	54	22
60	41	54
61	56	57
62	38	41
63	57	38

表 43 MbCBP422 与 CodeNum 的映射关系

CodeNum 值	MbCBP422 值 xx (10)	
	宏块帧内编码模式	宏块帧间编码模式
0	0	0
1	1	1
2	2	2
3	3	3

8.3 ce(v)的解析过程

ce(v)描述的语法元素采用 0 阶、1 阶、2 阶或 3 阶指数哥伦布码进行解析,其阶数由下面的规则决定:

- 帧内编码块亮度系数的 escape_level_diff 采用 1 阶指数哥伦布码。
- 帧间编码块亮度系数的 escape_level_diff 采用 0 阶指数哥伦布码。
- 色度系数的 escape_level_diff 采用 0 阶指数哥伦布码。
- 本部分定义了 19 个与 ce(v)有关的基本熵编码表,即 VLC0_Intra, VLC1_Intra, VLC2_Intra, VLC3_Intra, VLC4_Intra, VLC5_Intra, VLC6_Intra, VLC0_Inter, VLC1_Inter, VLC2_Inter, VLC3_Inter, VLC4_Inter, VLC5_Inter, VLC6_Inter, VLC0_Chroma, VLC1_Chroma, VLC2_Chroma, VLC3_Chroma, VLC4_Chroma, 见附录 D。不同的码表决定了 ce(v)所用的指数哥伦布码的阶数。其中 VLC0_Inter 采用 3 阶指数哥伦布码, VLC2_Chroma 及 VLC3_Chroma 采用 1 阶指数哥伦布码, VLC1_Chroma 及 VLC4_Chroma 采用 0 阶指数哥伦布码,其他码表均采用 2 阶指数哥伦布码。

本条中上述 19 个码表的选择按照以下规则进行:

- a) 编码块第 1 个解码量化系数的码表选择:
 - 1) 帧内预测编码块的亮度系数, CurrentVLCTable=VLC0_Intra, 见表 D.1;
 - 2) 帧间预测编码块的亮度系数, CurrentVLCTable=VLC0_Inter, 见表 D.8;
 - 3) 色度系数, CurrentVLCTable=VLC0_Chroma, 见表 D.15。
- b) 其他解码量化系数的码表选择:
 - 1) 对于帧内预测编码块的亮度系数, 如果 absLevel 大于 maxAbsLevel, 按如下方式进行码表切换:
 - 如果 absLevel 等于 1, 选择 CurrentVLCTable=VLC1_Intra, 见表 D.2;
 - 如果 absLevel 等于 2, 选择 CurrentVLCTable=VLC2_Intra, 见表 D.3;
 - 如果 absLevel 等于 3 或 4, 选择 CurrentVLCTable=VLC3_Intra, 见表 D.4;
 - 如果 absLevel 等于 5、6 或 7, 选择 CurrentVLCTable=VLC4_Intra, 见表 D.5;
 - 如果 absLevel 等于 8、9 或 10, 选择 CurrentVLCTable=VLC5_Intra, 见表 D.6;
 - 如果 absLevel 大于 10, 选择 CurrentVLCTable=VLC6_Intra, 见表 D.7。
 - 2) 对于帧间预测编码块的亮度系数, 如果 absLevel 大于 maxAbsLevel, 按如下方式进行码表切换:
 - 如果 absLevel 等于 1, 选择 CurrentVLCTable=VLC1_Inter, 见表 D.9;

- 如果 absLevel 等于 2, 选择 CurrentVLCTable=VLC2_Inter, 见表 D.10;
 - 如果 absLevel 等于 3, 选择 CurrentVLCTable=VLC3_Inter, 见表 D.11;
 - 如果 absLevel 等于 4、5 或 6, 选择 CurrentVLCTable=VLC4_Inter, 见表 D.12;
 - 如果 absLevel 等于 7、8 或 9, 选择 CurrentVLCTable=VLC5_Inter, 见表 D.13;
 - 如果 absLevel 大于 9, 选择 CurrentVLCTable=VLC6_Inter, 见表 D.14。
- 3) 对于色度块的系数, 如果 absLevel 大于 maxAbsLevel, 按如下方式进行码表切换:
- 如果 absLevel 等于 1, 选择 CurrentVLCTable=VLC1_Chroma, 见表 D.16;
 - 如果 absLevel 等于 2, 选择 CurrentVLCTable=VLC2_Chroma, 见表 D.17;
 - 如果 absLevel 等于 3 或 4, 选择 CurrentVLCTable=VLC3_Chroma, 见表 D.18;
 - 如果 absLevel 大于 4, 选择 CurrentVLCTable=VLC4_Chroma, 见表 D.19。

ce(v)描述的语法元素解析过程如下:

- a) 语法元素 trans_coefficient 等于 CodeNum。
- b) 如果 trans_coefficient 大于或等于 59, 解析下一个 ce(v)语法元素, 得到一个新的 CodeNum, escape_level_diff 等于该 CodeNum。

8.4 ae(v)的解析过程

8.4.1 概述

ae(v)描述的语法元素解析过程如下:

- a) 对条带进行解析前, 首先进行初始化, 见 8.4.2。
- b) 对二值化得到的二元符号串进行解析, 见 8.4.4。
 - 1) 二元符号串中每个二元符号的索引号是 binIdx, 对应唯一的 ctxIdx, 见 8.4.4.2;
 - 2) 根据 ctxIdx 解析二元符号, 见 8.4.4.3;
 - 3) 完成一个二元符号的解析后, 将得到的二元符号串与 8.4.3 中的表格进行比较。如果得到的二元符号串与表格中某个二元符号串相匹配, 则输出相应语法元素值; 否则继续解析二元符号。

上述解析过程用伪代码描述如下:

```

if(当前语法元素是条带的第 1 个语法元素){
    初始化所有上下文模型
    初始化高级熵编码解码器
}
语法元素二值化
binIdx=-1
do {
    binIdx++
    得到与 binIdx 对应的 ctxIdx
    得到与 ctxIdx 对应的上下文模型 ctx
    根据 ctx 解析二元符号
} while((b0, ..., bbinIdx)不是语法元素的二元符号串)
输出语法元素值
  
```

8.4.2 初始化

8.4.2.1 初始化上下文模型

每个上下文模型 ctx 需要初始化三个变量 mps、cycno 和 lgPmps。mps 的位宽为 1 位, cycno 的位

宽为 2 位,lgPmps 的位宽为 11 位。mps 和 cycno 的值应初始化为“0”;lgPmps 的值应初始化为“1023”。

8.4.2.2 初始化高级熵编码解码器

rS1、rT1、valueS 和 valueT 是用于高级熵编码解码器的变量。rT1 的位宽是 8 位,valueT 的位宽是 9 位。rS1 的值应初始化为“0”;rT1 的值应初始化为“0xFF”。

valueS 和 valueT 的初始化过程用伪代码描述如下:

```
valueS=0
valueT=read_bits(9)
while(!((valueT >> 8) & 0x01)){
valueT=(valueT<<1) | read_bits(1)
valueS++
}
valueT=valueT & 0xFF
```

注:初始化高级熵编码解码器也可参考附录 E 的实现方式。

8.4.3 二值化

不同语法元素的二值化过程如下:

- a) mb_skip_run、mb_qp_delta、P 帧的 mb_reference_index 和 P 帧的 mb_type 的二值化过程:语法元素的值为 synElVal,synElVal 与二元符号串的关系见表 44。

表 44 ynElVal 与二元符号串的关系

synElVal 的值	二元符号串					
0	1					
1	0	1				
2	0	0	1			
3	0	0	0	1		
4	0	0	0	0	1	
5	0	0	0	0	0	1
...						
binIdx 的值	0	1	2	3	4	5

- b) B 帧的 mb_type 的二值化过程:语法元素的值为 synElVal,synElVal 与二元符号串的关系见表 45。

表 45 synElVal 与二元符号串的关系

synElVal 的值	二元符号串					
0	0					
1	1	1				
2	1	0	1			

表 45 (续)

synElVal 的值	二元符号串					
3	1	0	0	1		
4	1	0	0	0	1	
5	1	0	0	0	0	1
...						
binIdx 的值	0	1	2	3	4	5

- c) mb_part_type 的二值化过程:语法元素的值为 synElVal, synElVal 与二元符号串的关系见表 46。

表 46 synElVal 与二元符号串的关系

synElVal 的值	二元符号串	
0	0	0
1	0	1
2	1	0
3	1	1
binIdx 的值	0	1

- d) intra_luma_pred_mode 的二值化过程:语法元素的值为 synElVal, synElVal 与二元符号串的关系见表 47。

表 47 synElVal 与二元符号串的关系

synElVal 的值	二元符号串			
0	1			
1	0	1		
2	0	0	1	
3	0	0	0	1
4	0	0	0	0
binIdx 的值	0	1	2	3

- e) intra_chroma_pred_mode 和 intra_chroma_pred_mode_422 的二值化过程:语法元素的值为 synElVal, synElVal 与二元符号串的关系见表 48。

表 48 synElVal 与二元符号串的关系

synElVal 的值	二元符号串		
0	0		
1	1	0	
2	1	1	0
3	1	1	1
binIdx 的值	0	1	2

- f) mv_diff_x 和 mv_diff_y 的二值化过程:语法元素分为绝对值 $mvdAbs$ 和符号位 $mvdSign$ 两部分,先解析 $mvdAbs$,再解析 $mvdSign$ 。 $mvdAbs$ 的值为 $synElVal$, $mvdSign$ 的解析过程见 8.4.4。 $synElVal$ 与二元符号串的关系见表 49。表 49 中,如果 $synElVal$ 的值大于或等于 3 并且 $synElVal$ 的值为奇数,二元符号串的前四位为“1110”,后续位为 $(synElVal-3)/2$ 对应的 0 阶指数哥伦布码(见表 40);如果 $synElVal$ 的值大于 3 并且 $synElVal$ 的值为偶数,二元符号串的前四位为“1111”,后续位为 $(synElVal-3)/2$ 对应的 0 阶指数哥伦布码。如果 $mvdAbs$ 为“0”,此时不应解析 $mvdSign$ 。 $mvdSign$ 的值为“0”表示 $mvdAbs$ 的值为 $synElVal$; $mvdSign$ 的值为 1 表示 $mvdAbs$ 的值为 $-synElVal$ 。

表 49 $synElVal$ 与二元符号串的关系

$synElVal$ 的值	二元符号串										
0	0										
1	1	0									
2	1	1	0								
3	1	1	1	0	1						
4	1	1	1	1	1						
5	1	1	1	0	0	1	0				
6	1	1	1	1	0	1	0				
7	1	1	1	0	0	1	1				
8	1	1	1	1	0	1	1				
9	1	1	1	0	0	0	1	0	0		
10	1	1	1	1	0	0	1	0	0		
11	1	1	1	0	0	0	1	0	1		
12	1	1	1	1	0	0	1	0	1		
13	1	1	1	0	0	0	1	1	0		
14	1	1	1	1	0	0	1	1	0		
...											
$binIdx$ 的值	0	1	2	3	4	5	6	7	8	9	10

- g) cbp 的二值化过程:语法元素的值为 $synElVal$,如果 $synElVal$ 小于 16,二元符号串为 $synElVal$ 的四位前缀,后缀为“0”;如果 $synElVal$ 大于 15 并且小于或等于 31,二元符号串为 $synElVal$ 的四位前缀,后续位为“100”;如果 $synElVal$ 大于 31 并且小于或等于 47,二元符号串为 $synElVal$ 的四位前缀,后续位为“101”;如果 $synElVal$ 大于 47,二元符号串为 $synElVal$ 的四位前缀,后续位为“11”。 $synElVal$ 与二元符号串的关系见表 50, $MbCBP$ 的值等于 $synElVal$ 的值。

表 50 synElVal 与二元符号串的关系

synElVal 的值	二元符号串						
	四位前缀				后缀		
1	1	0	0	0	0		
2	0	1	0	0	0		
3	1	1	0	0	0		
...							
15	1	1	1	1	0		
16	0	0	0	0	1	0	0
17	1	0	0	0	1	0	0
...							
31	1	1	1	1	1	0	0
32	0	0	0	0	1	0	1
33	1	0	0	0	1	0	1
...							
47	1	1	1	1	1	0	1
48	0	0	0	0	1	1	
49	1	0	0	0	1	1	
...							
63	1	1	1	1	1	1	
binIdx 的值	0	1	2	3	4	5	6

- h) cbp₄₂₂ 的二值化过程:语法元素的值为 synElVal。如果二元符号串为“00”,synElVal 的值为 0。如果二元符号串为“01”,synElVal 的值为 1。如果二元符号串为“10”,synElVal 的值为 2。如果二元符号串为“11”,synElVal 的值为 3。MbCBP₄₂₂ 的值等于 synElVal 的值。
- i) trans_coefficient 的二值化过程:语法元素可分为 coeffLevel、coeffSign 和 coeffRun 三部分。首先由 trans_coefficient 得到 coeffLevel,如果是块的第一个 coeffLevel 或者 coeffLevel 的值不等于“0”,则继续得到 coeffSign 和 coeffRun。AbsLevel 的值等于根据 coeffLevel 查表 44 得到的 synElVal。coeffSign 是一个二元符号“0”或“1”。RunVal 的值等于根据 coeffRun 查表 44 得到的 synElVal。
- j) B 帧的 mb_reference_index 的二值化过程:语法元素的值为 synElVal。如果二元符号串为“0”,synElVal 的值为“1”。如果二元符号串为“1”,synElVal 的值为“0”。
- k) weighting_prediction 的二值化过程:语法元素的值为 synElVal。如果二元符号串为“0”,synElVal 的值为“0”。如果二元符号串为“1”,synElVal 的值为“1”。

8.4.4 二元符号串解析

8.4.4.1 概述

对每个二元符号,置 contextWeighting 的值为“0”。如果二元符号是 mvdSign 或 coeffSign,或该二

元符号属于 mv_diff_x 和 mv_diff_y 的 0 阶指数哥伦布码部分,或该二元符号属于 cbp_422 ,或该二元符号属于 $intra_chroma_pred_mode_422$,则 BypassFlag 的值为“1”;否则 BypassFlag 的值为“0”。解析二元符号时,如果 BypassFlag 的值为“0”,则先由 binIdx 确定 ctxIdx(见 8.4.4.2),然后再解析二元符号(见 8.4.4.3);否则,直接解析二元符号(见 8.4.4.3)。

在解析二元符号串的过程中,binIdx 从 0 开始,每解析一个二元符号,binIdx 加 1,并且将得到的二元符号串与表 44~表 50 中相应的二元符号串比对,得到 synElVal。

8.4.4.2 确定 ctxIdx

ctxIdx 由起始索引号和 ctxIdxInc 决定。语法元素的起始索引号见表 51。

表 51 语法元素起始索引号

语法元素	类型	起始索引号
mb_skip_run		0
mb_type		4
mb_part_type		19
intra_luma_pred_mode		22
intra_chroma_pred_mode		26
mb_reference_index		30
mv_diff_x		36
mv_diff_y		42
cbp		48
mb_qp_delta		54
trans_coefficient	帧模式亮度	58
	帧模式色度	124
	场模式亮度	190
	场模式色度	256
weighting_prediction		322

确定各种语法元素的 ctxIdx 的步骤如下:

- a) 确定 mb_skip_run 的 ctxIdx:
 - 1) 第一步,根据 binIdx 得到 ctxIdxInc:
 $ctxIdxInc = \text{Min}(\text{binIdx}, 3)$
 - 2) 第二步,计算 ctxIdx:
 $ctxIdx = \text{起始索引号} + ctxIdxInc$
- b) 确定 mb_type 的 ctxIdx:
 - 1) 第一步,根据 binIdx 得到 ctxIdxInc:
——如果当前帧是 P 帧,则:

$ctxIdxInc = \text{Min}(binIdx, 4)$

——否则,如果当前帧是 B 帧,并且 $binIdx$ 等于 0,则:

$ctxIdxInc = 5 + a + b$

——否则,如果当前帧是 B 帧,并且 $binIdx$ 小于或等于 7,则:

$ctxIdxInc = 7 + binIdx$

——否则,如果当前帧是 B 帧,并且 $binIdx$ 大于 7,则:

$ctxIdxInc = 14$

2) 第二步,计算 $ctxIdx$:

$ctxIdx = \text{起始索引号} + ctxIdxInc$

如果当前块 E 的左边块 A(或上边块 B)“可用”,并且块 A(或块 B)的宏块类型不是 P_Skip、B_Skip 或 B_Direct_16x16,则 a(或 b)的值为“1”;否则 a(或 b)的值为“0”。块 E 和块 A、块 B 的关系见 9.4.3。

c) 确定 mb_part_type 的 $ctxIdx$:

1) 第一步,根据 $binIdx$ 得到 $ctxIdxInc$:

——如果 $binIdx$ 等于 0,则:

$ctxIdxInc = 0$

——否则,如果二元符号串的第一个二元符号为“0”,则:

$ctxIdxInc = 1$

——否则:

$ctxIdxInc = 2$

2) 第二步,计算 $ctxIdx$:

$ctxIdx = \text{起始索引号} + ctxIdxInc$

d) 确定 $intra_luma_pred_mode$ 的 $ctxIdx$:

1) 第一步,根据 $binIdx$ 得到 $ctxIdxInc$:

$ctxIdxInc = \text{Min}(binIdx, 3)$

2) 第二步,计算 $ctxIdx$:

$ctxIdx = \text{起始索引号} + ctxIdxInc$

e) 确定 $intra_chroma_pred_mode$ 的 $ctxIdx$:

1) 第一步,根据 $binIdx$ 得到 $ctxIdxInc$:

——如果 $binIdx$ 等于 0,则:

$ctxIdxInc = a + b$

——否则:

$ctxIdxInc = 3$

2) 第二步,计算 $ctxIdx$:

$ctxIdx = \text{起始索引号} + ctxIdxInc$

如果当前块 E 的左边块 A(或上边块 B)“可用”,并且块 A(或块 B)的预测模式不是 Intra_Chroma_DC,则 a(或 b)的值为 1;否则 a(或 b)的值为“0”。块 E 和块 A、块 B 的关系见 9.4.3。

f) 确定 $mb_reference_index$ 的 $ctxIdx$:

1) 第一步,根据 $binIdx$ 得到 $ctxIdxInc$:

——如果 $binIdx$ 等于 0,则:

$ctxIdxInc = a + 2 \times b$

——否则,如果 $binIdx$ 等于 1,则:

$ctxIdxInc = 4$

——否则:

$ctxIdxInc=5$

2) 第二步,计算 $ctxIdx$:

$ctxIdx=起始索引号+ctxIdxInc$

如果当前块 E 的左边块 A(或上边块 B)“可用”,并且块 A(或块 B)的参考索引值大于 0,则 a(或 b)的值为 1;否则 a(或 b)的值为“0”。块 E 和块 A、块 B 的关系见 9.4.3。

块 A 和块 B 的参考索引值的定义和赋值过程见 9.4.5 和 9.4.6.2。

g) 确定 mv_diff_x 、 mv_diff_y 的 $ctxIdx$:

1) 第一步,根据 $binIdx$ 得到 $ctxIdxInc$:

——如果 $binIdx$ 等于 0,则:

$if(mvda<2)$

$ctxIdxInc=0$

$else\ if(mvda<16)$

$ctxIdxInc=1$

$else$

$ctxIdxInc=2$

——否则,如果 $binIdx$ 等于 1,则:

$ctxIdxInc=3$

——否则,如果 $binIdx$ 等于 2,则:

$ctxIdxInc=4$

——否则,如果 $binIdx$ 等于 3,则:

$ctxIdxInc=5$

——否则,BypassFlag 的值为 1。

2) 第二步,计算 $ctxIdx$:

$ctxIdx=起始索引号+ctxIdxInc$

如果当前块 E 的左边 8×8 块 A“可用”,并且块 A 的 mv_diff_x 或 mv_diff_y 在位流中存在,则 $mvda$ 的值按以下方法决定:

——如果当前块 E 是前向预测块,并且块 A 是前向或双向预测块,则 $mvda$ 的值为块 A 的 mv_diff_x 或 mv_diff_y 的绝对值;

——否则,如果当前块 E 是双向预测块,并且块 A 是前向或双向预测块,则 $mvda$ 的值为块 A 的 mv_diff_x 或 mv_diff_y 的绝对值;

——否则,如果当前块 E 是后向预测块,并且块 A 是后向预测块,则 $mvda$ 的值为块 A 的 mv_diff_x 或 mv_diff_y 的绝对值;

——否则, $mvda$ 的值为“0”。

如果当前块 E 的左边 8×8 块 A“不可用”,则 $mvda$ 的值为“0”。块 E 和块 A 的关系见 9.4.3。

h) 确定 cbp 的 $ctxIdx$:

1) 第一步,根据 $binIdx$ 得到 $ctxIdxInc$:

——如果 $binIdx$ 的值小于或等于 3,则:

$ctxIdxInc=a+2\times b$

——否则,如果 $binIdx$ 的值等于 4,则:

$ctxIdxInc=4$

——否则:

$ctxIdxInc=5$

2) 第二步,计算 ctxIdx:

ctxIdx=起始索引号+ctxIdxInc

如果当前块 E 的左边 8×8 块 A(或上边 8×8 块 B)“可用”,并且块 A(或块 B)中不包含非零系数,则 a(或 b)的值为“1”;否则 a(或 b)的值为“0”。块 E 和块 A、块 B 的关系见 9.4.3。

i) 确定 mb_qp_delta 的 ctxIdx:

1) 第一步,根据 binIdx 得到 ctxIdxInc:

——如果 binIdx 等于 0,则:

if (PreviousDeltaQP != 0)

ctxIdxInc=1

else

ctxIdxInc=0

——否则,如果 binIdx 等于 1,则:

ctxIdxInc=2

——否则:

ctxIdxInc=3

2) 第二步,计算 ctxIdx:

ctxIdx=起始索引号+ctxIdxInc

j) 确定 trans_coefficient 的 coeffLevel 的 ctxIdx:

1) 第一步,根据 binIdx 得到 ctxIdxInc:

——如果 binIdx 不等于 0 或 lMax 等于 0,则:

contextWeighting=0

ctxIdxInc=priIdx $\times$ 3+secIdx-(priIdx != 0)

——否则:

contextWeighting=1

ctxIdxInc=priIdx $\times$ 3+secIdx-1

ctxIdxIncW=14+(pos >> 5) $\times$ 16+((pos >> 1) & 0x0F)

2) 第二步,计算 ctxIdx:

ctxIdx=起始索引号+ctxIdxInc

如果 contextWeighting=1,则 ctxIdxW=起始索引号+ctxIdxIncW

其中 priIdx 和 secIdx 分别见表 52 和表 53;pos 记录了待解码的量化系数幅值在逆扫描中的位置。对每个 8×8 块进行解码前,pos 应初始化为“0”。lMax 是当前 8×8 块已解码系数的最大幅值。对每个 8×8 块进行解码前,lMax 应初始化为“0”。

k) 确定 trans_coefficient 的 coeffRun 的 ctxIdx:

1) 第一步,根据 binIdx 和 AbsLevel 得到 ctxIdxInc:

ctxIdxInc=priIdx $\times$ 4+secIdx

2) 第二步,计算 ctxIdx:

ctxIdx=起始索引号+46+ctxIdxInc

其中 priIdx 和 secIdx 分别见表 52 和表 53。

l) 确定 weighting_prediction:

1) 第一步,ctxIdxInc 赋值为“0”;

2) 第二步,计算 ctxIdx:

ctxIdx=起始索引号+ctxIdxInc

表 52 priIdx 与 lMax 的关系

lMax 的值	priIdx 的值
0	0
1	1
2	2
3	3
4	3
≥ 5	4

表 53 secIdx 与 coeffLevel、coeffRun 的关系

secIdx 的值	含 义		
	coeffLevel 的含义		coeffRun 的含义
	lMax 的值为 0	lMax 的值不为 0	
0	binIdx 等于 0	binIdx 等于 0	AbsLevel 的值等于 1,并且 binIdx 等于 0
1	binIdx 大于或等于 1	binIdx 等于 1	AbsLevel 的值等于 1,并且 binIdx 大于或等于 1
2	—	binIdx 大于或等于 2	AbsLevel 的值大于 1,并且 binIdx 等于 0
3	—	—	AbsLevel 的值大于 1,并且 binIdx 大于或等于 1

8.4.4.3 二元符号解析

8.4.4.3.1 解析过程

二元符号的解析过程如下：

a) 首先,解析二元符号值 binVal

- 1) 如果 BypassFlag 的值为“1”,执行 decode_bypass 过程,见 8.4.4.3.3;
- 2) 否则,如果当前解码语法元素为 aec_mb_stuffing_bit,则执行 decode_aec_stuffing_bit 过程(见 8.4.4.3.4);
- 3) 否则,执行 decode_decision 过程,见 8.4.4.3.2。

b) 第二步,如果 binVal 的值为“0”,则二元符号为‘0’;如果 binVal 的值为“1”,则二元符号为‘1’。

8.4.4.3.2 decode_decision

如果 contextWeighting 的值为 1,decode_decision 过程的输入是 rS1、rT1、valueS、valueT 以及上下文模型 ctx1、ctx2;否则 decode_decision 过程的输入是 rS1、rT1、valueS、valueT 以及上下文模型 ctx。decode_decision 过程的输出是二元符号值 binVal。decode_decision 过程用伪代码描述如下：

```
decode_decision( )
{
    if(contextWeighting==1){
        if(ctx1->mps==ctx2->mps){
            predMps=ctx1->mps
```

```

        lgPmps=(ctx1->lgPmps+ctx2->lgPmps) / 2
    }
    else {
        if(ctx1->lgPmps<ctx2->lgPmps){
            predMps=ctx1->mps
            lgPmps=1 023-((ctx2->lgPmps-ctx1->lgPmps) >> 1)
        }
        else {
            predMps=ctx2->mps
            lgPmps=1 023 - ((ctx1->lgPmps-ctx2->l gPmps) >> 1)
        }
    }
}
else {
    predMps=ctx->mps
    lgPmps= ctx->lgPmps
}

if(rT1 >=(lgPmps >> 2)){
    rS2=rS1
    rT2=rT1-(lgPmps >> 2 )
    sFlag=0
}
else {
    rS2=rS1+1
    rT2=256+rT1-(lgPmps >> 2)
    sFlag=1
}

if( rS2>valueS || (rS2==valueS && valueT >=rT2)){
    binVal=! predMps
    if(sFlag==0 )
        tRlps=lgPmps >> 2
    else
        tRlps=rT1+(lgPmps >> 2)
    if(rS2==valueS )
        valueT=valueT-rT2
    else
        valueT=256+((valueT<<1)| read_bits(1))-rT2
    while(tRlps<0x100){
        tRlps=tRlps<<1
        valueT=(valueT<<1)| read_bits(1)
    }
}

```

```

    rS1=0
    rT1=tRlps & 0xFF
    valueS=0
    while(valueT<0x100){
        valueS++
        valueT=(valueT<<1) | read_bits(1)
    }
    valueT=valueT & 0xFF
}
else {
    binVal=predMps
    rS1=rS2
    rT1=rT2
}

if(contextWeighting==1){
    ctx1=update_ctx(binVal,ctx1)
    ctx2=update_ctx(binVal,ctx2)
}
else
    ctx=update_ctx(binVal,ctx)
return (binVal)
}

```

注：decode_decision 的实现也可参考附录 E 的实现方式。

8.4.4.3.3 decode_bypass

decode_bypass 过程的输入是 rS1、rS2、valueS 和 valueT。decode_bypass 过程的输出是二元符号值 binVal。decode_bypass 过程用伪代码描述如下：

```

decode_bypass( )
{
    predMps=0
    lgPmps=1 023
    if(rT1 >=(lgPmps >> 2)){
        rS2=rS1
        rT2=rT1-(lgPmps >> 2)
        sFlag=0
    }
    else {
        rS2=rS1+1
        rT2=256+rT1-(lgPmps >> 2)
        sFlag=1
    }
    if( rS2>valueS || (rS2==valueS && valueT >=rT2)){

```

```

    binVal = ! predMps
    if(sFlag == 0 )
        tRlps = lgPmps >> 2
    else
        tRlps = rT1 + (lgPmps >> 2)
    if(rS2 == valueS )
        valueT = valueT - rT2
    else
        valueT = ((valueT << 1) | read_bits(1)) - rT2 + 256
    while(tRlps < 0x100){
        tRlps = tRlps << 1
        valueT = (valueT << 1) | read_bits(1)
    }
    rS1 = 0
    rT1 = tRlps & 0xFF
    valueS = 0
    while(valueT < 0x100){
        valueS ++
        valueT = (valueT << 1) | read_bits(1)
    }
    valueT = valueT & 0xFF
}
else {
    binVal = predMps
    rS1 = rS2
    rT1 = rT2
}
return (binVal)
}

```

注：decode_bypass 的实现也可参考附录 E 的实现方式。

8.4.4.3.4 decode_aec_stuffing_bit

decode_aec_stuffing_bit 过程的输入是 rS1、rS2、valueS 和 valueT。decode_aec_stuffing_bit 过程的输出是二元符号值 binVal。decode_aec_stuffing_bit 过程用伪代码描述如下：

```

decode_aec_stuffing_bit( )
{
    predMps = 0
    lgPmps = 4
    if(rT1 >= (lgPmps >> 2)){
        rS2 = rS1
        rT2 = rT1 - (lgPmps >> 2 )
        sFlag = 0
    }
}

```

```

else {
    rS2=rS1+1
    rT2=256+rT1-(lgPmps >> 2)
    sFlag=1
}
if( rS2>valueS || (rS2==valueS && valueT >=rT2)){
    binVal=! predMps
    if(sFlag==0 )
        tRlps=lgPmps >> 2
    else
        tRlps=rT1+(lgPmps >> 2)
    if(rS2==valueS )
        valueT=valueT-rT2
    else
        valueT=256+((valueT<<1) | read_bits(1))-rT2
    while(tRlps<0x100){
        tRlps=tRlps<<1
        valueT=(valueT<<1) | read_bits(1)
    }
    rS1=0
    rT1=tRlps & 0xFF
    valueS=0
    while(valueT<0x100){
        valueS++
        valueT=(valueT<<1) | read_bits(1)
    }
    valueT=valueT & 0xFF
}
else {
    binVal=predMps
    rS1=rS2
    rT1=rT2
}
return (binVal)
}

```

注：decode_aec_stuffing_bit 的实现也可参考附录 E 的实现方式。

8.4.4.3.5 update_ctx

update_ctx 过程的输入是 binVal 和 ctx。update_ctx 过程的输出是更新后的 ctx。update_ctx 过程用伪代码描述如下：

```

update_ctx( )
{
    if(ctx->cycno <=1 )

```

```

        cwr=3
    else if(ctx->cycno==2 )
        cwr=4
    else
        cwr=5
    if(binVal != ctx->mps){
        if(ctx->cycno <=2 )
            ctx->cycno=ctx->cycno+1
        else
            ctx->cycno=3
    }
    else if(ctx->cycno==0)
        ctx->cycno=1

    if(binVal==ctx->mps )
        ctx->lgPmps=ctx->lgPmps-(ctx->lgPmps >> cwr)-(ctx->lgPmps >>
(cwr+2))
    else {
        switch (cwr) {
            case 3:
                ctx->lgPmps=ctx->lgPmps+197
                break
            case 4:
                ctx->lgPmps=ctx->lgPmps+95
                break
            default:
                ctx->lgPmps=ctx->lgPmps+46
        }
        if(ctx->lgPmps>1 023){
            ctx->lgPmps=2 047-ctx->lgPmps
            ctx->mps=! (ctx->mps)
        }
    }
    return (ctx)
}

```

9 解码过程

9.1 高层语法结构

如果 progressive_sequence 的值为“1”，解码器以帧周期的整数倍为时间间隔输出重建图像。

如果 progressive_sequence 的值为“0”，则解码器将重建图像拆成两场，以场周期的整数倍为时间间隔输出。

9.2 图像头解码

图像头解码过程如下：

如果当前图像起始码是 0x00001B3,则表示图像是 I 帧,PictureType 等于“0”。

如果当前图像起始码是 0x00001B6,并且 picture_coding_type 等于“01”,则表示图像是 P 帧,PictureType 等于“1”。

如果当前图像起始码是 0x00001B6,并且 picture_coding_type 等于“10”,则表示图像是 B 帧,PictureType 等于“2”。

预测量化参数 PreviousQP 初始化为 picture_qp。预测量化参数增量 PreviousDeltaQP 初始化为“0”。固定量化因子标志 FixedQP 等于 fixed_picture_qp。当前图像的宏块索引 MbIndex 初始化为“0”。

CurrentSceneModel 等于 weighting_quant_model。

加权量化参数 WeightingQuantParamDefault[i] = {128, 98, 106, 116, 116, 128}; 加权量化参数 WeightingQuantParamBase1[i] = {135, 143, 143, 160, 160, 213}; 加权量化参数 WeightingQuantParamBase2[i] = {128, 98, 106, 116, 116, 128}。

加权量化矩阵 WeightingQuantMatrix8x8 的元素 WeightingQuantMatrix8x8[i, j] (i, j = 0~7) 的值均初始化为“128”。

如果 weighting_quant_flag 的值为“1”,按以下步骤确定加权量化矩阵：

- a) 首先,初始化 wqP, wqP[0][i] = 128 (i = 0~5), wqP[1][i] = WeightingQuantParamBase1[i] (i = 0~5), wqP[2][i] = WeightingQuantParamBase2[i] (i = 0~5)。
- b) 第二步,确定当前图像的 wqP(wqP 的元素的取值范围应是 0~255):
 - 1) 如果 weighting_quant_param_index 的值为“00”,则 wqP[0][i] = WeightingQuantParamDefault[i] (i = 0~5)。
 - 2) 如果 weighting_quant_param_index 的值为“01”,从 weighting_quant_param_delta1[i] 解析得到 wqPDelta[1][i]。wqP[1][i] = wqPDelta[1][i] + WeightingQuantParamBase1[i] (i = 0~5)。
 - 3) 如果 weighting_quant_param_index 的值为“10”,从 weighting_quant_param_delta2[i] 解析得到 wqPDelta[2][i]。wqP[2][i] = wqPDelta[2][i] + WeightingQuantParamBase2[i] (i = 0~5)。
- c) 第三步,根据 CurrentSceneModel 确定 8×8 变换块的加权量化矩阵 wqM8x8[m]:

- 1) 如果 CurrentSceneModel 的值为“0”,则 wqM8x8[m] 如下:

$$wqM8 \times 8[m] = \begin{bmatrix} wqP[m][0] & wqP[m][0] & wqP[m][0] & wqP[m][4] & wqP[m][4] & wqP[m][4] & wqP[m][5] & wqP[m][5] \\ wqP[m][0] & wqP[m][0] & wqP[m][3] & wqP[m][3] & wqP[m][3] & wqP[m][3] & wqP[m][5] & wqP[m][5] \\ wqP[m][0] & wqP[m][3] & wqP[m][2] & wqP[m][2] & wqP[m][1] & wqP[m][1] & wqP[m][5] & wqP[m][5] \\ wqP[m][4] & wqP[m][3] & wqP[m][2] & wqP[m][2] & wqP[m][1] & wqP[m][5] & wqP[m][5] & wqP[m][5] \\ wqP[m][4] & wqP[m][3] & wqP[m][1] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] \\ wqP[m][4] & wqP[m][3] & wqP[m][1] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] \\ wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] \\ wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] & wqP[m][5] \end{bmatrix}, m=0 \sim 2$$
- 2) 如果 CurrentSceneModel 的值为“1”,则 wqM8x8[m] 如下:

$$\text{wqM8} \times 8[m] = \begin{bmatrix} \text{wqP}[m][0] & \text{wqP}[m][0] & \text{wqP}[m][0] & \text{wqP}[m][4] & \text{wqP}[m][4] & \text{wqP}[m][4] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][0] & \text{wqP}[m][0] & \text{wqP}[m][4] & \text{wqP}[m][4] & \text{wqP}[m][4] & \text{wqP}[m][4] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][0] & \text{wqP}[m][3] & \text{wqP}[m][2] & \text{wqP}[m][2] & \text{wqP}[m][2] & \text{wqP}[m][1] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][3] & \text{wqP}[m][3] & \text{wqP}[m][2] & \text{wqP}[m][2] & \text{wqP}[m][1] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][3] & \text{wqP}[m][3] & \text{wqP}[m][2] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][3] & \text{wqP}[m][3] & \text{wqP}[m][1] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] \end{bmatrix}, m=0 \sim 2$$

3) 如果 CurrentSceneModel 的值为“2”,则 $\text{wqM8} \times 8[m]$ 如下:

$$\text{wqM8} \times 8[m] = \begin{bmatrix} \text{wqP}[m][0] & \text{wqP}[m][0] & \text{wqP}[m][0] & \text{wqP}[m][4] & \text{wqP}[m][4] & \text{wqP}[m][3] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][0] & \text{wqP}[m][0] & \text{wqP}[m][4] & \text{wqP}[m][4] & \text{wqP}[m][3] & \text{wqP}[m][2] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][0] & \text{wqP}[m][4] & \text{wqP}[m][4] & \text{wqP}[m][3] & \text{wqP}[m][2] & \text{wqP}[m][1] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][4] & \text{wqP}[m][4] & \text{wqP}[m][3] & \text{wqP}[m][2] & \text{wqP}[m][1] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][4] & \text{wqP}[m][3] & \text{wqP}[m][2] & \text{wqP}[m][1] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][3] & \text{wqP}[m][2] & \text{wqP}[m][1] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] \\ \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] & \text{wqP}[m][5] \end{bmatrix}, m=0 \sim 2$$

9.3 条带解码

条带解码过程如下:

宏块索引 MbIndex 等于 MbRow × MbWidth。

如果 fixed_picture_qp 等于“0”,预测量化参数 PreviousQP 等于 slice_qp,预测量化参数增量 PreviousDeltaQP 初始化为“0”。固定量化因子标志 FixedQP 等于 fixed_slice_qp。

如果 mb_skip_run 存在,由 mb_skip_run 解析得到 SkipMbCount,否则 SkipMbCount 等于“0”。若 SkipMbCount 不等于“0”,从 MbIndex 至 MbIndex + SkipMbCount - 1 的所有宏块的 MbType 根据图像类型设为 P_Skip 或 B_Skip,这些宏块按照所设置的宏块类型进行处理。处理完成后,MbIndex 应加上 SkipMbCount。

如果当前条带的位流中存在 mb_weighting_flag,则 MbWeightingFlag 等于 mb_weighting_flag;否则 MbWeightingFlag 等于“0”。

9.4 宏块解码

9.4.1 初始化

解码索引值为 MbIndex 的宏块,完成后 MbIndex 加 1。

WeightingPrediction 的值等于 slice_weighting_flag 的值。

如果 slice_weighting_flag 和 mb_weighting_flag 的值均为“1”,则:

- 如果当前宏块的位流中存在 weighting_prediction,则当前宏块的 WeightingPrediction 等于 weighting_prediction;
- 如果当前宏块的位流中不存在 weighting_prediction,则当前宏块的 WeightingPrediction 等于“0”。

9.4.2 宏块类型

宏块类型变量 MbType 和宏块子类型变量 MbPartType 的解码过程如下:

- 如果当前图像是 I 帧:

- 如果满足以下条件之一,则 MbType 等于“I_8x8”(称为 I 宏块类型),MvNum 等于 0:
——PictureStructure 的值为“1”。

- PictureStructure 的值为“0”并且 $MbIndex < MbWidth \times MbHeight / 2$ 。
- 2) 如果 PictureStructure 的值为“0”并且 $MbIndex \geq MbWidth \times MbHeight / 2$ ：
- aec_enable 的值为“0”时，如果 skip_mode_flag 的值为“1”，MbTypeIndex 等于 mb_type 加 1；否则 MbTypeIndex 等于 mb_type。如果 MbTypeIndex 大于或等于 5，则 mb_type 中包含的 CBP 信息 CBPCodeNum 等于 MbTypeIndex 减 5，然后令 MbTypeIndex 等于 5。
- aec_enable 的值为“1”时，mb_type 和 MbTypeIndex 的关系见表 54。
- b) 如果当前图像是 P 帧：
- 1) aec_enable 的值为“0”时，如果 skip_mode_flag 的值为“1”，MbTypeIndex 等于 mb_type 加 1；否则 MbTypeIndex 等于 mb_type。如果 MbTypeIndex 大于或等于 5，则 mb_type 中包含的 CBP 信息 CBPCodeNum 等于 MbTypeIndex 减 5，然后令 MbTypeIndex 等于 5。
- 2) aec_enable 的值为“1”时，mb_type 和 MbTypeIndex 的关系见表 54。
- c) 如果当前图像是 B 帧：
- 1) aec_enable 的值为“0”时，如果 skip_mode_flag 的值为“1”，MbTypeIndex 等于 mb_type 加 1；否则 MbTypeIndex 等于 mb_type。如果 MbTypeIndex 大于或等于 24，则 mb_type 中包含的 CBP 信息 CBPCodeNum 等于 MbTypeIndex 减 24，然后令 MbTypeIndex 等于 24。
- 2) aec_enable 的值为“1”时，如果 skip_mode_flag 的值为“1”，MbTypeIndex 等于 mb_type 加 1；否则 MbTypeIndex 等于 mb_type。

表 54 mb_type 和 MbTypeIndex 的关系

mb_type 的值	MbTypeIndex 的值	
	skip_mode_flag 的值为“1”	skip_mode_flag 的值为“0”
0	5	5
1	1	0
2	2	1
3	3	2
4	4	3
5	—	4

如果当前图像是 P 帧，或者当前图像是 I 帧并且 PictureStructure 的值为 0 同时 $MbIndex \geq MbWidth \times MbHeight / 2$ ，MbType 和 MvNum 的值见表 55；如果当前图像是 B 帧，MbType 和 MvNum 的值见表 56。如果 MbType 等于“B_8x8”，MbPartType 和 MbPartMvNum 的值用 mb_part_type 查表 57，MvNum 是该宏块所有块的 MbPartMvNum 的和。

表 55 P 帧的宏块类型

MbTypeIndex 的值	宏块类型(MbType)	运动矢量数(MvNum)	预测模式(MbPredMode)	宏块分类
0	P_Skip	0	前向	P 宏块类型
1	P_16x16	1	前向	P 宏块类型
2	P_16x8	2	前向	P 宏块类型

表 55 (续)

MbTypeIndex 的值	宏块类型(MbType)	运动矢量数(MvNum)	预测模式(MbPredMode)	宏块分类
3	P_8x16	2	前向	P 宏块类型
4	P_8x8	4	前向	P 宏块类型
5	I_8x8	0	无	I 宏块类型

表 56 B 帧的宏块类型

MbTypeIndex 的值	宏块类型(MbType)	运动矢量数(MvNum)	预测模式(MbPredMode)	宏块分类
0	B_Skip	0	双向	B 宏块类型
1	B_Direct_16x16	0	双向	B 宏块类型
2	B_Fwd_16x16	1	前向	B 宏块类型
3	B_Bck_16x16	1	后向	B 宏块类型
4	B_Sym_16x16	1	双向	B 宏块类型
5	B_Fwd_Fwd_16x8	2	上前、下前	B 宏块类型
6	B_Fwd_Fwd_8x16	2	左前、右前	B 宏块类型
7	B_Bck_Bck_16x8	2	上后、下后	B 宏块类型
8	B_Bck_Bck_8x16	2	左后、右后	B 宏块类型
9	B_Fwd_Bck_16x8	2	上前、下后	B 宏块类型
10	B_Fwd_Bck_8x16	2	左前、右后	B 宏块类型
11	B_Bck_Fwd_16x8	2	上后、下前	B 宏块类型
12	B_Bck_Fwd_8x16	2	左后、右前	B 宏块类型
13	B_Fwd_Sym_16x8	2	上前、下双	B 宏块类型
14	B_Fwd_Sym_8x16	2	左前、右双	B 宏块类型
15	B_Bck_Sym_16x8	2	上后、下双	B 宏块类型
16	B_Bck_Sym_8x16	2	左后、右双	B 宏块类型
17	B_Sym_Fwd_16x8	2	上双、下前	B 宏块类型
18	B_Sym_Fwd_8x16	2	左双、右前	B 宏块类型
19	B_Sym_Bck_16x8	2	上双、下后	B 宏块类型
20	B_Sym_Bck_8x16	2	左双、右后	B 宏块类型
21	B_Sym_Sym_16x8	2	上双、下双	B 宏块类型
22	B_Sym_Sym_8x16	2	左双、右双	B 宏块类型
23	B_8x8	0~4	前向、后向、双向	B 宏块类型
24	I_8x8	0	无	I 宏块类型

表 57 B_8x8 的宏块子类型

mb_part_type 的值	宏块子类型(MbPartType)	运动矢量数(MbPartMvNum)	预测模式(MbPartPredMode)
0	SB_Direct_8x8	0	双向
1	SB_Fwd_8x8	1	前向
2	SB_Bck_8x8	1	后向
3	SB_Sym_8x8	1	双向

表 55~表 57 定义的 MbType 和 MbPartType 中,有 Skip 字样的称为跳过模式,有 Direct 字样的称为直接模式,有 Sym 字样的称为对称模式。表 55~表 57 中的 MvNum 表示当前宏块在位流中的运动矢量数。对称模式是一种双向预测模式,此时位流中只包含前向参考索引和前向运动矢量,对称模式的后向参考索引和后向运动矢量的推导见 9.9.1。

9.4.3 相邻块

一个块 E 和它的相邻块 A、B、C 和 D 之间的空间位置如图 11 所示。图 11 中 E 的大小可以是 16×16 、 16×8 、 8×16 或 8×8 。块 A 是块 E 的左边块,块 B 是块 E 的上边块。设块 E 左上角样本在图像中的坐标是 (x_0, y_0) ,右上角样本在图像中的坐标是 (x_1, y_1) ,块 X(X 是 A、B、C 或 D)是表 58 给出的样本所在的编码块。一个宏块 K 和它的相邻块 A、B、C、D、F 和 G 之间的空间位置如图 12 所示。设宏块 K 左上角样本在图像中的坐标是 (x_0, y_0) ,右上角样本在图像中的坐标是 (x_1, y_1) ,块 Y(Y 是 A、B、C、D、F 和 G)是表 58 给出的样本所在的编码块。表 58 中坐标均为样本在图像中的坐标。

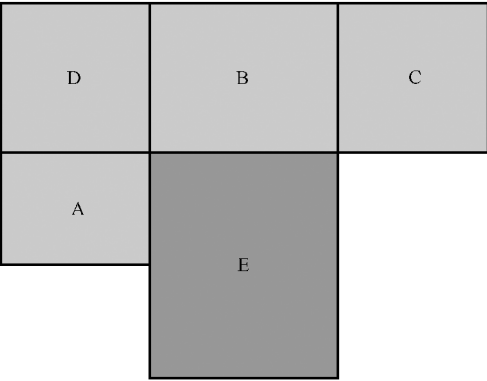


图 11 块 E 和相邻块的空间位置关系

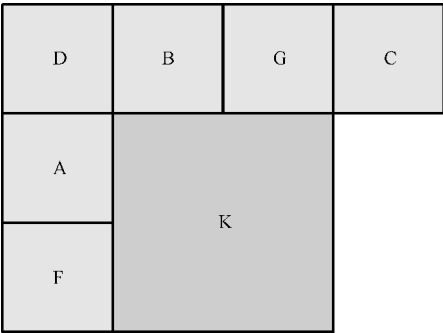


图 12 宏块 K 和相邻块的空间位置关系

表 58 相邻块的位置

PictureStructure 的值	块 A 右上角 样本位置	块 B 左下角 样本位置	块 C 左下角 样本位置	块 D 右下角 样本位置	块 F 右上角 样本位置	块 G 左下角 样本位置
0	(x_0-1, y_0)	(x_0, y_0-2)	(x_1+1, y_1-2)	(x_0-1, y_0-2)	(x_0-1, y_0+16)	(x_0+8, y_0-2)
1	(x_0-1, y_0)	(x_0, y_0-1)	(x_1+1, y_1-1)	(x_0-1, y_0-1)	(x_0-1, y_0+8)	(x_0+8, y_0-1)

相邻块 X(X 为 A、B、C 或 D)及相邻块 Y(Y 为 F 或 G)“存在”指该块应在图像内并且该块应与块 E 或 K 属于同一条带;否则相邻块“不存在”。

如果块“不存在”或者尚未解码,则此块“不可用”;否则此块“可用”。如果图像样本所在的块“不存在”或者此样本尚未解码,则此样本“不可用”;否则此样本“可用”。

9.4.4 帧内预测模式

9.4.4.1 概述

当前宏块的每一个 8×8 块使用 9.4.4.2 定义的方法确定其预测模式。

9.4.4.2 8×8 帧内预测模式

8×8 块采用以下方法确定其预测模式:

a) 如果当前块 E 是亮度块:

1) 计算当前块预测模式的预测值:

——如果左边块 A“存在”并且是帧内预测块,则将左边块的帧内预测模式赋值给 intraPredModeA;否则 intraPredModeA 等于“-1”。

——如果上边块 B“存在”并且是帧内预测块,则将上边块的帧内预测模式赋值给 intraPredModeB;否则 intraPredModeB 等于“-1”。

——如果 intraPredModeA 或 intraPredModeB 等于“-1”,则 predIntraPredMode 等于“2”;否则 predIntraPredMode 等于 $\text{Min}(\text{intraPredModeA}, \text{intraPredModeB})$ 。

2) aec_enable 的值为“1”时,置 BypassFlag 的值为“0”,解析得到 intra_luma_pred_mode。如果 intra_luma_pred_mode 的值为“0”,则 IntraLumaPredMode 等于 predIntraPredMode;否则:

——如果 intra_luma_pred_mode 的值为“4”,则 intra_luma_pred_mode 的值置为“0”。

——如果 intra_luma_pred_mode 小于 predIntraPredMode,则 IntraLumaPredMode 等于 intra_luma_pred_mode;否则 IntraLumaPredMode 等于 intra_luma_pred_mode 加 1。

3) aec_enable 的值为“0”时,如果 pred_mode_flag 的值为“1”,则预测模式 IntraLumaPredMode 等于 predIntraPredMode;否则,如果 intra_luma_pred_mode 小于 predIntraPredMode,则 IntraLumaPredMode 等于 intra_luma_pred_mode;否则 IntraLumaPredMode 等于 intra_luma_pred_mode 加 1。

b) 如果当前块 E 是色度块,块的顺序号为 4、5 时预测模式 IntraChromaPredMode 等于 intra_chroma_pred_mode;块的顺序号为 6、7 时 IntraChromaPredMode 等于 intra_chroma_pred_mode_422。

IntraLumaPredMode 的值与 8×8 亮度块预测模式的关系见表 59。IntraChromaPredMode 的值与 8×8 色度块预测模式的关系见表 60。

表 59 8×8 亮度块帧内预测模式

IntraLumaPredMode 的值	帧内预测模式
0	Intra_8x8_Vertical
1	Intra_8x8_Horizontal
2	Intra_8x8_DC
3	Intra_8x8_Down_Left
4	Intra_8x8_Down_Right

表 60 8×8 色度块帧内预测模式

IntraChromaPredMode 的值	帧内预测模式
0	Intra_Chroma_DC
1	Intra_Chroma_Horizontal
2	Intra_Chroma_Vertical
3	Intra_Chroma_Plane

表 59 所示的 8×8 亮度块帧内预测模式见图 13。

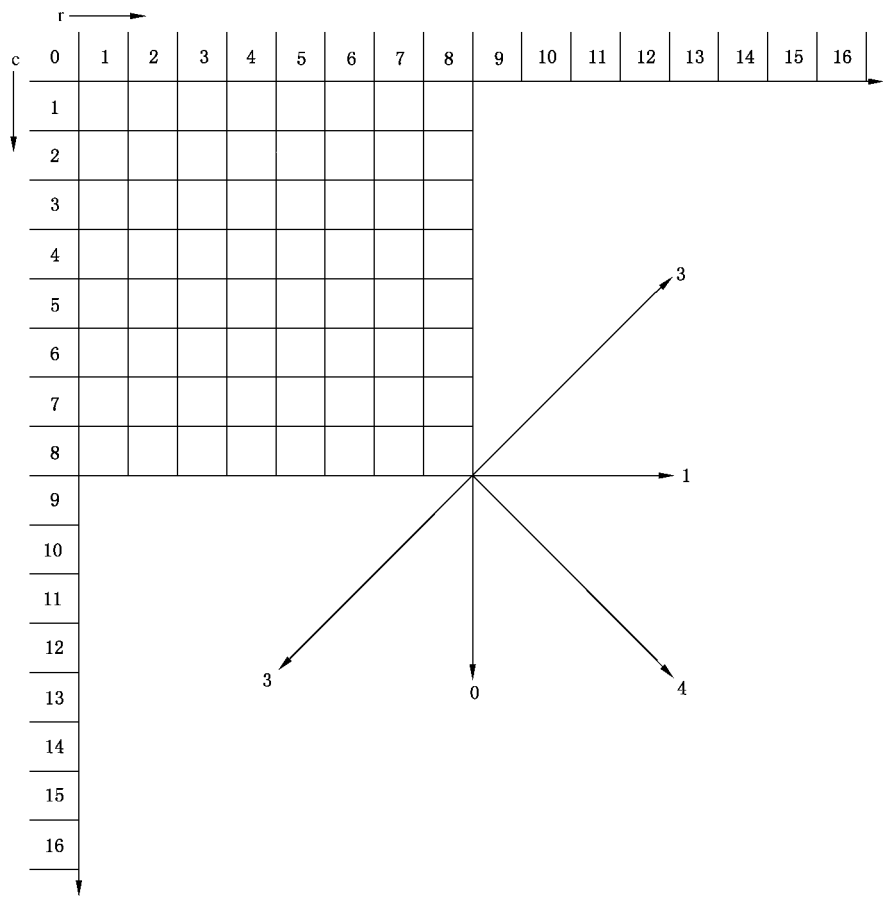


图 13 8×8 亮度块帧内预测模式

9.4.5 参考图像选择

每帧图像使用的参考图像除该帧本身外不应超过两个,它们应是最近解码的 I 帧或 P 帧。
参考索引值用来确定对当前图像进行解码处理所用的参考图像,参考索引的取值范围是 0~3。参考图像的两场可有不同的参考索引值。参考索引值随着参考图像与当前图像距离(显示顺序)的增加而增加,参考索引值为“0”的参考图像与当前图像的距离最近,参考索引值为 1 的图像距离其次,参考索引值为 3 的图像距离最远。前向预测和后向预测的参考索引值单独处理。参考图像(或场)的参考索引值的标记方法如下[图 14~图 20 中的数字表示参考索引值,箭头所指的是参考图像(或场)]:

- a) 如果当前图像是 I 帧并且 PictureStructure 等于“0”,同时当前解码场在显示顺序上是第二场,标记方法如图 14 所示。此时参考图像(或场)的个数 NumberOfReference 等于“1”。
- b) 如果当前图像是 P 帧并且 PictureStructure 等于“1”,标记方法如图 15 所示。此时 NumberOfReference 等于“2”。
- c) 如果当前图像是 P 帧并且 PictureStructure 等于“0”,同时当前解码场在显示顺序上是第一场,标记方法如图 16 所示。此时 NumberOfReference 等于“4”。
- d) 如果当前图像是 P 帧并且 PictureStructure 等于“0”,同时当前解码场在显示顺序上是第二场,标记方法如图 17 所示。此时 NumberOfReference 等于“4”。
- e) 如果当前图像是 B 帧并且 PictureStructure 等于“1”,标记方法如图 18 所示。此时 NumberOfReference 等于“2”。
- f) 如果当前图像是 B 帧并且 PictureStructure 等于“0”,同时当前解码场在显示顺序上是第一场,标记方法如图 19 所示。此时 NumberOfReference 等于“4”。
- g) 如果当前图像是 B 帧并且 PictureStructure 等于“0”,同时当前解码场在显示顺序上是第二场,标记方法如图 20 所示。此时 NumberOfReference 等于“4”。



图 14 参考索引标记方法 1

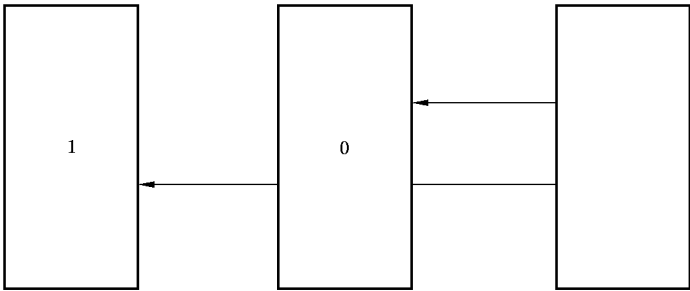


图 15 参考索引标记方法 2

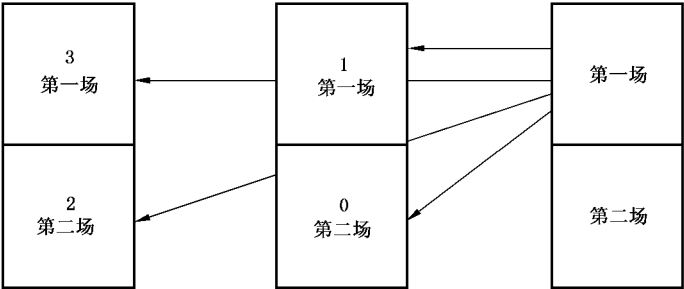


图 16 参考索引标记方法 3

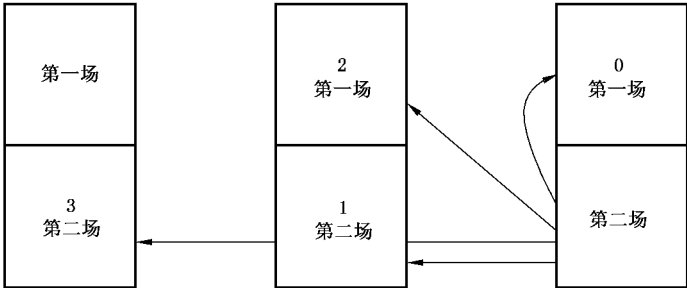


图 17 参考索引标记方法 4

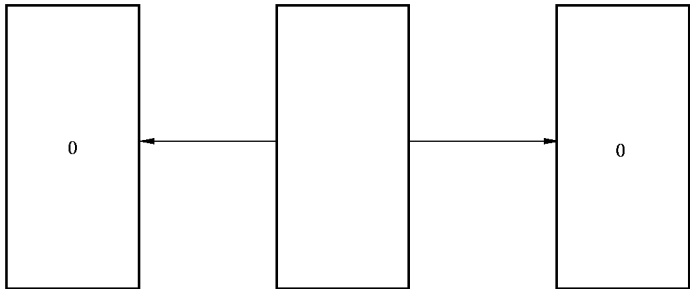


图 18 参考索引标记方法 5

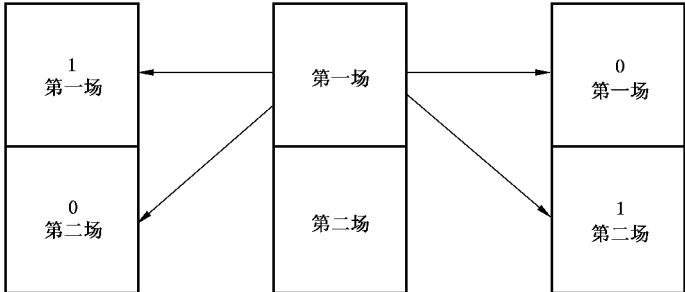


图 19 参考索引标记方法 6

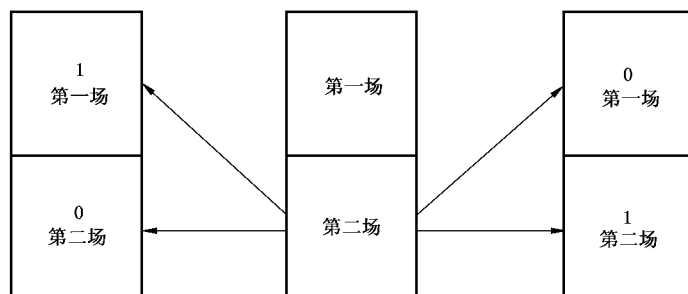


图 20 参考索引标记方法 7

如果当前图像的位流中不出现 `mb_reference_index`, 参考索引值为 0; 否则参考索引值等于 `mb_reference_index` 的值。

如果使用加权预测, 解码的加权预测参数按参考索引值由小到大依次分配给各参考图像(或场)。如果当前图像是 B 帧, 对参考索引值相同的前后参考图像(或场), 解码的加权预测参数先分配给前向参考图像(或场), 再分配给后向参考图像(或场)。

9.4.6 运动矢量

9.4.6.1 概述

块的距离索引 `DistanceIndex` 定义如下: 如果块的所有像素都属于所在隔行扫描图像的第二场(显示顺序)或者都属于所在逐行扫描图像的底场, `DistanceIndex` 等于 `picture_distance` 乘 2 加 1; 否则 `DistanceIndex` 等于 `picture_distance` 乘 2。

当前块(属于当前图像)和它的运动矢量所指向的参考块(属于参考图像)之间的距离 `BlockDistance` 计算如下:

- 如果参考块在当前块之前(显示顺序), `BlockDistance` 等于当前块的 `DistanceIndex` 减去参考块的 `DistanceIndex` 的差加上 512 的和模 512。
- 如果参考块在当前块之后(显示顺序), `BlockDistance` 等于参考块的 `DistanceIndex` 减去当前块的 `DistanceIndex` 的差加上 512 的和模 512。

9.4.6.2 亮度运动矢量预测

图 11 中块 A、B、C、D 的原始运动矢量或转换后的运动矢量为 `mvA`、`mvB`、`mvC`、`mvD`, 对应的 `BlockDistance` 为 `BlockDistanceA`、`BlockDistanceB`、`BlockDistanceC`、`BlockDistanceD`, 对应的参考索引为 `referenceIndexA`、`referenceIndexB`、`referenceIndexC`、`referenceIndexD`。

- 如果 A“不可用”或者采用帧内预测模式, 或者与当前块 E 没有同一预测方向的运动矢量, `mvA` 为零矢量, `BlockDistanceA` 等于“1”, A 的参考索引值为“−1”。
- 如果 B“不可用”或者采用帧内预测模式, 或者与当前块 E 没有同一预测方向的运动矢量, `mvB` 为零矢量, `BlockDistanceB` 等于“1”, B 的参考索引值为“−1”。
- 如果 D“不可用”或者采用帧内预测模式, 或者与当前块 E 没有同一预测方向的运动矢量, `mvD` 为零矢量, `BlockDistanceD` 等于“1”, D 的参考索引值为“−1”。
- 如果 C“不可用”, 那么 `mvC` 等于 `mvD`, `BlockDistanceC` 等于 `BlockDistanceD`, C 的参考索引值等于 D 的参考索引值。
- 如果 C 采用帧内预测模式, 或者与当前块 E 没有同一预测方向的运动矢量, `mvC` 为零矢量, `BlockDistanceC` 等于“1”, C 的参考索引值为“−1”。

当前块 E 的运动矢量预测值 MVEPred 计算过程如下：

- a) 第一步,如果 A、B、C 三者中只有一个块 X 的参考索引值不为“-1”,那么 MVEPred 等于 mvX(X 为 A、B 或 C);否则进行第二步。
- b) 第二步,如果 E 所在宏块按 16×8 或 8×16 模式编码,计算过程如下(见图 21):
 - 1) 8×16 模式:
 - E 为左块:如果 A 和 E 的参考索引值相同,MVEPred 等于 mvA;否则进行第三步。
 - E 为右块:如果 C 和 E 的参考索引值相同,MVEPred 等于 mvC;否则进行第三步。
 - 2) 16×8 模式:
 - E 为上块:如果 B 和 E 的参考索引值相同,MVEPred 等于 mvB;否则进行第三步。
 - E 为下块:如果 A 和 E 的参考索引值相同,MVEPred 等于 mvA;否则进行第三步。

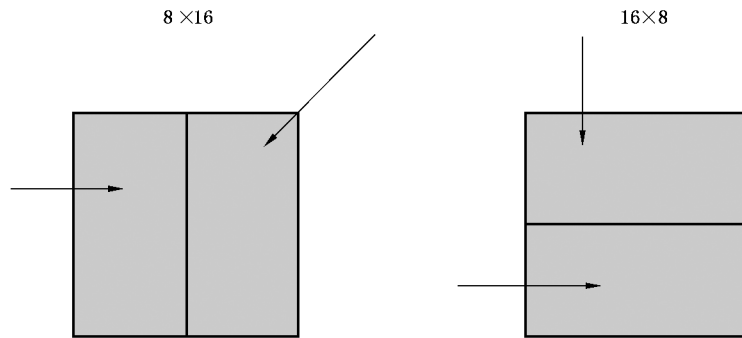


图 21 8×16 或 16×8 模式预测

- c) 第三步,根据 BlockDistanceA、BlockDistanceB、BlockDistanceC、BlockDistanceE 对 mvA(mvA_x, mvA_y)、mvB(mvB_x, mvB_y)、mvC(mvC_x, mvC_y) 进行缩放,得到 MVA(MVA_x, MVA_y)、MVB(MVB_x, MVB_y)、MVC(MVC_x, MVC_y)。其中 BlockDistanceE 是当前块 E 对应的 BlockDistance。

计算 MVA(MVA_x, MVA_y)、MVB(MVB_x, MVB_y)、MVC(MVC_x, MVC_y):

- 1) 如果 BlockDistanceA 等于 0, $MVA_x = mvA_x$, $MVA_y = mvA_y$;
- 2) 否则,

$$MVA_x = \text{Sign}(mvA_x) \times ((\text{Abs}(mvA_x) \times \text{BlockDistanceE} \times (512/\text{BlockDistanceA}) + 256) \gg 9)$$

$$MVA_y = \text{Sign}(mvA_y) \times ((\text{Abs}(mvA_y) \times \text{BlockDistanceE} \times (512/\text{BlockDistanceA}) + 256) \gg 9)$$
- 3) 如果 BlockDistanceB 等于 0, $MVB_x = mvB_x$, $MVB_y = mvB_y$;
- 4) 否则,

$$MVB_x = \text{Sign}(mvB_x) \times ((\text{Abs}(mvB_x) \times \text{BlockDistanceE} \times (512/\text{BlockDistanceB}) + 256) \gg 9)$$

$$MVB_y = \text{Sign}(mvB_y) \times ((\text{Abs}(mvB_y) \times \text{BlockDistanceE} \times (512/\text{BlockDistanceB}) + 256) \gg 9)$$
- 5) 如果 BlockDistanceC 等于 0, $MVC_x = mvC_x$, $MVC_y = mvC_y$;
- 6) 否则,

$$MVC_x = \text{Sign}(mvC_x) \times ((\text{Abs}(mvC_x) \times \text{BlockDistanceE} \times (512/\text{BlockDistanceC}) + 256) \gg 9)$$

$$\text{MVC}_y = \text{Sign}(\text{mvC}_y) \times ((\text{Abs}(\text{mvC}_y) \times \text{BlockDistanceE} \times (512/\text{BlockDistanceC}) + 256) >> 9)$$

d) 第四步,定义距离 $\text{Dist}(\text{MV1}, \text{MV2}) = \text{Abs}(x1 - x2) + \text{Abs}(y1 - y2)$,其中运动矢量 $\text{MV1} = [x1, y1]$, $\text{MV2} = [x2, y2]$ 。定义 VAB 等于 $\text{Dist}(\text{MVA}, \text{MVB})$, VBC 等于 $\text{Dist}(\text{MVB}, \text{MVC})$, VCA 等于 $\text{Dist}(\text{MVC}, \text{MVA})$ 。MVEPred 的计算如下:

- 1) 计算 FMV 等于 $\text{Median}(\text{VAB}, \text{VBC}, \text{VCA})$;
- 2) 如果 FMV 和 VAB 相等, MVEPred 等于 MVC ;
- 3) 否则,如果 FMV 和 VBC 相等, MVEPred 等于 MVA ;
- 4) 否则, MVEPred 等于 MVB 。

9.4.6.3 亮度运动矢量解码

当前块 E 的运动矢量 mvE 等于 MVEPred 加上由 mv_diff_x 和 mv_diff_y 解码得到的运动矢量增量的和。运动矢量基本单位为 $1/4$ 样本。当前块 E 的运动矢量应符合 6.2.5 的范围规定。

9.4.7 宏块编码模板

当 aec_enable 的值为“0”时,如果当前宏块的 cbp 在位流中,解析 cbp 得到 CodeNum ;否则,如果当前宏块的类型不等于 P_Skip 或 B_Skip ,令 CodeNum 的值等于 CBPCodeNum (见 9.4.2)。由 CodeNum 查表 42 得到 MbCBP (见 8.2)。 MbCBP 表示宏块中顺序号为 $0 \sim 5$ 的 4 个 8×8 亮度块和 2 个 8×8 色度块是否包含非零变换系数。 MbCBP 的第 n 位为“0”表示顺序号为 n 的 8×8 块没有非零系数,等于“1”表示该 8×8 块至少有一个非零系数(顺序号见 6.5)。

当 aec_enable 的值为“1”时,如果当前宏块的 cbp 在位流中,解析 cbp 得到 MbCBP (见 8.4)。 MbCBP 表示宏块中顺序号为 $0 \sim 5$ 的 4 个 8×8 亮度块和 2 个 8×8 色度块是否包含非零变换系数。 MbCBP 的第 n 位为“0”表示顺序号为 n 的 8×8 块没有非零系数,等于“1”表示该 8×8 块至少有一个非零系数(顺序号见 6.5)。

当 chroma_format 的值为“10”并且 aec_enable 的值为“0”时,如果当前宏块的 cbp_422 在位流中,解析 cbp_422 得到 CodeNum 。由 CodeNum 查表 43 得到 MbCBP422 (见 8.2)。 MbCBP422 表示 $4:2:2$ 格式的宏块中顺序号为 6 和 7 的 2 个 8×8 色度块是否包含非零变换系数。 MbCBP422 的第 n 位为“0”表示顺序号为 $n+6$ 的 8×8 色度块没有非零系数,等于“1”表示该 8×8 色度块至少有一个非零系数(顺序号见 6.5)。

当 chroma_format 的值为“10”并且 aec_enable 的值为“1”时,如果当前宏块的 cbp_422 在位流中,解析 cbp_422 得到 MbCBP422 (见 8.2)。 MbCBP422 表示 $4:2:2$ 格式的宏块中顺序号为 6 和 7 的 2 个 8×8 色度块是否包含非零变换系数。 MbCBP422 的第 n 位为“0”表示顺序号为 $n+6$ 的 8×8 色度块没有非零系数,等于“1”表示该 8×8 色度块至少有一个非零系数(顺序号见 6.5)。

9.4.8 量化参数

如果当前宏块的 mb_qp_delta 不在位流中,当前宏块的量化参数 CurrentQP 等于 PreviousQP 。

如果 aec_enable 的值为“0”并且当前宏块的 mb_qp_delta 在位流中,则解析 mb_qp_delta 得到 mbQpDelta , CurrentQP 等于 PreviousQP 加上 mbQpDelta 。

如果 aec_enable 的值为“1”并且当前宏块的 mb_qp_delta 在位流中,则置 BypassFlag 的值为“0”,解析 mb_qp_delta 得到 mbQpDelta 。如果 mbQpDelta 的值不为“0”,则按以下方式计算:

$$\begin{aligned} & \text{if}(\text{mbQpDelta} \% 2 = 0) \\ & \quad \text{mbQpDelta} = -(\text{mbQpDelta} + 1) / 2 \end{aligned}$$

else

$$\text{mbQpDelta} = (\text{mbQpDelta} + 1) / 2$$

CurrentQP 等于 PreviousQP 加上 mbQpDelta。

宏块解码完成后,将 PreviousQP 的值设为 CurrentQP,将 PreviousDeltaQP 的值设为 mbQpDelta。mbQpDelta 的取值范围应是 $-32 \sim 31$ 。CurrentQP 的取值范围应是 $0 \sim 63$ 。

9.4.9 加权量化矩阵

如果 weighting_quant_flag 的值为“1”,加权量化矩阵 WeightingQuantMatrix8x8 等于 wqM8x8 [QMode]。其中,QMode 等于 weighting_quant_param_index 的值。

9.5 块解码

9.5.1 基本熵编码的解码

基本熵编码的解码过程定义了 aec_enable 的值为“0”时由语法元素生成量化系数值数组(Level 数组)和量化系数游程数组(Run 数组)的过程。Level 数组的元素为非 0 量化系数的幅值,Run 数组的元素为当前非 0 量化系数前的连续 0 的个数。生成量化系数数组所需语法元素的解析过程见 8.3。

生成 Run 数组和 Level 数组的过程如下(数组地址初始值为 0,maxAbsLevel 初始值为 0)：

- a) 如果 trans_coefficient 小于“59”：
 - 1) 如果 CurrentVLCTable 中包含此 trans_coefficient 值的索引项,则以 trans_coefficient 为索引查 CurrentVLCVTable,得到量化系数和游程,把它们分别放入 Level 数组和 Run 数组；
 - 2) 如果表中不包含此 trans_coefficient 值的索引项,则以(trans_coefficient-1)为索引查 CurrentVLCTable,得到量化系数和游程,absLevel 等于量化系数的绝对值,并将量化系数符号取反后放入 Level 数组,将游程放入 Run 数组。
- b) 如果 trans_coefficient 大于或等于“59”：
 - 1) 将(trans_coefficient-59)/2 放入 Run 数组；
 - 2) 由 CurrentVLCTable 查表 D.20 得到 MaxRun。如果(trans_coefficient-59)/2 大于 MaxRun,RefAbsLevel 等于“1”；否则以(trans_coefficient-59)/2 为索引查 CurrentVLCTable 得到 RefAbsLevel；
 - 3) 如果 trans_coefficient 是奇数,将-(RefAbsLevel+escape_level_diff)放入 Level 数组；否则,将(RefAbsLevel+escape_level_diff)放入 Level 数组。absLevel 等于(RefAbsLevel+escape_level_diff)。
- c) 如果 trans_coefficient 等于“EOB”,结束块系数解码；否则数组地址加 1,首先按 8.3 定义的方法更新 CurrentVLCTable,然后如果 absLevel 大于 maxAbsLevel,则 maxAbsLevel 等于 absLevel,最后解码下一个系数值和游程。

9.5.2 高级熵编码的解码

本条定义 aec_enable 的值为“1”时由语法元素生成量化系数值数组(Level 数组)和量化系数游程数组(Run 数组)的过程。Level 数组的元素为非 0 量化系数的幅值,Run 数组的元素为当前非 0 量化系数前的连续 0 的个数。生成量化系数数组所需语法元素的解析过程见 8.4。

生成 Run 数组和 Level 数组的过程如下(数组地址初始值为“0”):

- a) 由 trans_coefficient 解析得到 AbsLevel。如果是块的第一个 AbsLevel,则 $\text{AbsLevel} = \text{AbsLevel} + 1$ 。
- b) 如果 AbsLevel 不等于“0”:
 - 1) level 等于 AbsLevel;
 - 2) 由 trans_coefficient 解析得到 coeffSign。如果 coeffSign 为“0”,将 level 放入 Level 数组;否则,将 -level 放入 Level 数组;
 - 3) 由 trans_coefficient 解析得到 RunVal,将 RunVal 放入 Run 数组。
- c) 如果 AbsLevel 等于“0”,结束块系数解码(即 trans_coefficient 等于 EOB);否则数组地址加 1,解码下一个系数值和游程。

9.5.3 逆扫描

由解码的 Level 数组和 Run 数组生成数组 QuantCoeffArray(8×8 块包含 64 个量化系数)的步骤如下:

- a) 首先,将 QuantCoeffArray 数组的所有元素初始化为“0”。
- b) 第二步,将非 0 量化系数的值赋给 QuantCoeffArray 数组中相应的元素。定义 j 和 coeffNum,令 j 等于 9.5.1 或 9.5.2 中 Run 数组或 Level 数组中最后一个赋值元素的地址索引,coeffNum 等于“-1”,


```
while( $j \geq 0$ )
{
    coeffNum += (Run[j] + 1)
    QuantCoeffArray[coeffNum] = Level[j]
    j--
}
```
- c) 第三步,确定逆块扫描方法:
 - 1) 如果 progressive_frame 的值为“1”,或者 progressive_frame 的值为“0”并且 picture_structure 的值为“1”,逆块扫描方法 1 见图 22;
 - 2) 如果 progressive_frame 的值为“0”并且 picture_structure 的值为“0”,逆块扫描方法 2 见图 23。
- d) 第四步,通过逆块扫描将 QuantCoeffArray 数组映射为 QuantCoeffMatrix 数组。设 QuantCoeffArray 数组中某元素的地址为 k ,在扫描表中找到值为 k 的单元对应的列号 i 和行号 j ,然后将 QuantCoeffArray[k]赋给 QuantCoeffMatrix[i, j]。

	0	1	2	3	4	5	6	7	<i>i</i>
0	0	1	5	6	14	15	27	28	
1	2	4	7	13	16	26	29	42	
2	3	8	12	17	25	30	41	43	
3	9	11	18	24	31	40	44	53	
4	10	19	23	32	39	45	52	54	
5	20	22	33	38	46	51	55	60	
6	21	34	37	47	50	56	59	61	
7	35	36	48	49	57	58	62	63	
<i>j</i>									

图 22 8×8 逆块扫描方法 1

	0	1	2	3	4	5	6	7	<i>i</i>
0	0	3	11	16	22	32	38	55	
1	1	6	12	20	25	33	42	57	
2	2	7	15	21	28	37	43	58	
3	4	10	19	27	31	39	47	59	
4	5	14	24	30	36	44	50	60	
5	8	17	26	35	41	48	52	61	
6	9	18	29	40	46	51	54	62	
7	13	23	34	45	49	53	56	63	
<i>j</i>									

图 23 8×8 逆块扫描方法 2

9.6 反量化

9.6.1 确定量化参数

本条确定的亮度量化参数和色度量化参数的取值范围应是 0~63。

如果当前块是 8×8 亮度块,该块的量化参数 QP 等于其所在宏块的 CurrentQP。

如果当前块是色度块,确定 Cb、Cr 色度块的 CurrentQPCb、CurrentQPCr 计算如下:

$CurrentQPCb = CurrentQP + chroma_quant_param_delta_cb$

$CurrentQPCr = CurrentQP + chroma_quant_param_delta_cr$

然后分别以 CurrentQPCb 和 CurrentQPCr 为索引查表 61 得到 Cb、Cr 色度块的量化参数 QP。

表 61 色度块的 QP 与 CurrentQPCb、CurrentQPCr 的映射关系

CurrentQPCb、CurrentQPCr 的值	QP 的值
< 43	CurrentQPCb、CurrentQPCr
43	42
44	43
45	43
46	44
47	44
48	45
49	45
50	46
51	46
52	47
53	47
54	48
55	48
56	48
57	49
58	49
59	49
60	50
61	50
62	50
63	51

9.6.2 反量化过程

本条定义将二维量化系数矩阵 QuantCoeffMatrix 转换为二维变换系数矩阵 CoeffMatrix 的过程。

从符合本部分的位流中解码得到的 8×8 块量化系数的取值范围应为 $-2^{n+3} \sim 2^{n+3} - 1$ (n 是样本点精

度)。

8×8 块的二维变换系数矩阵 $\text{CoeffMatrix}_{8 \times 8}$ 计算如下：

$$\begin{aligned} \text{CoeffMatrix}_{8 \times 8}[i, j] = & (((((\text{QuantCoeffMatrix}[i, j] \times \text{WeightingQuantMatrix}_{8 \times 8}[i, j]) \\ & \gg 3) \times \text{DequantTable}(\text{QP})) \gg 4) + 2^{\text{ShiftTable}(\text{QP}-1)}) \gg \\ & \text{ShiftTable}(\text{QP}) \quad i, j = 0 \sim 7 \end{aligned}$$

从符合本部分的位流中解码得到的 $\text{QuantCoeffMatrix}[i, j] (i, j = 0 \sim 7)$ 的值的范围应为 $-2^{n+3} \sim 2^{n+3} - 1$ (n 是样本点精度)。

QP 与 DequantTable 和 ShiftTable 的关系见表 62。

表 62 QP 与 DequantTable 和 ShiftTable 的关系

QP 的值	DequantTable(QP)的值	ShiftTable(QP)的值
0	32 768	14
1	36 061	14
2	38 968	14
3	42 495	14
4	46 341	14
5	50 535	14
6	55 437	14
7	60 424	14
8	32 932	13
9	35 734	13
10	38 968	13
11	42 495	13
12	46 177	13
13	50 535	13
14	55 109	13
15	59 933	13
16	65 535	13
17	35 734	12
18	38 968	12
19	42 577	12
20	46 341	12
21	50 617	12
22	55 027	12
23	60 097	12
24	32 809	11
25	35 734	11
26	38 968	11

表 62 (续)

QP 的值	DequantTable(QP)的值	ShiftTable(QP)的值
27	42 454	11
28	46 382	11
29	50 576	11
30	55 109	11
31	60 056	11
32	65 535	11
33	35 734	10
34	38 968	10
35	42 495	10
36	46 320	10
37	50 515	10
38	55 109	10
39	60 076	10
40	65 535	10
41	35 744	9
42	38 968	9
43	42 495	9
44	46 341	9
45	50 535	9
46	55 099	9
47	60 087	9
48	65 535	9
49	35 734	8
50	38 973	8
51	42 500	8
52	46 341	8
53	50 535	8
54	55 109	8
55	60 097	8
56	32 771	7
57	35 734	7
58	38 965	7
59	42 497	7
60	46 341	7
61	50 535	7
62	55 109	7
63	60 099	7

从符合本部分的位流中解码得到的 CoeffMatrix8x8 的元素的取值范围应为 $-2^{n+5} \sim 2^{n+5} - 1$ (n 是样本点精度)。

9.7 反变换

本条定义将 8×8 变换系数矩阵 CoeffMatrix8x8 转换为 8×8 残差样值矩阵 ResidueMatrix 的过程,步骤如下:

- a) 首先,对变换系数矩阵进行如下水平反变换:

$$H' = \text{CoeffMatrix8x8} \times T_8^T$$

其中, T_8 是 8×8 反变换矩阵, T_8^T 是 T_8 的转置矩阵, H' 表示水平反变换后的中间结果。

$$T_8 = \begin{bmatrix} 8 & 10 & 10 & 9 & 8 & 6 & 4 & 2 \\ 8 & 9 & 4 & -2 & -8 & -10 & -10 & -6 \\ 8 & 6 & -4 & -10 & -8 & 2 & 10 & 9 \\ 8 & 2 & -10 & -6 & 8 & 9 & -4 & -10 \\ 8 & -2 & -10 & 6 & 8 & -9 & -4 & 10 \\ 8 & -6 & -4 & 10 & -8 & -2 & 10 & -9 \\ 8 & -9 & 4 & 2 & -8 & 10 & -10 & 6 \\ 8 & -10 & 10 & -9 & 8 & -6 & 4 & -2 \end{bmatrix}$$

- b) 第二步,矩阵 H' 的元素 h''_{ij} 计算如下:

$$h''_{ij} = \text{Clip3}(-2^{n+7}, 2^{n+7} - 1, (h'_{ij} + 4)) \gg 3 \quad i, j = 0 \sim 7; n \text{ 是样本点精度}$$

- c) 第三步,对矩阵 H'' 进行如下垂直反变换:

$$H = T_8 \times H''$$

其中, H 表示反变换后的 8×8 矩阵。

- d) 第四步,残差样值矩阵 ResidueMatrix 的元素 r_{ij} 计算如下:

$$r_{ij} = \text{Clip3}(-2^{n+7}, 2^{n+7} - 1, (h_{ij} + 2^6)) \gg 7 \quad i, j = 0 \sim 7; n \text{ 是样本点精度}$$

其中 h_{ij} 是 H 矩阵的元素。

9.8 帧内预测

9.8.1 概述

帧内预测完成后得到预测样本矩阵 predMatrix。应使用 9.8.2 定义的方法获得当前 8×8 块的参考样本,并应使用 9.8.3、9.8.4 定义的方法进行帧内预测。

当前 8×8 块由其上边和左边的参考样本 $r[i]$ ($i=0 \sim 16$) 和 $c[i]$ ($i=0 \sim 16$) 来预测 (r 、 c 可表示亮度或色度参考样本),见图 13,其中 $r[0]$ 等于 $c[0]$ 。如果帧内预测需要用到 i 大于 16 的上边和左边的参考样本,则 $r[i] = r[16]$, $c[i] = c[16]$ ($i > 16$)。

9.8.2 8×8 块参考样本的获得

如果当前块所在的图像的 PictureStructure 的值为“1”,则当前块所在图像的样本矩阵为 I;否则当前块所在的场的样本矩阵为 I。I 可表示亮度或色度矩阵。

设当前块左上角样本在样本矩阵 I 中的坐标是 (x_0, y_0) ,其参考样本按以下规则获得:

- 如果坐标为 $(x_0 + i - 1, y_0 - 1)$ ($i = 1 \sim 8$) 的样本“可用”,则 $r[i]$ 等于 $I[x_0 + i - 1, y_0 - 1]$, $r[i]$ “可用”;否则 $r[i]$ “不可用”;
- 如果坐标为 $(x_0 + i - 1, y_0 - 1)$ ($i = 9 \sim 16$) 的样本“可用”,则 $r[i]$ 等于 $I[x_0 + i - 1, y_0 - 1]$, $r[i]$

- “可用”；否则 $r[i]$ 等于 $r[8]$, $r[i]$ 是否“可用”由 $r[8]$ 是否“可用”决定；
- c) 如果坐标为 (x_0-1, y_0+i-1) ($i=1\sim 8$) 的样本“可用”，则 $c[i]$ 等于 $I[x_0-1, y_0+i-1]$, $c[i]$ “可用”；否则 $c[i]$ “不可用”；
- d) 如果坐标为 (x_0-1, y_0+i-1) ($i=9\sim 16$) 的样本“可用”，则 $c[i]$ 等于 $I[x_0-1, y_0+i-1]$, $c[i]$ “可用”；否则 $c[i]$ 等于 $c[8]$, $c[i]$ 是否“可用”由 $c[8]$ 是否“可用”决定；
- e) 如果坐标为 (x_0-1, y_0-1) 的样本“可用”，则 $r[0]$ 等于 $I[x_0-1, y_0-1]$, $r[0]$ “可用”；否则
- 1) 如果 $r[1]$ “可用”并且 $c[1]$ “不可用”，则 $r[0]$ 等于 $r[1]$, $r[0]$ “可用”；否则
 - 2) 如果 $c[1]$ “可用”并且 $r[1]$ “不可用”，则 $r[0]$ 等于 $c[1]$, $r[0]$ “可用”；否则
 - 3) 如果 $r[1]$ “可用”并且 $c[1]$ “可用”，则 $r[0]$ 等于 $r[1]$, $r[0]$ “可用”；否则 $r[0]$ “不可用”。

9.8.3 8×8 亮度块帧内预测

根据 IntraLumaPredMode 决定亮度块帧内预测方法。

- a) IntraLumaPredMode 等于“0”(Intra_8x8_Vertical 预测)
当 $r[i]$ ($i=1\sim 8$) “可用”时, 该模式才被使用, 此时
 $\text{predMatrix}[x, y] = r[x+1]$ ($x, y=0\sim 7$)
- b) IntraLumaPredMode 等于“1”(Intra_8x8_Horizontal 预测)
当 $c[i]$ ($i=1\sim 8$) “可用”时, 该模式才被使用, 此时
 $\text{predMatrix}[x, y] = c[y+1]$ ($x, y=0\sim 7$)
- c) IntraLumaPredMode 等于“2”(Intra_8x8_DC 预测)
- 1) 如果 $r[i], c[i]$ ($i=0\sim 9$) 都“可用”, 则
 $\text{predMatrix}[x, y] = ((r[x]+2\times r[x+1]+r[x+2]+2)>>2+(c[y]+2\times c[y+1]+c[y+2]+2)>>2)>>1$ ($x, y=0\sim 7$); 否则
 - 2) 如果 $r[i]$ ($i=0\sim 9$) “可用”, 则
 $\text{predMatrix}[x, y] = (r[x]+2\times r[x+1]+r[x+2]+2)>>2$ ($x, y=0\sim 7$); 否则
 - 3) 如果 $c[i]$ ($i=0\sim 9$) “可用”, 则
 $\text{predMatrix}[x, y] = (c[y]+2\times c[y+1]+c[y+2]+2)>>2$ ($x, y=0\sim 7$); 否则
 - 4) $\text{predMatrix}[x, y] = 2^{n-1}$ ($x, y=0\sim 7$; n 是样本点精度)。
- d) IntraLumaPredMode 等于“3”(Intra_8x8_Down_Left 预测)
当 $r[i], c[i]$ ($i=1\sim 16$) 均“可用”时, 该模式才被使用, 此时
 $\text{predMatrix}[x, y] = ((r[x+y+1]+2\times r[x+y+2]+r[x+y+3]+2)>>2+(c[x+y+1]+2\times c[x+y+2]+c[x+y+3]+2)>>2)>>1$ ($x, y=0\sim 7$)。
- e) IntraLumaPredMode 等于“4”(Intra_8x8_Down_Right 预测)
当 $r[i], c[i]$ ($i=0\sim 16$) 均“可用”时, 该模式才被使用, 此时
- 1) 如果 x 等于 y , $\text{predMatrix}[x, y] = (c[1]+2\times r[0]+r[1]+2)>>2$ ($x, y=0\sim 7$); 否则
 - 2) 如果 x 大于 y , $\text{predMatrix}[x, y] = (r[x-y+1]+2\times r[x-y]+r[x-y-1]+2)>>2$ ($x, y=0\sim 7$); 否则
 - 3) 如果 y 大于 x , $\text{predMatrix}[x, y] = (c[y-x+1]+2\times c[y-x]+c[y-x-1]+2)>>2$ ($x, y=0\sim 7$)。

9.8.4 8×8 色度块帧内预测

根据 IntraChromaPredMode 决定色度块帧内预测方法。

- a) IntraChromaPredMode 等于“0”(Intra_Chroma_DC 预测)
- 1) 如果 $r[i], c[i]$ ($i=0\sim 9$) 都“可用”, 则

$\text{predMatrix}[x, y] = ((r[x] + 2 \times r[x+1] + r[x+2] + 2) >> 2 + (c[y] + 2 \times c[y+1] + c[y+2] + 2) >> 2) >> 1 (x, y = 0 \sim 7)$; 否则

- 2) 如果 $r[i] (i = 0 \sim 9)$ “可用”, 则
 $\text{predMatrix}[x, y] = (r[x] + 2 \times r[x+1] + r[x+2] + 2) >> 2 (x, y = 0 \sim 7)$; 否则
 - 3) 如果 $c[i] (i = 0 \sim 9)$ “可用”, 则
 $\text{predMatrix}[x, y] = (c[y] + 2 \times c[y+1] + c[y+2] + 2) >> 2 (x, y = 0 \sim 7)$; 否则
 - 4) $\text{predMatrix}[x, y] = 2^{n-1} (x, y = 0 \sim 7; n \text{ 是样本点精度})$ 。
- b) IntraChromaPredMode 等于“1”(Intra_Chroma_Horizontal 预测)
 当 $c[i] (i = 1 \sim 8)$ “可用”时, 该模式才被使用, 此时
 $\text{predMatrix}[x, y] = c[y+1] (x, y = 0 \sim 7)$ 。
- c) IntraChromaPredMode 等于“2”(Intra_Chroma_Vertical 预测)
 当 $r[i] (i = 1 \sim 8)$ “可用”时, 该模式才被使用, 此时
 $\text{predMatrix}[x, y] = r[x+1] (x, y = 0 \sim 7)$ 。
- d) IntraChromaPredMode 等于“3”(Intra_Chroma_Plane 预测)
 当 $r[i], c[i] (i = 1 \sim 8)$ 均“可用”时, 该模式才被使用, 此时
 $\text{predMatrix}[x, y] = \text{Clip1}((ia + (x-3) \times ib + (y-3) \times ic + 16) >> 5) (x, y = 0 \sim 7)$ 。
 其中, $ia = (r[8] + c[8]) << 4, ib = (17 \times ih + 16) >> 5, ic = (17 \times iv + 16) >> 5,$
 $ih = \sum_{i=0}^3 (i+1) \times (r[5+i] - r[3-i]), iv = \sum_{i=0}^3 (i+1) \times (c[5+i] - c[3-i])$ 。

9.9 帧间预测

9.9.1 亮度运动矢量导出

如果当前宏块类型或者当前宏块子类型是跳过模式、直接模式或对称模式, 其运动矢量按照下面定义的方法导出(具体见图 24~图 29), 否则将 9.4.6 解码得到的运动矢量, 按照宏块划分顺序(在图 6 中定义)分配给相应的子块。

- a) 如果当前宏块类型为 P_Skip:
 - 1) 如果 PFieldSkip 的值等于 1, picture_coding_type 等于“01”, 并且 picture_structure 的值等于“0”, 当前块的前向参考图像为图 14~图 20 中标记为 1 的图像; 否则, 当前块的前向参考图像为缺省参考图像, 即图 14~图 20 中标记为 0 的图像;
 - 2) 如果当前宏块的上边宏块 B 或左边宏块 A “不可用”, mvE 等于零矢量;
 - 3) 通过 9.4.6.2 定义的方法得到 mvA、mvB, 如果 mvA 为零矢量并且宏块 A 的参考图像与当前块的参考图像相同, 或 mvB 为零矢量并且宏块 B 的参考图像与当前块的参考图像相同, mvE 等于零矢量;
 - 4) 对于其他情况, 通过 9.4.6.2 定义的方法得到 MVEPred, mvE = MVEPred。
- b) 如果当前宏块类型为 B_Skip 或 B_Direct_16x16, 或当前宏块子类型为 SB_Direct_8x8, 对每个 8×8 块分别进行以下步骤:
 - 1) 第一步:

——如果当前块符合以下条件之一, 则当前块的前后向参考图像均为缺省参考图像, 即图 14~图 20 中标记为 0 的图像, 当前块的前后向运动矢量是分别根据 9.4.6 得到的当前块所在宏块(9.4.6 中块 E 为当前块所在宏块)的前后向运动矢量预测值。

 - 当前块所在图像的 PictureStructure 等于 1, 且后向参考图像中与当前 8×8 块的左上角样本位置对应的样本所在的编码宏块类型为“I_8x8”。

- 当前块所在图像的 PictureStructure 等于 0,其左上角样本位置为 (x_0, y_0) 。
 - 如果 BFieldEnhanced 的值等于 1,且 $MbIndex \geq MbWidth \times MbHeight / 2$,并且在后向参考图像中与 $(x_0, y_0 - 2 \times top_field_first + 1)$ 样本位置对应的样本所在的编码块的宏块类型为“I_8x8”。
 - 如果 BFieldEnhanced 的值等于 0,且后向参考图像中与当前 8×8 块的左上角样本位置对应的样本所在的编码宏块类型为“I_8x8”。
 - $MbIndex < MbWidth \times MbHeight / 2$ 且后向参考图像中与当前 8×8 块的左上角样本位置对应的样本所在的编码宏块类型为“I_8x8”。
 - 否则,如果当前块所在图像的 PictureStructure 等于 1:
 - 当前块的前后向参考图像均为缺省参考图像,即图 18 中标记为 0 的图像,其前后向距离索引分别为 DistanceIndexFw 和 DistanceIndexBw;当前块的前后向 BlockDistance 分别为 BlockDistanceFw 和 BlockDistanceBw。
 - 在后向参考图像中与当前块的左上角样本位置对应的样本所在的编码块的运动矢量为 $mvRef(mvRef_x, mvRef_y)$,该编码块的距离索引为 DistanceIndexCol,该运动矢量指向的参考块的距离索引为 DistanceIndexRef。
 - 否则,如果当前块所在图像的 PictureStructure 等于 0:
 - 当前块的左上角样本位置为 (x_0, y_0) 。如果 BFieldEnhanced 的值等于 1 且 $MbIndex \geq MbWidth \times MbHeight / 2$,则在后向参考图像中与 $(x_0, y_0 - 2 \times top_field_first + 1)$ 样本位置对应的样本所在的编码块的运动矢量为 $mvRef(mvRef_x, mvRef_y)$,该编码块的距离索引为 DistanceIndexCol,该运动矢量指向的参考块的距离索引为 DistanceIndexRef;否则,在后向参考图像中与 (x_0, y_0) 样本位置对应的样本所在的编码块的运动矢量为 $mvRef(mvRef_x, mvRef_y)$,该编码块的距离索引为 DistanceIndexCol,该运动矢量指向的参考块的距离索引为 DistanceIndexRef。
 - 当前块的参考索引标记为 0 和 1 的前向参考场中的参考块的距离索引分别为 DistanceIndexFw0 和 DistanceIndexFw1。当 BFieldEnhanced 的值等于 0,或者当前块的 $MbIndex < MbWidth \times MbHeight / 2$ 时,如果 DistanceIndexRef 等于 DistanceIndexFw0,则当前块的前向参考场是参考索引标记为 0 的场,DistanceIndexFw 等于 DistanceIndexFw0;否则当前块的前向参考场是参考索引标记为 1 的场,DistanceIndexFw 等于 DistanceIndexFw1。当 BFieldEnhanced 的值等于 1,并且当前块的 $MbIndex \geq MbWidth \times MbHeight / 2$ 时,当前块的前向参考场是参考索引标记为 0 的场,DistanceIndexFw 等于 DistanceIndexFw0。
 - 当前块的参考索引标记为 0 和 1 的后向参考场中的参考块的距离索引分别为 DistanceIndexBw0 和 DistanceIndexBw1。当 BFieldEnhanced 的值等于 0 时,如果当前块的 $MbIndex < MbWidth \times MbHeight / 2$,则当前块的后向参考场是参考索引标记为 0 的场,DistanceIndexBw 等于 DistanceIndexBw0;否则当前块的后向参考场是参考索引标记为 1 的场,DistanceIndexBw 等于 DistanceIndexBw1。当 BFieldEnhanced 的值等于 1 时,当前块的后向参考场是参考索引标记为 0 的场,DistanceIndexBw 等于 DistanceIndexBw0。
- 2) 第二步:
- $BlockDistanceRef = (DistanceIndexCol - DistanceIndexRef + 512) \% 512$
- 当前块的距离索引为 DistanceIndexCur,则

$$\text{BlockDistanceFw} = (\text{DistanceIndexCur} - \text{DistanceIndexFw} + 512) \% 512$$

$$\text{BlockDistanceBw} = (\text{DistanceIndexBw} - \text{DistanceIndexCur} + 512) \% 512$$

——如果当前块所在图像的 PictureStructure 等于 1, 并且后向参考图像的 PictureStructure 等于 0, 则 $\text{mvRef}_y = \text{mvRef}_y \times 2$ 。

——如果当前块所在图像的 PictureStructure 等于 0, 并且后向参考图像的 PictureStructure 等于 1, 则 $\text{mvRef}_y = \text{mvRef}_y / 2$ 。

3) 第三步:

——计算 delta1, 初始化 deltaFw, deltaBw:

- 置 delta1, deltaFw, deltaBw 的值为“0”。
- 如果当前图像的 PictureStructure 的值为“0”, 并且 BFieldEnhanced 值为 1:

——如果后向参考场为顶场, mvRef 指向的场为底场, 则 $\text{delta1} = 2$ 。

——如果后向参考场为底场, mvRef 指向的场为顶场, 则 $\text{delta1} = -2$ 。

——当前块的前向运动矢量 mvFw(mvFw_x, mvFw_y) 为:

- 如果当前图像的 PictureStructure 的值为“0”, 并且 BFieldEnhanced 值为 1:

——如果当前块所在的场为顶场, 并且当前块的前向参考场为底场, 则 $\text{deltaFw} = 2$ 。

——如果当前块所在的场为底场, 并且当前块的前向参考场为顶场, 则 $\text{deltaFw} = -2$ 。

- 如果 mvRef_x 小于 0, 则 $\text{mvFw}_x = -(((16\ 384 / \text{BlockDistanceRef}) \times (1 - \text{mvRef}_x \times \text{BlockDistanceFw}) - 1) >> 14)$; 否则 $\text{mvFw}_x = (((16\ 384 / \text{BlockDistanceRef}) \times (1 + \text{mvRef}_x \times \text{BlockDistanceFw}) - 1) >> 14)$ 。

- 如果 $(\text{mvRef}_y + \text{delta1})$ 小于 0, 则 $\text{mvFw}_y = -(((16\ 384 / \text{BlockDistanceRef}) \times (1 - (\text{mvRef}_y + \text{delta1}) \times \text{BlockDistanceFw}) - 1) >> 14) - \text{deltaFw}$; 否则 $\text{mvFw}_y = (((16\ 384 / \text{BlockDistanceRef}) \times (1 + (\text{mvRef}_y + \text{delta1}) \times \text{BlockDistanceFw}) - 1) >> 14) - \text{deltaFw}$ 。

——当前块的后向运动矢量 mvBw(mvBw_x, mvBw_y) 为:

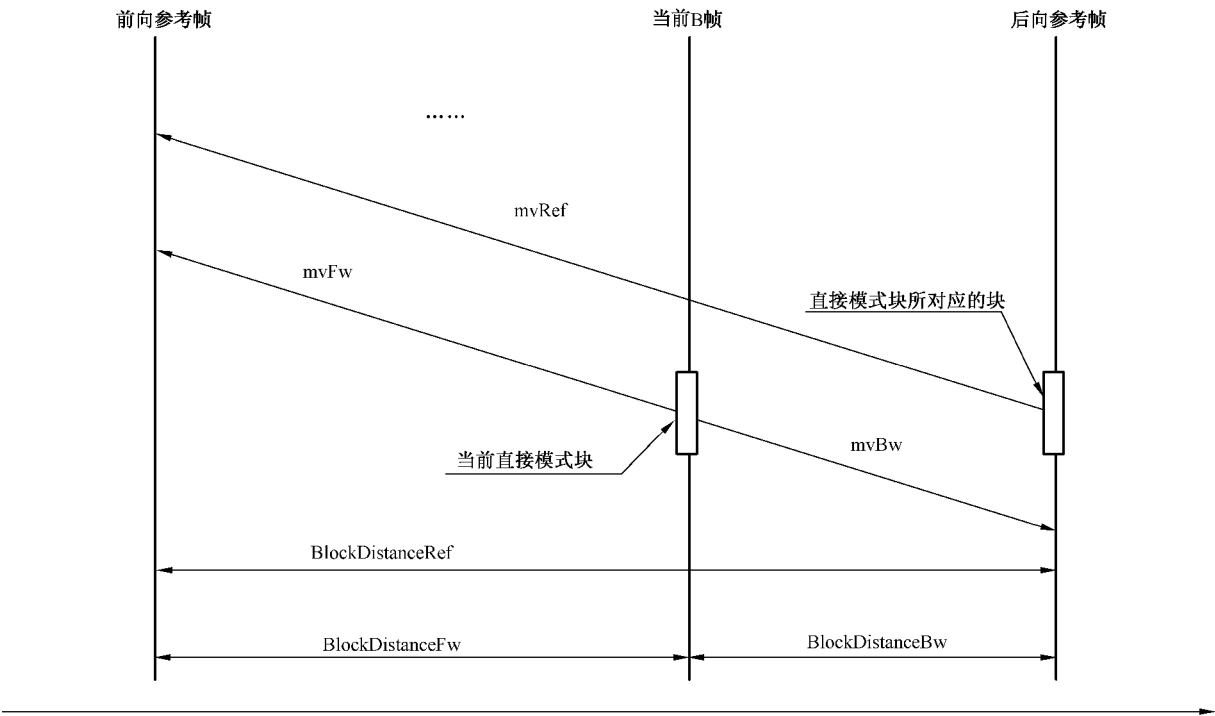
- 如果当前图像的 PictureStructure 的值为“0”, 并且 BFieldEnhanced 值为 1:

——如果当前块所在的场为顶场, 并且当前块的后向参考场为底场, 则 $\text{deltaBw} = 2$ 。

——如果当前块所在的场为底场, 并且当前块的后向参考场为顶场, 则 $\text{deltaBw} = -2$ 。

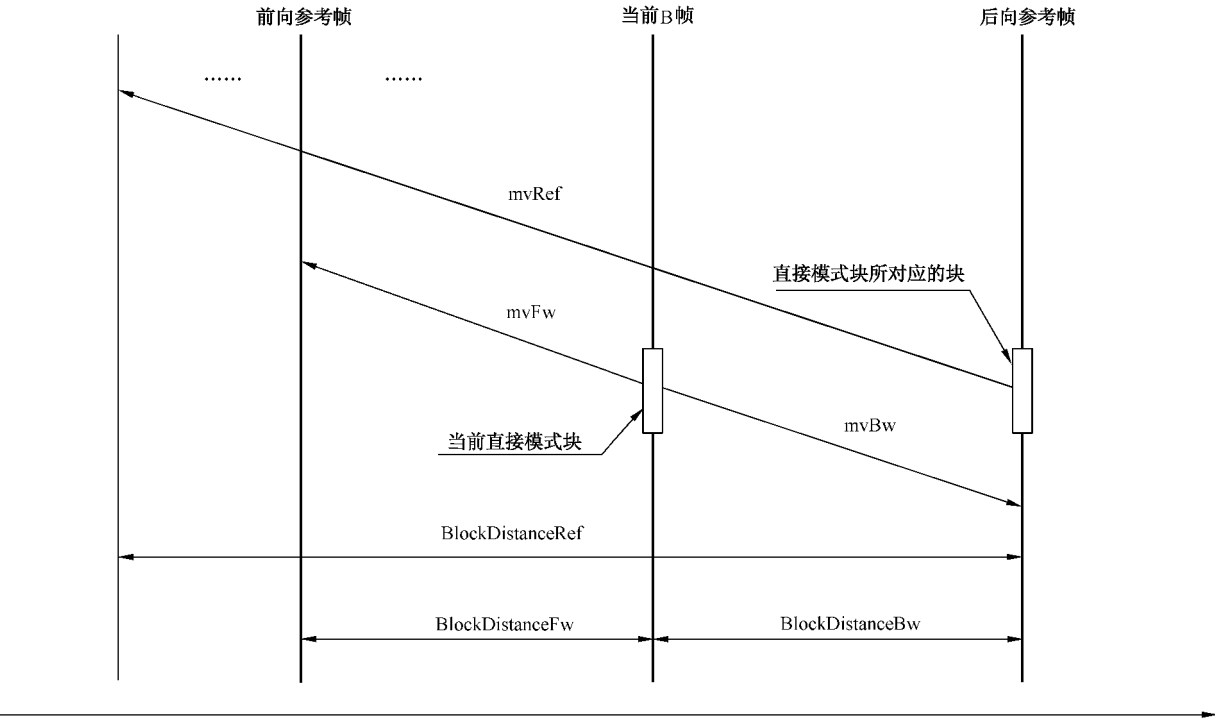
- 如果 mvRef_x 小于 0, 则 $\text{mvBw}_x = (((16\ 384 / \text{BlockDistanceRef}) \times (1 - \text{mvRef}_x \times \text{BlockDistanceBw}) - 1) >> 14)$; 否则 $\text{mvBw}_x = -(((16\ 384 / \text{BlockDistanceRef}) \times (1 + \text{mvRef}_x \times \text{BlockDistanceBw}) - 1) >> 14)$ 。

- 如果 $(\text{mvRef}_y + \text{delta1})$ 小于 0, 则 $\text{mvBw}_y = (((16\ 384 / \text{BlockDistanceRef}) \times (1 - (\text{mvRef}_y + \text{delta1}) \times \text{BlockDistanceBw}) - 1) >> 14) - \text{deltaBw}$; 否则 $\text{mvBw}_y = -(((16\ 384 / \text{BlockDistanceRef}) \times (1 + (\text{mvRef}_y + \text{delta1}) \times \text{BlockDistanceBw}) - 1) >> 14) - \text{deltaBw}$ 。



注：当前帧的 PictureStructure 等于 1 时，直接或跳过模式块所对应的块指向标记为 0 的参考图像。

图 24 直接模式的运动矢量推导过程 1



注：当前帧的 PictureStructure 等于 1 时，直接或跳过模式块所对应的块指向标记为 1 的参考图像。

图 25 直接模式的运动矢量推导过程 2

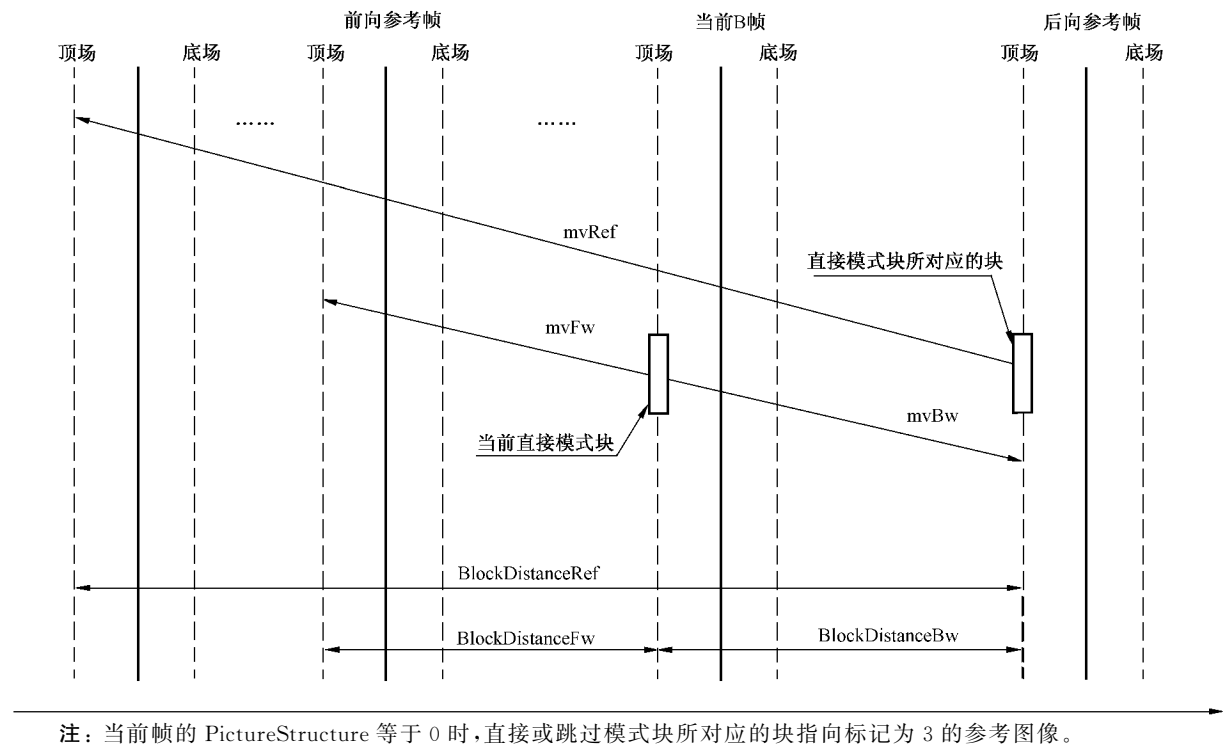


图 26 直接模式的运动矢量推导过程 3

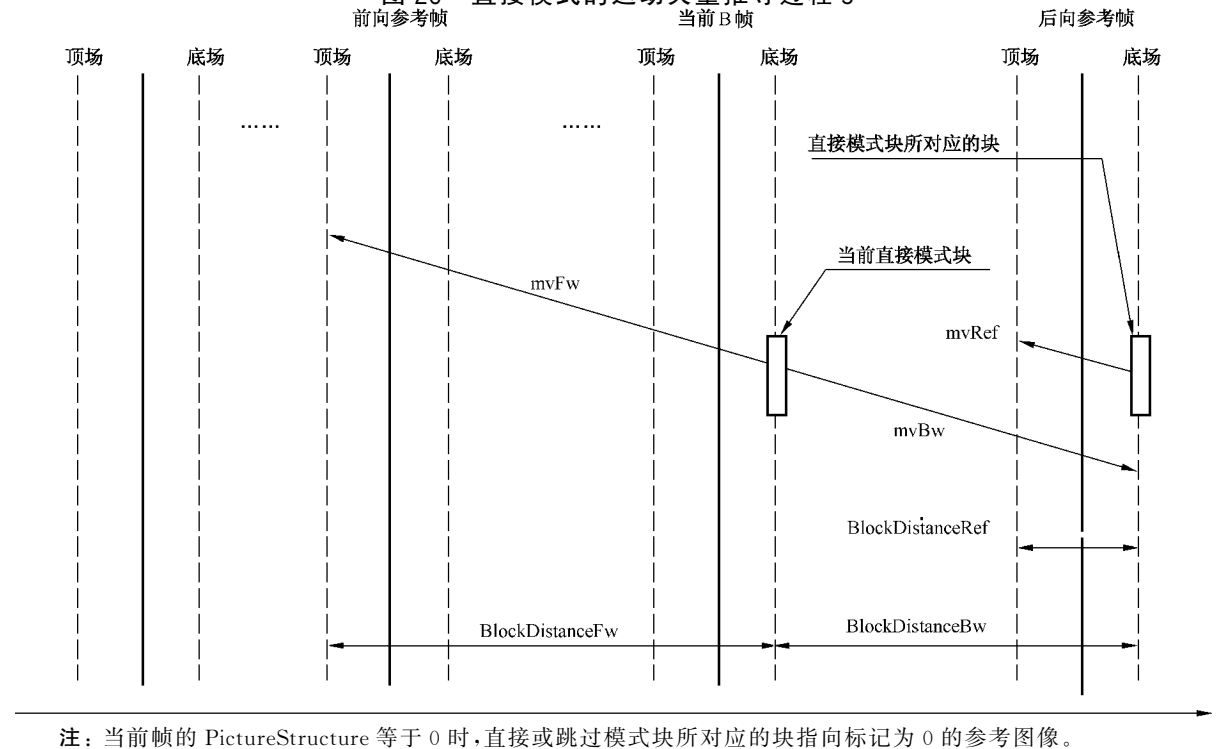


图 27 直接模式的运动矢量推导过程 4

- c) 如果当前块类型为对称模式,运动矢量按照下面定义的方法导出:
- 前向参考索引由位流中的 `mb_reference_index` 解码得到。如果 `PictureStructure` 等于“1”,则后向参考索引与前向参考索引相等;否则,后向参考索引等于 1 减去前向参考索引。对称模式块的前向运动矢量 `mvFw` 按照 9.4.6 的方法得到,当前块的后向运动矢量 `mvBw`

(mvBw_x, mvBw_y)为：
$$mvBw_x = -((mvFw_x \times BlockDistanceBw \times (512/BlockDistanceFw) + 256) >> 9)$$
$$mvBw_y = -((mvFw_y \times BlockDistanceBw \times (512/BlockDistanceFw) + 256) >> 9)$$

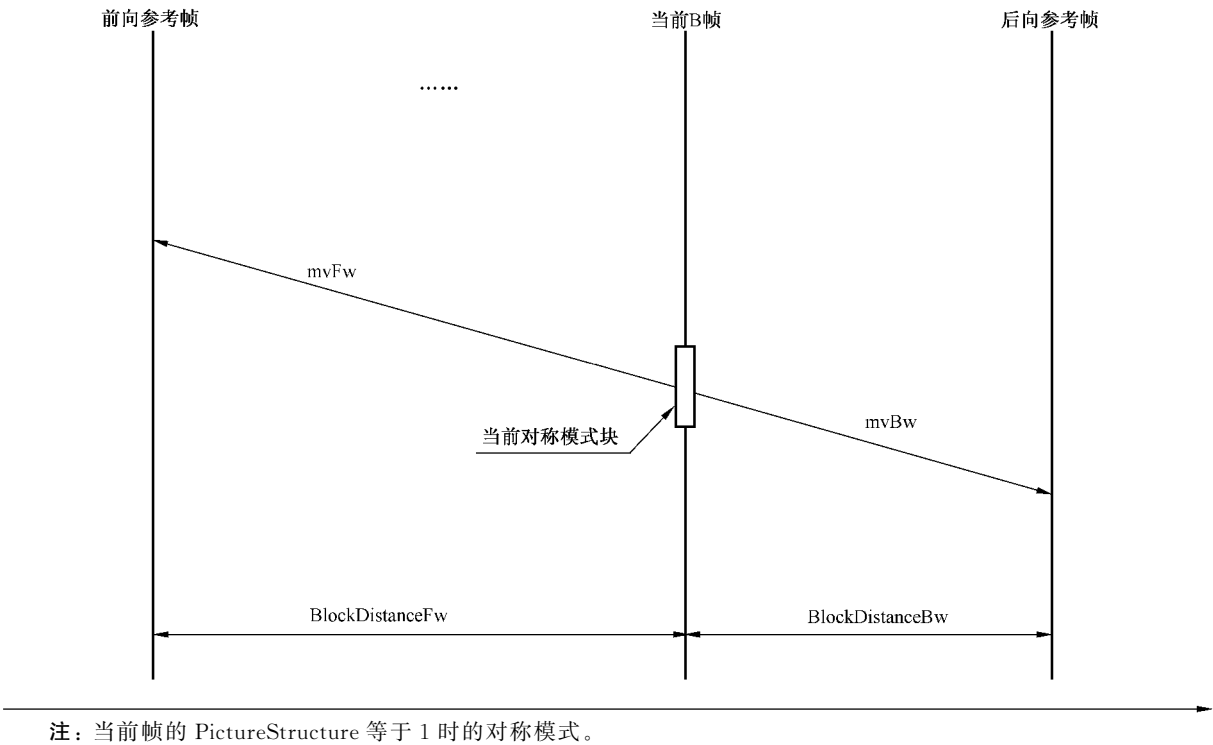


图 28 对称模式的运动矢量推导过程 1

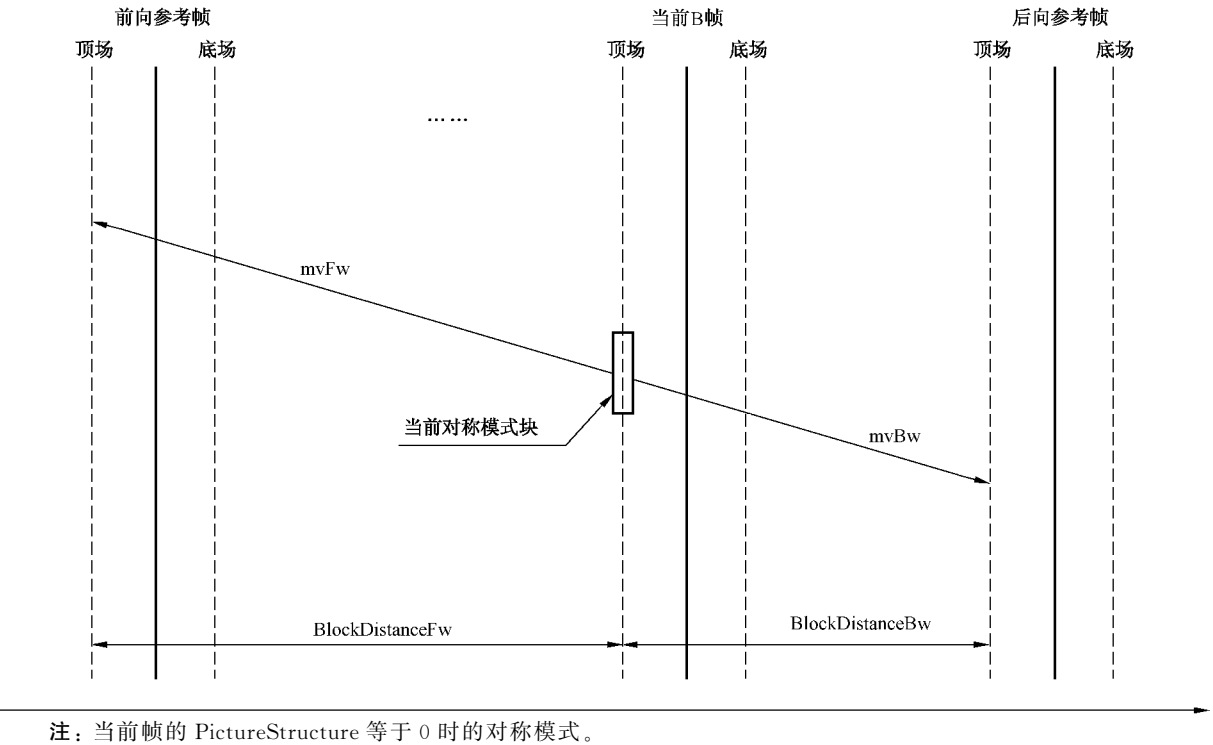


图 29 对称模式的运动矢量推导过程 2

按本条定义的方法导出的亮度运动矢量的范围应符合 6.2.5 规定。

9.9.2 参考样本的导出过程

9.9.2.1 概述

根据运动矢量对亮度进行 1/2 样本、1/4 样本的插值,得到相应的参考样本。

如果在插值过程中所参考的整数样本在参考图像外,应用该图像内距离参考样本最近的整数样本(边缘或角样本)代替,即运动矢量能指向参考图像外的样本。

9.9.2.2 亮度样本插值过程

图 30 给出了参考图像(或场)整数样本、1/2 样本和 1/4 样本的位置,其中用大写字母标记的为整数样本位置,用小写字母标记的为 1/2 和 1/4 样本位置。

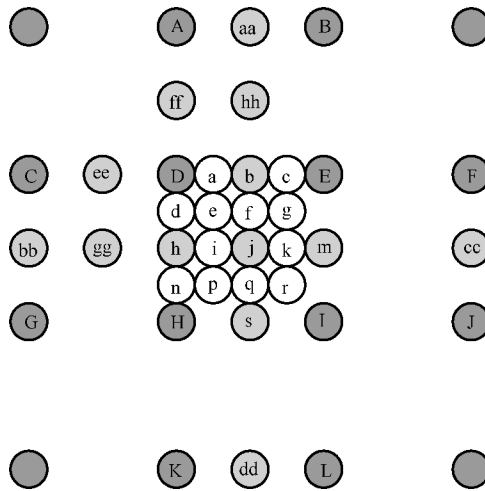


图 30 整数样本、1/2 样本和 1/4 样本的位置

1/2 样本位置的预测值通过 4 抽头滤波器 $F_1(-1, 5, 5, -1)$ 计算得到。1/4 样本位置的预测值通过 4 抽头滤波器 $F_2(1, 7, 7, 1)$ 计算得到。

1/2 样本的计算过程如下:

- 1/2 样本 b : 首先用 F_1 对水平方向上最近的 4 个整数样本滤波, 得到中间值 $b' = (-C + 5D + 5E - F)$; 最终的预测值 $b = \text{Clip1}((b' + 4) \gg 3)$ 。
- 1/2 样本 h : 首先用 F_1 对垂直方向上最近的 4 个整数样本滤波, 得到中间值 $h' = (-A + 5D + 5H - K)$; 最终的预测值 $h = \text{Clip1}((h' + 4) \gg 3)$ 。
- 1/2 样本 j : 首先用 F_1 在水平或垂直方向上对最近的 4 个 1/2 样本中间值滤波, 得到中间值 $j' = (-bb + 5h + 5m - cc)$, 或者 $j' = (-aa + 5b + 5s - dd)$ 。其中 aa, dd 和 s 是相应位置 1/2 样本中间值(用 F_1 在水平方向滤波得到), bb', cc' 和 m' 是相应位置 1/2 样本中间值(用 F_1 在垂直方向滤波得到)。最终的预测值 $j = \text{Clip1}((j' + 32) \gg 6)$ 。采用水平方向或垂直方向滤波得到的值相同。

1/4 样本的计算过程如下:

- 1/4 样本 a : 首先用 F_2 在水平方向上对 ee', D', b' 和 E' 4 个值滤波, 得到中间值 $a' = (ee' + 7D' + 7b' + E')$; 最终的预测值 $a = \text{Clip1}((a' + 64) \gg 7)$ 。其中 ee' 和 b' 是相应位置 1/2 样本中间值, D' 和 E' 是相应位置整数样本放大 8 倍的值。1/4 样本 c 的插值过程与 a 的插值过程相同。

- b) 1/4 样本 d : 首先用 F_2 在垂直方向上对 ff', D', h' 和 H' 4 个值滤波, 得到中间值 $d' = (ff' + 7 D' + 7 h' + H')$; 最终的预测值 $d = \text{Clip1}((d' + 64) \gg 7)$ 。其中 ff' 和 h' 是相应位置 1/2 样本中间值, D' 和 H' 是相应位置整数样本放大 8 倍的值。1/4 样本 n 的插值过程与 d 的插值过程相同。
- c) 1/4 样本 i : 首先用 F_2 在水平方向上对 gg', h'', j' 和 m'' 4 个值滤波, 得到中间值 $i' = (gg' + 7 h'' + 7 j' + m'')$; 最终的预测值 $i = \text{Clip1}((i' + 512) \gg 10)$ 。其中 gg' 和 j' 是相应位置 1/2 中间值, h'' 和 m'' 是相应位置 1/2 样本中间值放大 8 倍的值。1/4 样本 k 的插值过程与 i 的插值过程相同。
- d) 1/4 样本 f : 首先用 F_2 在垂直方向上对 hh', b'', j' 和 s'' 4 个值滤波, 得到中间值 $f' = (hh' + 7 b'' + 7 j' + s'')$; 最终的预测值 $f = \text{Clip1}((f' + 512) \gg 10)$ 。其中 hh' 和 j' 是相应位置 1/2 中间值, b'' 和 s'' 是相应位置 1/2 样本中间值放大 8 倍的值。1/4 样本 q 的插值过程与 f 的插值过程相同。
- e) 1/4 样本 e, g, p 和 r :
- $$e = \text{Clip1}((D'' + j' + 64) \gg 7)$$
- $$g = \text{Clip1}((E'' + j' + 64) \gg 7)$$
- $$p = \text{Clip1}((H'' + j' + 64) \gg 7)$$
- $$r = \text{Clip1}((I'' + j' + 64) \gg 7)$$

其中 D'', E'', H'' 和 I'' 是相应位置整数样本放大 64 倍的值, j' 是相应位置 1/2 样本中间值。

根据当前块运动矢量 mvE 查表 63 得到预测样本矩阵的元素 $\text{predMatrix}[x, y]$ 。表 63 中 xFracL 等于 mvE 的水平分量 mvE_x&3, yFracL 等于 mvE 的垂直分量 mvE_y&3。

表 63 预测样本矩阵元素

xFracL 的值	yFracL 的值	predMatrix[x, y] 的值
0	0	D
0	1	d
0	2	h
0	3	n
1	0	a
1	1	e
1	2	i
1	3	p
2	0	b
2	1	f
2	2	j
2	3	q
3	0	c
3	1	g
3	2	k
3	3	r

9.9.2.3 色度样本插值过程

色度样本插值使用与对应亮度块的运动矢量 mvE (mvE 的水平分量为 mvE_x , 垂直分量为 mvE_y) 对应的运动矢量 mvC 。 mvC 的水平分量为 mvC_x , 垂直分量为 mvC_y 。 mvC 的基本单位为 $1/8$ 样本。 mvC 的范围应符合 6.2.4 规定。如果 $chroma_format$ 的值为“01”, 则 mvC_x 的值等于 mvE_x , mvC_y 的值等于 mvE_y ; 如果 $chroma_format$ 的值为“10”, 则 mvC_x 的值等于 mvE_x , mvC_y 的值等于 $mvE_y \times 2$ 。色度样本插值如图 31 所示, A, B, C, D 是被插值样本周围的整数样本值, dx 与 dy 分别是预测样本与 A 的水平和垂直距离。 dx 等于 $mvC_x \& 7$, dy 等于 $mvC_y \& 7$ 。这里各变量与参考样本的位置关系如图 31 所示。

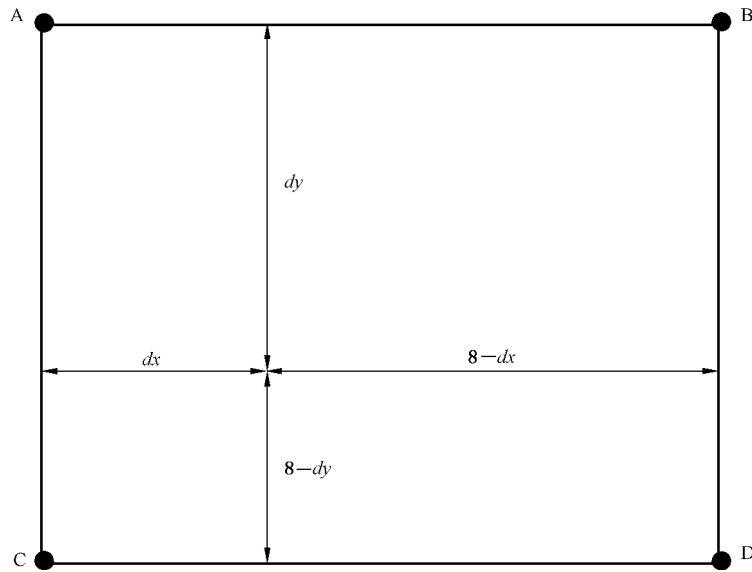


图 31 色度插值

预测样本矩阵的元素 $predMatrix[x, y]$ 计算如下:

$$predMatrix[x, y] = Clip1(((8 - dx) \times (8 - dy) \times A + dx \times (8 - dy) \times B + (8 - dx) \times dy \times C + dx \times dy \times D + 32) \gg 6)$$

9.9.3 加权预测

如果当前宏块的 $WeightingPrediction$ 的值为 1 并且不是 I 宏块类型, 应使用本条定义的方法进行加权预测。

如果当前宏块是 P 宏块类型, 当前块的参考索引值为 i , 亮度预测样本矩阵计算如下:

$$predMatrix[x, y] = Clip1(((predMatrix[x, y] \times luma_scale[i] + 16) \gg 5) + luma_shift[i])$$

如果当前宏块是 P 宏块类型, 当前块的参考索引值为 i , 色度预测样本矩阵计算如下:

$$predMatrix[x, y] = Clip1(((predMatrix[x, y] \times chroma_scale[i] + 16) \gg 5) + chroma_shift[i])$$

如果当前宏块是 B 宏块类型, 当前块的前向参考索引值为 i , 前向亮度预测样本矩阵计算如下:

$$predMatrixFw[x, y] = Clip1(((predMatrixFw[x, y] \times luma_scale[i \times 2] + 16) \gg 5) + luma_shift[i \times 2])$$

如果当前宏块是 B 宏块类型, 当前块的后向参考索引值为 j , 后向亮度预测样本矩阵计算如下:

$$predMatrixBw[x, y] = Clip1(((predMatrixBw[x, y] \times luma_scale[j \times 2 + 1] + 16) \gg 5) + luma_shift[j \times 2 + 1])$$

如果当前宏块是 B 宏块类型, 当前块的前向参考索引值为 i , 前向色度预测样本矩阵计算如下:

$$predMatrixFw[x, y] = Clip1(((predMatrixFw[x, y] \times chroma_scale[i \times 2] + 16) \gg 5) + chroma_shift[i \times 2])$$

_shift[i×2])

如果当前宏块是 B 宏块类型,当前块的后向参考索引值为 j,后向色度预测样本矩阵计算如下:

$$\text{predMatrixBw}[x,y] = \text{Clip1}(((\text{predMatrixBw}[x,y] \times \text{chroma_scale}[j \times 2 + 1] + 16) >> 5) + \text{chroma_shift}[j \times 2 + 1])$$

9.10 重建

如果当前块是 I 或 P 宏块类型,重建系数矩阵 RecMatrix 计算如下:

$$\text{RecMatrix}[x,y] = \text{Clip1}(\text{predMatrix}[x,y] + \text{ResidueMatrix}[x,y])$$

如果当前块是 B 宏块类型,并且使用两帧参考图像,重建系数矩阵 RecMatrix 计算如下:

$$\text{RecMatrix}[x,y] = \text{Clip1}((\text{predMatrixFw}[x,y] + \text{predMatrixBw}[x,y] + 1) >> 1 + \text{ResidueMatrix}[x,y])$$

如果当前块是 B 宏块类型,并且只使用前向参考图像,重建系数矩阵 RecMatrix 计算如下:

$$\text{RecMatrix}[x,y] = \text{Clip1}(\text{predMatrixFw}[x,y] + \text{ResidueMatrix}[x,y])$$

如果当前块是 B 宏块类型,并且只使用后向参考图像,重建系数矩阵 RecMatrix 计算如下:

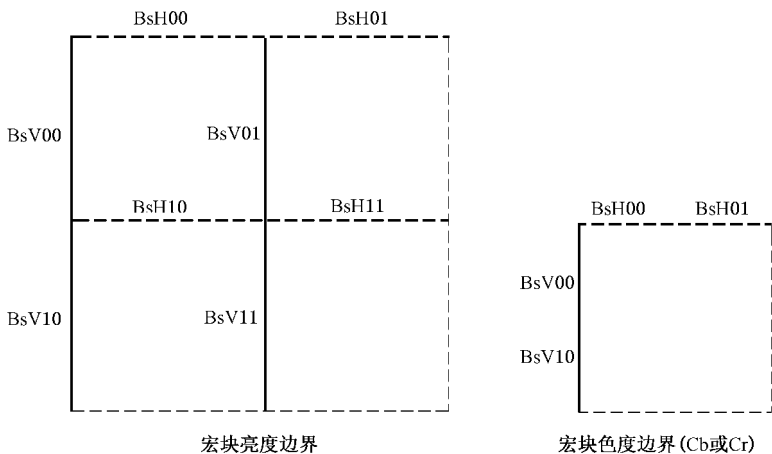
$$\text{RecMatrix}[x,y] = \text{Clip1}(\text{predMatrixBw}[x,y] + \text{ResidueMatrix}[x,y])$$

其中 predMatrixFw 是前向参考图像的预测样本矩阵,predMatrixBw 是后向参考图像的预测样本矩阵。

9.11 环路滤波

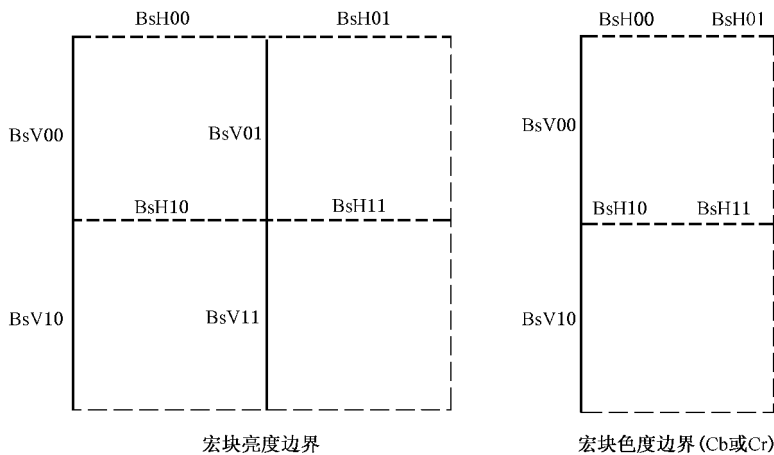
9.11.1 概述

8×8 亮度块之间的每条边界有一个“边界强度”Bs,色度块的边界强度用对应位置亮度块边界的 Bs 代替,如图 32 和图 33 所示。如果 Bs 等于 0 则不要对边界滤波,否则根据局部样本值的特性和 Bs 值对边界滤波。



注: 实线为块垂直边界,粗虚线为水平边界,细虚线为下一宏块待滤波的边界。

图 32 4 : 2 : 0 宏块中需要滤波的边界示意



注：实线为块垂直边界，粗虚线为水平边界，细虚线为下一宏块待滤波的边界。

图 33 4 : 2 : 2 宏块中需要滤波的边界示意

除图像边界及条带的边界之外,宏块与相邻宏块的上边界和左边界,以及宏块内部各个 8×8 块的边界都应进行滤波。

环路滤波以宏块为单位,按照解码顺序依次处理。图像中每个宏块的滤波过程如下:

对亮度和色度分别做环路滤波,如图 32 和图 33 所示,首先从左到右对垂直边界滤波,然后从上到下对水平边界滤波。当前宏块的上边或者左边的样本值可能在以前的宏块环路滤波过程中已经被修改,当前宏块的环路滤波的输入为这些可能被修改的样本值,并且当前宏块环路滤波可能进一步修改这些样本值。当前宏块垂直边界滤波过程中修改的样本值作为水平边界滤波过程的输入。

帧内预测使用环路滤波前的重建图像样本值。

9.11.2 边界滤波强度的推导过程

根据宏块类型、宏块中 8×8 亮度块的运动矢量,按如下方法计算 Bs 的值:

- 如果边界两边的两个 8×8 块有一个或两个都属于帧内预测宏块,则 Bs 等于“2”。
- 否则,如果当前图像是 P 帧、I 帧的第二场,并且满足以下两个条件中的任一个,则 Bs 等于“1”;如果当前图像是 P 帧、I 帧的第二场,并且以下两个条件都不满足,则 Bs 等于“0”:
 - 两个块的参考图像不同;
 - 两个块的参考图像相同,但是两个运动矢量分量中任一个分量的差值大于或等于一个整像素。
- 否则,按以下方法计算 Bs 的值:
 - 如果两个块 p 和 q 的前向索引参考值和后向索引参考值分别相等:
 - 以下两个条件中任一个成立,则 Bs 等于“1”:
 - 两个块的前向运动矢量分量中任一个分量的差值大于或等于一个整像素;
 - 两个块的后向运动矢量分量中任一个分量的差值大于或等于一个整像素。
 - 否则, Bs 等于“0”。
 - 否则, Bs 等于“1”。

9.11.3 块边界阈值的推导过程

图 34 表示块 p 和块 q 在水平或垂直边界两侧的 6 个样本点(边界用黑色粗线表示)。用 P_0 、 P_1 、 Q_0

和 Q_1 分别表示 p_0 、 p_1 、 q_0 和 q_1 滤波后的样本值。

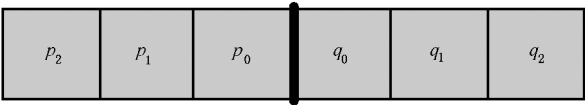


图 34 8×8 块水平或垂直边界样本

如果下式为真,对边界样本按照 9.11.4 和 9.11.5 进行滤波:

$$Bs ! = 0 \& \& Abs(p_0 - q_0) < \alpha \& \& Abs(p_1 - p_0) < \beta \& \& Abs(q_1 - q_0) < \beta$$

其中 α 和 β 为块边界阈值,可以根据两个块的 QP 平均值 QP_{av} 以及 AlphaOffset 和 BetaOffset 计算查表索引 IndexA 和 IndexB。

两个块的 QP 平均值 QP_{av} 为:

$$QP_{av} = (QP_p + QP_q + 1) >> 1$$

对于亮度分量, QP_p 和 QP_q 应使用宏块的量化参数;对于色度分量, QP_p 和 QP_q 应使用色度块的量化参数。

索引 IndexA 和 IndexB 为:

$$IndexA = Clip3(0, 63, QP_{av} + AlphaOffset)$$
$$IndexB = Clip3(0, 63, QP_{av} + BetaOffset)$$

根据索引 IndexA 和 IndexB 与阈值 α 和 β 间的对应关系,由表 64 得到 α 、 β 的取值,根据 IndexA 查表 64 得到 α ,根据 IndexB 查表 64 得到 β 。

表 64 块边界阈值 α 和 β 与 IndexA 和 IndexB 的关系

索引	α 的值	β 的值
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0
5	0	0
6	1	1
7	1	1
8	1	1
9	1	1
10	1	1
11	2	1
12	2	1
13	2	2
14	3	2
15	3	2
16	4	2

表 64（续）

索引	α 的值	β 的值
17	4	2
18	5	3
19	5	3
20	6	3
21	7	3
22	8	4
23	9	4
24	10	4
25	11	4
26	12	5
27	13	5
28	15	5
29	16	5
30	18	6
31	20	6
32	22	6
33	24	7
34	26	7
35	28	7
36	30	8
37	33	8
38	33	8
39	35	9
40	35	9
41	36	10
42	37	10
43	37	11
44	39	11
45	39	12
46	42	13
47	44	14
48	46	15
49	48	16
50	50	17

表 64 (续)

索引	α 的值	β 的值
51	52	18
52	53	19
53	54	20
54	55	21
55	56	22
56	57	23
57	58	23
58	59	24
59	60	24
60	61	25
61	62	25
62	63	26
63	64	27

9.11.4 Bs 等于 2 时的边界滤波过程

首先定义 $ap = \text{Abs}(p_2 - p_0)$, $aq = \text{Abs}(q_2 - q_0)$ 。

对亮度块边界两边的样本 p_0 、 p_1 、 q_0 和 q_1 的滤波过程如下：

```
if( $ap < \beta$  &&  $\text{Abs}(p_0 - q_0) < ((\alpha > 2) + 2)$ ) {  
     $P_0 = (p_1 + 2 \times p_0 + q_0 + 2) >> 2$   
     $P_1 = (2 \times p_1 + p_0 + q_0 + 2) >> 2$   
}  
else  
     $P_0 = (2 \times p_1 + p_0 + q_0 + 2) >> 2$   
if( $aq < \beta$  &&  $\text{Abs}(p_0 - q_0) < ((\alpha > 2) + 2)$ ) {  
     $Q_0 = (q_1 + 2 \times q_0 + p_0 + 2) >> 2$   
     $Q_1 = (2 \times q_1 + q_0 + p_0 + 2) >> 2$   
}  
else  
     $Q_0 = (2 \times q_1 + q_0 + p_0 + 2) >> 2$ 
```

其中 P_0 和 Q_0 分别为 p_0 和 q_0 滤波后的值。如果在上面的滤波过程中 P_1 (或 Q_1) 不被赋值, 则不对 p_1 (或 q_1) 滤波; 否则, P_1 或 Q_1 为 p_1 或 q_1 滤波后的值。

色度块边界两边的样本 p_0 和 q_0 的采用同样的方法滤波, 不对 p_1 和 q_1 滤波。

9.11.5 Bs 等于 1 时的边界滤波过程

边界滤波强度 Bs 的值为 1 时, 对 p_0 和 q_0 滤波的计算过程如下 (P_0 和 Q_0 分别为 p_0 和 q_0 滤波后的值):

$$\mathit{delta} = \mathit{Clip3}(-C, C, (((q_0 - p_0) \times 3 + (p_1 - q_1) + 4) \gg 3))$$

$$P_0 = \mathit{Clip1}(p_0 + \mathit{delta})$$

$$Q_0 = \mathit{Clip1}(q_0 - \mathit{delta})$$

然后判断是否需要 p_1 和 q_1 滤波,计算过程如下:

- a) 如果为色度边界,不对 p_1 和 q_1 滤波。
- b) 如果在亮度边界处有 ap 小于 β ,则对 p_1 滤波,滤波后的值为
$$P_1 = \mathit{Clip1}(p_1 + \mathit{Clip3}(-C, C, (((P_0 - p_1) \times 3 + (p_2 - Q_0) + 4) \gg 3)))$$
- c) 如果在亮度边界处有 aq 小于 β ,则对 q_1 滤波,滤波后的值为
$$Q_1 = \mathit{Clip1}(q_1 - \mathit{Clip3}(-C, C, (((q_1 - Q_0) \times 3 + (P_0 - q_2) + 4) \gg 3)))$$

上述滤波过程中, ap 和 aq 的定义见 9.11.3, C 称为滤波裁减参数, C 与 IndexA 之间的关系见表 65。

表 65 滤波裁减参数 C 与 IndexA 的关系

IndexA	C 的值
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0
10	0
11	0
12	0
13	0
14	0
15	0
16	1
17	1
18	1
19	1
20	1
21	1
22	1
23	1

表 65 (续)

<i>IndexA</i>	C 的值
24	1
25	1
26	1
27	1
28	1
29	1
30	2
31	2
32	2
33	2
34	2
35	2
36	2
37	2
38	3
39	3
40	3
41	3
42	3
43	3
44	3
45	4
46	4
47	4
48	5
49	5
50	5
51	6
52	6
53	6
54	7
55	7
56	7
57	7

表 65 (续)

<i>IndexA</i>	<i>C</i> 的值
58	8
59	8
60	8
61	9
62	9
63	9

附 录 A
(规范性附录)
伪起始码

本附录定义防止在位流中出现伪起始码的方法。起始码的形式、含义,以及为了使起始码字节对齐而进行填充的方法见 7.1.1 和 5.9.2。

为了防止出现伪起始码,编码时应按照以下方法处理:写入一位时,如果该位是一个字节的第二最低有效位,检查该位之前写入的 22 位,如果这 22 位都是“0”,在该位之前插入“10”,该位成为下一个字节的最高有效位。

解码时应按以下方法处理:每读入一个字节时,检查前面读入的两个字节和当前字节,如果这三个字节构成位串“0000 0000 0000 0000 0000 0010”,丢弃当前字节的最低两个有效位。丢弃一个字节最低两个有效位可采用任意等效的方式,本部分不做规定。

在编码和解码时对于序列头、序列显示扩展、版权扩展、用户数据、摄像机参数扩展、图像显示扩展、区域扩展、视频扩展数据保留字节中的数据不应采用上述方法。

附录 B
(规范性附录)
档次和级别

B.1 概述

档次和级别提供了一种定义本部分的语法和语义的子集的手段。档次和级别对位流进行了各种限制,同时也就规定了对某一特定位流解码所需要的解码器能力。档次是本部分规定的语法、语义及算法的子集。符合某个档次规定的解码器应完全支持该档次定义的子集。级别是在某一档次下对语法元素和语法元素参数值的限定集合。在给定档次的情况下,不同级别往往意味着对解码器能力和存储器容量的不同要求。

本附录描述了不同档次和级别所对应的各种限制。所有未被限定的语法元素和参数可以取任何本部分所允许的值。如果一个解码器能对某个档次和级别所规定的语法元素的所有允许值正确解码,则称此解码器在这个档次和级别上符合本部分。如果一个位流中不存在某个档次和级别所不允许的语法元素,并且其所含有的语法元素的值不超过此档次和级别所允许的范围,则认为此位流在这个档次和级别上符合本部分。

profile_id 和 level_id 定义了位流的档次和级别。

B.2 档次

本部分定义的档次见表 B.1。

表 B.1 档次

profile_id 的值	档次
0x00	禁止
0x20	见 GB/T 20090.2—2013 的定义
0x48	广播档次(Broadcasting profile)
其他	保留

对于一个给定的档次,不同的级别支持相同的语法子集。

广播档次的位流应满足以下条件:

- a) profile_id 的值应为 0x48;
- b) progressive_sequence 的值为“0”时,整个视频序列中所有图像的 top_field_first 的值均应相同;
- c) advanced_pred_mode_disable 的值应为“1”;
- d) chroma_format 的值应为“01”或“10”;
- e) sample_precision 的值应为“001”;
- f) 视频序列起始码与随后第一个视频序列结束码之间,或视频序列起始码与随后第一个视频编辑码之间所有编码图像的 PictureStructure 的值均应相同;
- g) B.3.2 规定的级别限制;
- h) 广播档次支持的级别包括: 2.0.0.08.30、2.1.0.08.30、2.1.0.08.15、4.0.0.08.30、4.0.2.08.60、4.2.0.08.30、6.0.0.08.60、6.0.1.08.60、6.0.3.08.60、6.0.5.08.60 和 6.2.0.08.60。

B.3 级别

B.3.1 本部分定义的级别

本部分定义的级别见表 B.2。

表 B.2 级别

level_id 的值	级别
0x00	禁止
0x10	2.0.0.08.30
0x12	2.1.0.08.15
0x14	2.1.0.08.30
0x20	4.0.0.08.30
0x22	4.2.0.08.30
0x2A	4.0.2.08.60
0x40	6.0.0.08.60
0x41	6.0.1.08.60
0x42	6.2.0.08.60
0x44	6.0.3.08.60
0x46	6.0.5.08.60
其他	保留

B.3.2 与档次无关的级别限制

对于所有档次,每个宏块编码后最大二进制位数的限制见表 B.3,其中 n 是样本点精度。

表 B.3 宏块编码后最大二进制位数

图像格式	宏块编码后最大二进制位数
4 : 2 : 0	$128 + 256 \times n \times 1.5$
4 : 2 : 2	$128 + 256 \times n \times 2$

级别 2.0.0.08.30、2.1.0.08.15 和 2.1.0.08.30 的参数限制见表 B.4。

表 B.4 级别 2.0.0.08.30、2.1.0.08.15 和 2.1.0.08.30 的参数限制

参 数	级 别		
	2.0.0.08.30	2.1.0.08.15	2.1.0.08.30
每行最大样本数	352	352	352
每帧最大行数	288	288	288
每秒最大帧数	30	15	30

表 B.4 (续)

参 数	级 别		
	2.0.0.08.30	2.1.0.08.15	2.1.0.08.30
样本精度(位)	8	8	8
亮度样本速率	3 041 280	1 520 640	3 041 280
最大比特率(位每秒)	1 000 000	1 500 000	2 500 000
每秒最大二元符号数	8 110 080	12 165 120	20 275 200
BBV 缓冲区大小(位)	122 880	196 608	311 296
每帧最大宏块个数	396	396	396
每秒最大宏块个数	11 880	5 940	11 880
帧编码时最大垂直运动矢量范围(亮度样本数)	$[-128, +127.75]$	$[-128, +127.75]$	$[-128, +127.75]$
场编码时最大垂直运动矢量范围(亮度样本数)	—	—	—
最大水平运动矢量范围(亮度样本数)	$[-2\ 048, +2\ 047.75]$	$[-2\ 048, +2\ 047.75]$	$[-2\ 048, +2\ 047.75]$
图像格式	4 : 2 : 0	4 : 2 : 0	4 : 2 : 0

级别 4.0.0.08.30 和 4.2.0.08.30 的参数限制见表 B.5。

表 B.5 级别 4.0.0.08.30 和 4.2.0.08.30 的参数限制

参 数	级 别	
	4.0.0.08.30	4.2.0.08.30
每行最大样本数	720	720
每帧最大行数	576	576
每秒最大帧数	30	30
样本精度(位)	8	8
亮度样本速率	10 368 000	10 368 000
最大比特率(位每秒)	10 000 000	15 000 000
每秒最大二元符号数	82 944 000	110 592 000
BBV 缓冲区大小(位)	1 228 800	1 851 392
每帧最大宏块个数	1 620	1 620
每秒最大宏块个数	40 500	40 500
帧编码时最大垂直运动矢量范围(亮度样本数)	$[-256, +255.75]$	$[-256, +255.75]$
场编码时最大垂直运动矢量范围(亮度样本数)	$[-128, +127.75]$	$[-128, +127.75]$
最大水平运动矢量范围(亮度样本数)	$[-2\ 048, +2\ 047.75]$	$[-2\ 048, +2\ 047.75]$
图像格式	4 : 2 : 0	4 : 2 : 2

级别 4.0.2.08.60 的参数限制见表 B.6。

表 B.6 级别 4.0.2.08.60 的参数限制

参 数	级 别
	4.0.2.08.60
每行最大样本数	720
每帧最大行数	576
每秒最大帧数	60
样本精度(位)	8
亮度样本速率	20 736 000
最大比特率(位每秒)	20 000 000
每秒最大二元符号数	165 888 000
BBV 缓冲区大小(位)	10 485 760
每帧最大宏块个数	1 620
每秒最大宏块个数	81 000
帧编码时最大垂直运动矢量范围(亮度样本数)	[−256,+255.75]
场编码时最大垂直运动矢量范围(亮度样本数)	[−128,+127.75]
最大水平运动矢量范围(亮度样本数)	[−2 048,+2 047.75]
图像格式	4 : 2 : 0

级别 6.0.0.08.60 和 6.0.1.08.60 的参数限制见表 B.7。

表 B.7 级别 6.0.0.08.60 和 6.0.1.08.60 的参数限制

参 数	级 别	
	6.0.0.08.60	6.0.1.08.60
每行最大样本数	1 920	1 920
每帧最大行数	1 152	1 152
每秒最大帧数	60	60
样本精度(位)	8	8
亮度样本速率	62 668 800	62 668 800
最大比特率(位每秒)	20 000 000	50 000 000
每秒最大二元符号数	200 540 160	501 350 400
BBV 缓冲区大小(位)	2 457 600	62 488 576
每帧最大宏块个数	8 160	8 160
每秒最大宏块个数	244 800	244 800
帧编码时最大垂直运动矢量范围(亮度样本数)	[−512,+511.75]	[−512,+511.75]
场编码时最大垂直运动矢量范围(亮度样本数)	[−256,+255.75]	[−256,+255.75]
最大水平运动矢量范围(亮度样本数)	[−2 048,+2047.75]	[−2 048,+2047.75]
图像格式	4 : 2 : 0	4 : 2 : 0

级别 6.0.3.08.60、6.0.5.08.60 和 6.2.0.08.60 的参数限制见表 B.8。

表 B.8 级别 6.0.3.08.60、6.0.5.08.60 和 6.2.0.08.60 的参数限制

参 数	级 别		
	6.0.3.08.60	6.0.5.08.60	6.2.0.08.60
每行最大样本数	1 920	4 096	1 920
每帧最大行数	1 152	2 160	1 152
每秒最大帧数	60	60	60
样本精度(位)	8	8	8
亮度样本速率	125 337 600	265 420 800	62 668 800
最大比特率(位每秒)	100 000 000	200 000 000	30 000 000
每秒最大二元符号数	1 002 700 800	2 005 401 600	401 080 320
BBV 缓冲区大小(位)	62 488 576	249 954 304	3 686 400
每帧最大宏块个数	8 160	34 560	8 160
每秒最大宏块个数	489 600	1 036 800	244 800
帧编码时最大垂直运动矢量范围(亮度样本数)	$[-512, +511.75]$	$[-1\,024, +1\,023.75]$	$[-512, +511.75]$
场编码时最大垂直运动矢量范围(亮度样本数)	$[-256, +255.75]$	$[-512, +511.75]$	$[-256, +255.75]$
最大水平运动矢量范围(亮度样本数)	$[-2\,048, +2\,047.75]$	$[-4\,096, +4\,095.75]$	$[-2\,048, +2\,047.75]$
图像格式	4 : 2 : 0	4 : 2 : 0	4 : 2 : 0 或 4 : 2 : 2

某一级别下 BBV 缓冲区大小是 16 384 位的倍数。

与表 B.4、表 B.5、表 B.6、表 B.7 和表 B.8 有关的语法元素包括: horizontal_size、vertical_size、frame_rate_code、bbv_buffer_size、chroma_format。

如果 aec_enable 的值为“1”，一帧图像中二元符号的最大个数应满足表 B.4、表 B.5、表 B.6、表 B.7 和表 B.8 的限制。

附 录 C

(规范性附录)

位流虚拟参考解码器

C.1 概述

本附录定义了位流虚拟参考解码器(以下简称 BBV)。

BBV 有一个输入缓冲区,称为 BBV 缓冲区。编码数据按照 C.3.1 定义的方式进入 BBV 缓冲区,按照 C.3.2 定义的方式移出 BBV 缓冲区。符合本部分的位流不应导致 BBV 缓冲区上溢。如果 low_delay 的值为“0”,符合本部分的位流不应导致 BBV 缓冲区下溢;如果 low_delay 的值为“1”,符合本部分的位流可能导致 BBV 缓冲区下溢,此时应按 C.3.2.2 定义的方式处理。

本附录中所有的运算都是实数运算,不存在舍入误差,例如 BBV 缓冲区中的位数不必是整数值。

C.2 约定

C.2.1 约定一

BBV 和视频编码器的时钟频率和帧率相同,并且同步操作。

C.2.2 约定二

BBV 缓冲区的大小为 BBS,单位为二进制位。

C.2.3 约定三

编码数据输入 BBV 缓冲区的最大速率 $R_{\max}$ (单位为位每秒)按式(C.1)计算。

$$R_{\max} = \text{BitRate} \times 400 \dots\dots\dots (C.1)$$

式中:

BitRate ——以 400 bit/s 为单位计算视频位流的比特率;

$R_{\max}$ ——编码数据输入 BBV 缓冲区的最大速率。

C.3 基本操作

C.3.1 数据输入

本附录定义了两种方法来计算编码数据进入 BBV 缓冲区的速率。这两种方法不应同时使用。

C.3.1.1 方法一

C.3.1.1.1 操作过程

在 BBV 中,第 n 帧图像的编码数据 $f(n)$ 包括以下数据(如果存在):

- a) 序列头、跟在序列头之后的扩展和用户数据、视频编辑码。这些数据定义为 $b(n)$, $b(n)$ 与本帧图像起始码之间不应有其他图像的数据;
- b) 本帧图像的所有编码数据;
- c) 跟在本帧图像头后的图像显示扩展;
- d) 跟在本帧图像后的填充数据、视频序列结束码。

如果 BbvDelay 的值不等于 BbvDelayMax,第 n 帧图像进入 BBV 缓冲区的速率 $R(n)$ 按式(C.2)

计算。

$$R(n) = d_n^* \div (\tau(n) - \tau(n+1) + t(n+1) - t(n)) \quad \dots\dots\dots (C.2)$$

式中：

- d_n^* ——从第 n 帧的起始码后第 1 个位到第 $n+1$ 帧的起始码后第 1 个位之间所有的位数；
- $\tau(n)$ ——第 n 帧的 BbvDelay 的值,单位为秒(s)；
- $t(n)$ ——第 n 帧图像的编码数据从 BBV 缓冲区移出的时间,单位为秒(s)；
- $t(n+1) - t(n)$ ——第 $n+1$ 帧和第 n 帧图像的解码时间间隔,单位为秒(s)；
- $R(n)$ ——第 n 帧图像进入 BBV 缓冲区的速率。

在视频序列开始和结束时,可能缺少参数来无歧义地确定这个时间间隔,此时可采用下面的方法来处理。

图 C.1 为第 n 帧图像的编码数据按方法一进入 BBV 缓冲区的示意图。

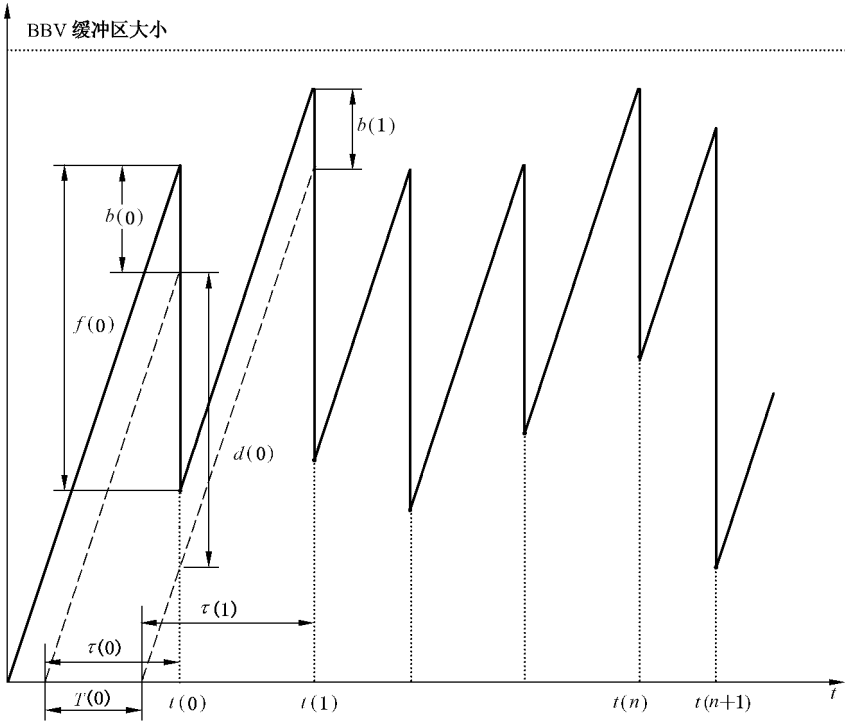


图 C.1 BBV 缓冲区占用情况一

C.3.1.1.2 视频序列开始时的歧义性

对视频序列进行随机访问时,序列头后的第 1 帧图像缺少在此之前的 I 帧或 P 帧。在这种情况下,如果位流是系统流的一部分,可从系统流来确定解码时间间隔。

如果还不能无歧义地确定解码时间间隔,就无法确定 $R(n)$ 。此时在一段有限的时间内(这个时间总是小于 BbvDelay 的最大值)BBV 不能准确地确定充满度的变化轨迹,从而无法在整个位流中严格地验证 BBV 缓冲区。编码器总是知道在每个重复序列头后 $t(n+1) - t(n)$ 的值,因此也知道如何生成不违反 BBV 限制的位流。

C.3.1.1.3 视频序列结束时的歧义性

如果一帧图像的编码数据后第 1 个起始码是视频序列结束码,此时无法确定该帧图像编码数据的位数。在这种情况下,应存在一个输入速率,这个速率不应导致 BBV 缓冲区上溢;在 low_delay 的值为“1”时,这个速率也不应导致缓冲区下溢。该速率应小于在视频序列头中定义的最大速率。

视频序列的第 1 帧图像的起始码前的所有数据和这个起始码输入 BBV 缓冲区后,每一帧图像的编码数据应在由位流中的 BbvDelay 规定的时间内输入 BBV 缓冲区,并在这个时间开始解码处理。输入速率由公式 C.2 确定。

所有位流的 $R(n)$ 均不应大于 $R_{\max}$ 。

在 CBR 情况下,视频序列中的 $R(n)$ 在 BbvDelay 允许的精度范围内是一个常数。

C.3.1.2 方法二

如果 BbvDelay 的值为 BbvDelayMax,编码数据按以下方式输入 BBV 缓冲区:

- 如果 BBV 缓冲区没有充满,数据以速率 $R_{\max}$ 输入缓冲区;
- 如果 BBV 缓冲区充满,则数据不能进入该缓冲区直到缓冲区中的部分数据被移出。

视频序列的第 1 帧图像的起始码前的所有数据和这个起始码输入 BBV 缓冲区后,数据继续输入 BBV 缓冲区直到缓冲区充满,并在这个时间开始解码处理,见图 C.2。

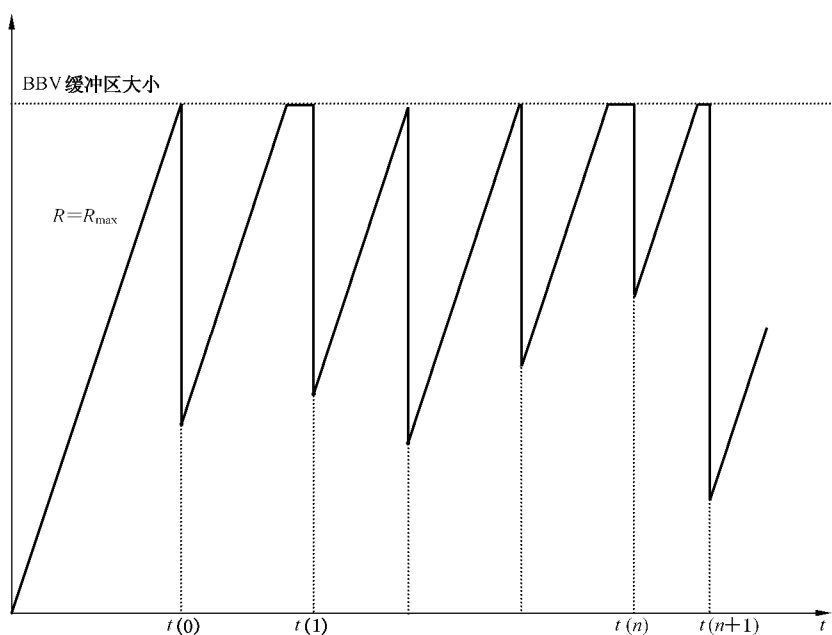


图 C.2 BBV 缓冲区占用情况二

C.3.2 数据移出

C.3.2.1 非低延迟

low_delay 的值为“0”时,数据按照下面的方式从 BBV 缓冲区中移出:在每帧图像的解码时刻 $t(n)$,如果 BBV 缓冲区充满度 $B(n)$ 小于 $f(n)$ 则发生下溢,否则移出该帧图像编码数据并解码。

在 $t(n)$ 时刻,即移出第 n 帧图像的位之前和检测 BBV 缓冲区之时,该图像的所有编码数据均应在

缓冲区中,然后该图像被瞬时移出。

如果 low_delay 的值为“0”,BBV 缓冲区不应发生下溢。

C.3.2.2 低延迟

low_delay 的值为“1”时,在移出一帧图像的编码数据之前,要检测 BBV 缓冲区 BbvCheckTimes 加 1 次。如果 BbvCheckTimes 大于“0”,当前解码图像定义为大图像。

第 n 帧图像的数据按照下面的方式从 BBV 缓冲区中移出:每隔一个检测时间间隔检测一次 BBV 缓冲区,完成 BbvCheckTimes 加 1 次检测,此时 BBV 缓冲区充满度应小于 BBS,该图像的所有编码数据均应在缓冲区中,然后该图像被瞬时移出。

如果当前解码图像不是大图像,检测时间间隔在 C.4 定义。

如果当前解码图像是大图像,检测时间间隔是一个帧时间间隔。

一个视频序列的最后一帧不应是大图像。

C.4 缓冲区检测时间间隔

C.4.1 非低延迟

当 low_delay 的值为“0”时,视频位流中可包含 B 帧,会产生进行图像重排序而造成的延迟。

当 progressive_sequence 的值为“1”并且 low_delay 的值为“0”时,BBV 缓冲区检测时间间隔 $t(n+1)-t(n)$ 是帧率的倒数 T 的倍数:

- 如果第 n 帧图像是 repeat_first_field 的值为“0”的 B 帧,检测时间间隔等于 T ;
- 如果第 n 帧图像是 repeat_first_field 的值为“1”的 B 帧,并且 top_field_first 的值为“0”,检测时间间隔等于 $2T$;
- 如果第 n 帧图像是 repeat_first_field 的值为“1”的 B 帧,并且 top_field_first 的值为“1”,检测时间间隔等于 $3T$;
- 如果第 n 帧图像是 P 帧或 I 帧,并且前一个 P 帧或 I 帧的 repeat_first_field 的值为“0”,检测时间间隔等于 T ;
- 如果第 n 帧图像是 P 帧或 I 帧,并且前一个 P 帧或 I 帧的 repeat_first_field 的值为“1”,top_field_first 的值为“0”,检测时间间隔等于 $2T$;
- 如果第 n 帧图像是 P 帧或 I 帧,并且前一个 P 帧或 I 帧的 repeat_first_field 的值为“1”,top_field_first 的值为“1”,检测时间间隔等于 $3T$ 。

当 progressive_sequence 的值为“0”并且 low_delay 的值为“0”时,BBV 缓冲区检测时间间隔 $t(n+1)-t(n)$ 是帧率的倒数 T 的倍数:

- 如果第 n 帧图像是 repeat_first_field 的值为“0”的 B 帧,检测时间间隔等于 T ;
- 如果第 n 帧图像是 repeat_first_field 的值为“1”的 B 帧,检测时间间隔等于 $1.5T$;
- 如果第 n 帧图像是 P 帧或 I 帧,并且前一个 P 帧或 I 帧的 repeat_first_field 的值为“0”,检测时间间隔等于 T ;
- 如果第 n 帧图像是 P 帧或 I 帧,并且前一个 P 帧或 I 帧的 repeat_first_field 的值为“1”,检测时间间隔等于 $1.5T$ 。

C.4.2 低延迟

当 progressive_sequence 的值为“1”并且 low_delay 的值为“1”时,BBV 缓冲区检测时间间隔 $t(n+1)-t(n)$ 是帧率的倒数 T 的倍数:

- 如果第 n 帧图像是 repeat_first_field 的值为“0”的 P 帧或 I 帧,检测时间间隔等于 T ;

- b) 如果第 n 帧图像是 repeat_first_field 的值为“1”并且 top_field_first 等于“0”的 P 帧或 I 帧,检测时间间隔等于 $2 T$;
- c) 如果第 n 帧图像是 repeat_first_field 的值为“1”并且 top_field_first 等于“1”的 P 帧或 I 帧,检测时间间隔等于 $3 T$ 。

当 progressive_sequence 的值为“0”并且 low_delay 的值为“1”时,BBV 缓冲区检测时间间隔 $t(n+1)-t(n)$ 是帧率的倒数 T 的倍数:

- a) 如果第 n 帧图像是 repeat_first_field 的值为“0”的 P 帧或 I 帧,检测时间间隔等于 T ;
- b) 如果第 n 帧图像是 repeat_first_field 的值为“1”的 P 帧或 I 帧,检测时间间隔等于 $1.5 T$ 。

附 录 D
(规范性附录)
基本熵编码码表

本附录给出了基本熵编码码表。表 D.1~表 D.20 中 Run 表示量化系数游程,Level 表示量化系数值,EOB 栏及 Level 栏的各列数据表示语法元素 trans_coefficient 的值。解码时根据 trans_coefficient 可以查表得到 Level 和 Run。表中 EOB 栏对应的数字表示代表 EOB 的 trans_coefficient 的值。

用于解码帧内编码亮度块的游程和非零量化系数值的映射表 VLC0_Intra 见表 D.1。

表 D.1 VLC0_Intra(用于解码帧内编码亮度块的游程和非零量化系数值的映射表)

Run 的值	Level>0			RefAbsLevel 的值	Run 的值	Level>0			RefAbsLevel 的值
	1	2	3			1	2	3	
0	0	22	38	4	12	26	—	—	2
1	2	32	—	3	13	28	—	—	2
2	4	44	—	3	14	30	—	—	2
3	6	50	—	3	15	34	—	—	2
4	8	54	—	3	16	36	—	—	2
5	10	—	—	2	17	40	—	—	2
6	12	—	—	2	18	42	—	—	2
7	14	—	—	2	19	46	—	—	2
8	16	—	—	2	20	48	—	—	2
9	18	—	—	2	21	52	—	—	2
10	20	—	—	2	22	56	—	—	2
11	24	—	—	2					

用于解码帧内编码亮度块的游程和非零量化系数值的映射表 VLC1_Intra 见表 D.2。

表 D.2 VLC1_Intra(用于解码帧内编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0						RefAbsLevel 的值
		1	2	3	4	5	6	
	8	—	—	—	—	—	—	
0	—	0	4	15	27	41	55	7
1	—	2	17	35	—	—	—	4
2	—	6	25	53	—	—	—	4
3	—	9	33	—	—	—	—	3
4	—	11	39	—	—	—	—	3
5	—	13	45	—	—	—	—	3

表 D.2 (续)

Run 的值	EOB	Level>0						RefAbsLevel 的值
		1	2	3	4	5	6	
	8	—	—	—	—	—	—	
6	—	19	49	—	—	—	—	3
7	—	21	51	—	—	—	—	3
8	—	23	—	—	—	—	—	2
9	—	29	—	—	—	—	—	2
10	—	31	—	—	—	—	—	2
11	—	37	—	—	—	—	—	2
12	—	43	—	—	—	—	—	2
13	—	47	—	—	—	—	—	2
14	—	57	—	—	—	—	—	2

用于解码帧内编码亮度块的游程和非零量化系数值的映射表 VLC2_Intra 见表 D.3。

表 D.3 VLC2_Intra(用于解码帧内编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0									RefAbsLevel 的值
		1	2	3	4	5	6	7	8	9	
	8	—	—	—	—	—	—	—	—	—	
0	—	0	2	6	13	17	27	35	45	55	10
1	—	4	11	21	33	49	—	—	—	—	6
2	—	9	23	37	—	—	—	—	—	—	4
3	—	15	29	51	—	—	—	—	—	—	4
4	—	19	39	—	—	—	—	—	—	—	3
5	—	25	43	—	—	—	—	—	—	—	3
6	—	31	53	—	—	—	—	—	—	—	3
7	—	41	—	—	—	—	—	—	—	—	2
8	—	47	—	—	—	—	—	—	—	—	2
9	—	57	—	—	—	—	—	—	—	—	2

用于解码帧内编码亮度块的游程和非零量化系数值的映射表 VLC3_Intra 见表 D.4。

表 D.4 VLC3_Intra(用于解码帧内编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0												RefAbsLevel 的值
		1	2	3	4	5	6	7	8	9	10	11	12	
	8	—	—	—	—	—	—	—	—	—	—	—	—	
0	—	0	2	4	9	11	17	21	25	33	39	45	55	13
1	—	6	13	19	29	35	47	—	—	—	—	—	—	7
2	—	15	27	41	57	—	—	—	—	—	—	—	—	5
3	—	23	37	53	—	—	—	—	—	—	—	—	—	4
4	—	31	51	—	—	—	—	—	—	—	—	—	—	3
5	—	43	—	—	—	—	—	—	—	—	—	—	—	2
6	—	49	—	—	—	—	—	—	—	—	—	—	—	2

用于解码帧内编码亮度块的游程和非零量化系数值的映射表 VLC4_Intra 见表 D.5。

表 D.5 VLC4_Intra(用于解码帧内编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0																	RefAbsLevel 的值
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	
	6	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
0	—	0	2	4	7	9	11	15	17	21	23	29	33	35	43	47	49	57	18
1	—	13	19	27	31	37	45	55	—	—	—	—	—	—	—	—	—	—	8
2	—	25	41	51	—	—	—	—	—	—	—	—	—	—	—	—	—	—	4
3	—	39	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	2
4	—	53	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	2

用于解码帧内编码亮度块的游程和非零量化系数值的映射表 VLC5_Intra 见表 D.6。

表 D.6 VLC5_Intra(用于解码帧内编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0																				RefAbs	
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	Level 的值
	0	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
0	—	1	3	5	7	9	11	13	15	17	19	23	25	27	31	33	37	41	45	49	51	55	22
1	—	21	29	35	43	47	53	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	7
2	—	39	57	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	3

用于解码帧内编码亮度块的游程和非零量化系数值的映射表 VLC6_Intra 见表 D.7。

表 D.7 VLC6_Intra(用于解码帧内编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0																					RefAbs	
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	Level 的值	
	0	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
0	—	1	3	5	7	9	11	13	15	17	19	21	23	25	27	29	31	35	37	39	41	43	27	
1	—	33	45	55	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
Run 的值		Level>0																						
		22	23	24	25	26	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
0	—	47	49	51	53	57	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	27
1	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—

用于解码帧间编码亮度块的游程和非零量化系数值的映射表 VLC0_Inter 见表 D.8。

表 D.8 VLC0_Inter(用于解码帧间编码亮度块的游程和非零量化系数值的映射表)

Run 的值	Level>0			RefAbsLevel 的值	Run 的值	Level>0			RefAbsLevel 的值
	1	2	3			1	2	3	
0	0	26	40	4	13	28	—	—	2
1	2	46	—	3	14	30	—	—	2
2	4	—	—	2	15	32	—	—	2
3	6	—	—	2	16	34	—	—	2
4	8	—	—	2	17	36	—	—	2
5	10	—	—	2	18	38	—	—	2
6	12	—	—	2	19	42	—	—	2
7	14	—	—	2	20	44	—	—	2
8	16	—	—	2	21	48	—	—	2
9	18	—	—	2	22	50	—	—	2
10	20	—	—	2	23	52	—	—	2
11	22	—	—	2	24	54	—	—	2
12	24	—	—	2	25	56	—	—	2

用于解码帧间编码亮度块的游程和非零量化系数值的映射表 VLC1_Inter 见表 D.9。

表 D.9 VLC1_Inter(用于解码帧间编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0				RefAbsLevel 的值
		1	2	3	4	
	2	—	—	—	—	
0	—	0	13	29	47	5

D.9 (续)

Run 的值	EOB	Level>0				RefAbsLevel 的值
		1	2	3	4	
	2	—	—	—	—	
1	—	3	23	57	—	4
2	—	5	35	—	—	3
3	—	7	39	—	—	3
4	—	9	43	—	—	3
5	—	11	49	—	—	3
6	—	15	55	—	—	3
7	—	17	—	—	—	2
8	—	19	—	—	—	2
9	—	21	—	—	—	2
10	—	25	—	—	—	2
11	—	27	—	—	—	2
12	—	31	—	—	—	2
13	—	33	—	—	—	2
14	—	37	—	—	—	2
15	—	41	—	—	—	2
16	—	45	—	—	—	2
17	—	51	—	—	—	2
18	—	53	—	—	—	2

用于解码帧间编码亮度块的游程和非零量化系数值的映射表 VLC2_Inter 见表 D.10。

表 D.10 VLC2_Inter(用于解码帧间编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0						RefAbsLevel 的值
		1	2	3	4	5	6	
	2	—	—	—	—	—	—	
0	—	0	5	11	23	35	47	7
1	—	3	13	27	49	—	—	5
2	—	7	21	45	—	—	—	4
3	—	9	29	55	—	—	—	4
4	—	15	37	—	—	—	—	3
5	—	17	41	—	—	—	—	3
6	—	19	53	—	—	—	—	3

表 D.10 (续)

Run 的值	EOB	Level>0						RefAbsLevel 的值
		1	2	3	4	5	6	
	2	—	—	—	—	—	—	
7	—	25	—	—	—	—	—	2
8	—	31	—	—	—	—	—	2
9	—	33	—	—	—	—	—	2
10	—	39	—	—	—	—	—	2
11	—	43	—	—	—	—	—	2
12	—	51	—	—	—	—	—	2
13	—	57	—	—	—	—	—	2

用于解码帧间编码亮度块的游程和非零量化系数值的映射表 VLC3_Inter 见表 D.11。

表 D.11 VLC3_Inter(用于解码帧间编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0									RefAbsLevel 的值
		1	2	3	4	5	6	7	8	9	
	2	—	—	—	—	—	—	—	—	—	
0	—	0	3	7	13	17	27	35	43	55	10
1	—	5	11	21	33	51	—	—	—	—	6
2	—	9	23	37	57	—	—	—	—	—	5
3	—	15	29	47	—	—	—	—	—	—	4
4	—	19	41	—	—	—	—	—	—	—	3
5	—	25	49	—	—	—	—	—	—	—	3
6	—	31	—	—	—	—	—	—	—	—	2
7	—	39	—	—	—	—	—	—	—	—	2
8	—	45	—	—	—	—	—	—	—	—	2
9	—	53	—	—	—	—	—	—	—	—	2

用于解码帧间编码亮度块的游程和非零量化系数值的映射表 VLC4_Inter 见表 D.12。

表 D.12 VLC4_Inter(用于解码帧间编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0												RefAbsLevel 的值
		1	2	3	4	5	6	7	8	9	10	11	12	
	2	—	—	—	—	—	—	—	—	—	—	—	—	
0	—	0	3	5	9	11	17	21	25	33	41	45	55	13

D.12 (续)

Run 的值	EOB	Level>0												RefAbsLevel 的值
		1	2	3	4	5	6	7	8	9	10	11	12	
	2	—	—	—	—	—	—	—	—	—	—	—	—	
1	—	7	13	19	29	35	49	—	—	—	—	—	—	7
2	—	15	27	43	57	—	—	—	—	—	—	—	—	5
3	—	23	37	51	—	—	—	—	—	—	—	—	—	4
4	—	31	53	—	—	—	—	—	—	—	—	—	—	3
5	—	39	—	—	—	—	—	—	—	—	—	—	—	2
6	—	47	—	—	—	—	—	—	—	—	—	—	—	2

用于解码帧间编码亮度块的游程和非零量化系数值的映射表 VLC5_Inter 见表 D.13。

表 D.13 VLC5_Inter(用于解码帧间编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0																RefAbsLevel 的值
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
	0	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
0	—	1	3	5	7	9	13	15	17	21	25	29	33	39	43	49	53	17
1	—	11	19	27	31	41	45	57	—	—	—	—	—	—	—	—	—	8
2	—	23	37	51	—	—	—	—	—	—	—	—	—	—	—	—	—	4
3	—	35	55	—	—	—	—	—	—	—	—	—	—	—	—	—	—	3
4	—	47	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	2

用于解码帧间编码亮度块的游程和非零量化系数值的映射表 VLC6_Inter 见表 D.14。

表 D.14 VLC6_Inter(用于解码帧间编码亮度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0																				RefAbs	
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	Level 的值
	0	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
0	—	1	3	5	7	9	11	13	17	19	21	23	25	29	33	35	39	41	43	47	49	57	22
1	—	15	27	37	45	55	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	6
2	—	31	51	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	3
3	—	53	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	2

用于解码色度块的游程和非零量化系数值的映射表 VLC0_Chroma 见表 D.15。

表 D.15 VLC0_Chroma(用于解码色度块的游程和非零量化系数值的映射表)

Run 的值	Level>0				RefAbsLevel 的值	Run 的值	Level>0				RefAbsLevel 的值
	1	2	3	4			1	2	3	4	
0	0	14	32	56	5	13	28	—	—	—	2
1	2	48	—	—	3	14	30	—	—	—	2
2	4	—	—	—	2	15	34	—	—	—	2
3	6	—	—	—	2	16	36	—	—	—	2
4	8	—	—	—	2	17	38	—	—	—	2
5	10	—	—	—	2	18	40	—	—	—	2
6	12	—	—	—	2	19	42	—	—	—	2
7	16	—	—	—	2	20	44	—	—	—	2
8	18	—	—	—	2	21	46	—	—	—	2
9	20	—	—	—	2	22	50	—	—	—	2
10	22	—	—	—	2	23	52	—	—	—	2
11	24	—	—	—	2	24	54	—	—	—	2
12	26	—	—	—	2						

用于解码色度块的游程和非零量化系数值的映射表 VLC1_Chroma 见表 D.16。

表 D.16 VLC1_Chroma(用于解码色度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0					RefAbsLevel 的值
		1	2	3	4	5	
	0	—	—	—	—	—	
0	—	1	5	15	29	43	6
1	—	3	21	45	—	—	4
2	—	7	37	—	—	—	3
3	—	9	41	—	—	—	3
4	—	11	53	—	—	—	3
5	—	13	—	—	—	—	2
6	—	17	—	—	—	—	2
7	—	19	—	—	—	—	2
8	—	23	—	—	—	—	2
9	—	25	—	—	—	—	2
10	—	27	—	—	—	—	2
11	—	31	—	—	—	—	2
12	—	33	—	—	—	—	2

表 D.16 (续)

Run 的值	EOB	Level>0					RefAbsLevel 的值
		1	2	3	4	5	
	0	—	—	—	—	—	
13	—	35	—	—	—	—	2
14	—	39	—	—	—	—	2
15	—	47	—	—	—	—	2
16	—	49	—	—	—	—	2
17	—	51	—	—	—	—	2
18	—	55	—	—	—	—	2
19	—	57	—	—	—	—	2

用于解码色度块的游程和非零量化系数值的映射表 VLC2_Chroma 见表 D.17。

表 D.17 VLC2_Chroma(用于解码色度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0									RefAbsLevel 的值
		1	2	3	4	5	6	7	8	9	
	2	—	—	—	—	—	—	—	—	—	
0	—	0	3	7	11	17	27	33	47	53	10
1	—	5	13	21	37	55	—	—	—	—	6
2	—	9	23	41	—	—	—	—	—	—	4
3	—	15	31	57	—	—	—	—	—	—	4
4	—	19	43	—	—	—	—	—	—	—	3
5	—	25	45	—	—	—	—	—	—	—	3
6	—	29	—	—	—	—	—	—	—	—	2
7	—	35	—	—	—	—	—	—	—	—	2
8	—	39	—	—	—	—	—	—	—	—	2
9	—	49	—	—	—	—	—	—	—	—	2
10	—	51	—	—	—	—	—	—	—	—	2

用于解码色度块的游程和非零量化系数值的映射表 VLC3_Chroma 见表 D.18。

表 D.18 VLC3_Chroma(用于解码色度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0													RefAbsLevel 的值
		1	2	3	4	5	6	7	8	9	10	11	12	13	
	0	—	—	—	—	—	—	—	—	—	—	—	—	—	
0	—	1	3	5	7	11	15	19	23	29	35	43	47	53	14

表 D.18 (续)

Run 的值	EOB	Level>0													RefAbsLevel 的值
		1	2	3	4	5	6	7	8	9	10	11	12	13	
	0	—	—	—	—	—	—	—	—	—	—	—	—	—	
1	—	9	13	21	31	39	51	—	—	—	—	—	—	—	7
2	—	17	27	37	—	—	—	—	—	—	—	—	—	—	4
3	—	25	41	—	—	—	—	—	—	—	—	—	—	—	3
4	—	33	55	—	—	—	—	—	—	—	—	—	—	—	3
5	—	45	—	—	—	—	—	—	—	—	—	—	—	—	2
6	—	49	—	—	—	—	—	—	—	—	—	—	—	—	2
7	—	57	—	—	—	—	—	—	—	—	—	—	—	—	2

用于解码色度块的游程和非零量化系数值的映射表 VLC4_Chroma 见表 D.19。

表 D.19 VLC4_Chroma(用于解码色度块的游程和非零量化系数值的映射表)

Run 的值	EOB	Level>0																			RefAbs Level 的值
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	
	0	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	
0	—	1	3	5	7	9	11	13	15	19	21	23	27	29	33	37	41	43	51	55	20
1	—	17	25	31	39	45	53	—	—	—	—	—	—	—	—	—	—	—	—	—	7
2	—	35	49	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	3
3	—	47	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	2
4	—	57	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	2

CurrentVLCTable 和 MaxRun 的映射关系见表 D.20。

表 D.20 CurrentVLCTable 和 MaxRun 的映射关系

CurrentVLCTable	MaxRun 的值	CurrentVLCTable	MaxRun 的值	CurrentVLCTable	MaxRun 的值
VLC0_Intra	22	VLC0_Inter	25	VLC0_Chroma	24
VLC1_Intra	14	VLC1_Inter	18	VLC1_Chroma	19
VLC2_Intra	9	VLC2_Inter	13	VLC2_Chroma	10
VLC3_Intra	6	VLC3_Inter	9	VLC3_Chroma	7
VLC4_Intra	4	VLC4_Inter	6	VLC4_Chroma	4
VLC5_Intra	2	VLC5_Inter	4		
VLC6_Intra	1	VLC6_Inter	3		

附录 E (资料性附录)

高级熵编码解码器参考实现方法

E.1 概述

本附录描述了高级熵编码解码器的参考实现方法,包括初始化过程的参考实现方法(见 E.2)、decode_decision 过程的参考实现方法(见 E.3)、decode_bypass 过程的参考实现方法(见 E.4)和 decode_aec_stuffing_bit 过程的参考实现方法(见 E.5)。

E.2~E.5 的参考实现方法分别代替 8.4.2.2、8.4.4.3.2、8.4.4.3.3 和 8.4.4.3.4 的方法。本附录描述的参考实现方法应同时使用。

E.2 初始化高级熵编码解码器

rS1、rT1、valueS、valueT、bFlag 和 boundS 是用于高级熵编码解码器的变量。rT1 的位宽是 8 位, valueT 的位宽是 9 位。rS1 的值应初始化为“0”;rT1 的值应初始化为“0xFF”,boundS 的值应初始化为“0xFE”。

valueS 和 valueT 的初始化过程用伪代码描述如下:

```
valueS=0
valueT=read_bits(9)
while ( ! ((valueT >> 8) & 0x01) & valueS < boundS ) {
    valueT = (valueT << 1) | read_bits(1)
    valueS++
}
if ( valueT < 0x100 )
    bFlag=1
else
    bFlag=0
valueT=valueT & 0xFF
```

E.3 decode_decision

如果 contextWeighting 的值为 1, decode_decision 过程的输入是 rS1、rT1、valueS、valueT、bFlag 以及上下文模型 ctx1、ctx2;否则 decode_decision 过程的输入是 rS1、rT1、valueS、valueT、bFlag 以及上下文模型 ctx。decode_decision 过程的输出是二元符号值 binVal。decode_decision 过程用伪代码描述如下:

```
decode_decision( )
{
    if(contextWeighting==1) {
        if(ctx1->mps==ctx2->mps) {
            predMps=ctx1->mps
            lgPmps=(ctx1->lgPmps+ctx2->lgPmps)/2
        }
        else {
```

```

    if( ctx1->lgPmps < ctx2->lgPmps ) {
        predMps=ctx1->mps
        lgPmps=1 023-((ctx2->lgPmps-ctx1->lgPmps) >> 1)
    }
    else {
        predMps=ctx2->mps
        lgPmps=1 023-((ctx1->lgPmps-ctx2->lgPmps) >> 1)
    }
}
}
else {
    predMps=ctx->mps
    lgPmps= ctx->lgPmps
}

if ( rT1 >= (lgPmps >> 2) ) {
    rS2=rS1
    rT2=rT1-(lgPmps >> 2)
    sFlag=0
}
else {
    rS2=rS1+ 1
    rT2=256+ rT1-(lgPmps >> 2)
    sFlag=1
}

if( (rS2>valueS || (rS2 == valueS && valueT >= rT2)) && bFlag == 0 ) {
    binVal=! predMps
    if ( sFlag == 0 )
        tRlps=lgPmps >> 2
    else
        tRlps=rT1+ (lgPmps >> 2)
    if ( rS2 == valueS )
        valueT=valueT-rT2
    else
        valueT=256+ ((valueT << 1) | read_bits(1))-rT2
    while (tRlps < 0x100 ) {
        tRlps=tRlps << 1
        valueT=(valueT << 1) | read_bits(1)
    }
    rT1=tRlps & 0xFF
}
else {

```

```

        binVal=predMps
        rS1=rS2
        rT1=rT2
    }

    if ( binVal != predMps || ( binVal == predMps && bFlag == 1 && rS2 == boundS )
) {
        rS1=0
        valueS=0
        while ( valueT < 0x100 && valueS < boundS ) {
            valueS++
            valueT=(valueT << 1) | read_bits(1)
        }
        if ( valueT < 0x100 )
            bFlag=1
        else
            bFlag=0
        valueT=valueT & 0xFF
    }

    if ( contextWeighting == 1 ) {
        ctx1=update_ctx(binVal,ctx1)
        ctx2=update_ctx(binVal,ctx2)
    }
    else
        ctx=update_ctx(binVal,ctx)
    return (binVal)
}

```

E.4 decode_bypass

decode_bypass 过程的输入是 rS1、rS2、valueS、valueT 和 bFlag。decode_bypass 过程的输出是二元符号值 binVal。decode_bypass 过程用伪代码描述如下：

```

decode_bypass( )
{
    predMps=0
    lgPmps=1 023
    if ( rT1 >= (lgPmps >> 2) ) {
        rS2=rS1
        rT2=rT1-(lgPmps >> 2)
        sFlag=0
    }
    else {
        rS2=rS1+ 1

```

```

    rT2=256+ rT1-(lgPmps >> 2)
    sFlag=1
}

if( (rS2>valueS || (rS2 == valueS && valueT >= rT2)) && bFlag == 0 ) {
    binVal=! predMps
    if ( sFlag == 0 )
        tRlps=lgPmps >> 2
    else
        tRlps=rT1+ (lgPmps >> 2)
    if ( rS2 == valueS )
        valueT=valueT-rT2
    else
        valueT=((valueT << 1 ) | read_bits(1))-rT2+ 256
    while (tRlps < 0x100 ) {
        tRlps=tRlps << 1
        valueT=(valueT << 1 ) | read_bits(1)
    }
    rT1=tRlps & 0xFF
}
else {
    binVal=predMps
    rS1=rS2
    rT1=rT2
}

if(binVal != predMps || (binVal==predMps && bFlag==1 && rS2==boundS ) ) {
    rS1=0
    valueS=0
    while (valueT < 0x100 && valueS < boundS ) {
        valueS++
        valueT=(valueT << 1 ) | read_bits(1)
    }
    if ( valueT < 0x100 )
        bFlag=1
    else
        bFlag=0
    valueT=valueT & 0xFF
}

return (binVal)
}

```

E.5 decode_aec_stuffing_bit

decode_aec_stuffing_bit 过程的输入是 rS1、rS2、valueS、valueT 和 bFlag。decode_aec_stuffing_bit 过程的输出是二元符号值 binVal。decode_aec_stuffing_bit 过程用伪代码描述如下：

```

decode_aec_stuffing_bit( )
{
    predMps=0
    lgPmps= 4
    if ( rT1 >= (lgPmps >> 2) ) {
        rS2=rS1
        rT2=rT1-(lgPmps >> 2 )
        sFlag=0
    }
    else {
        rS2=rS1+ 1
        rT2=256+ rT1-(lgPmps >> 2)
        sFlag=1
    }

    if( (rS2>valueS || (rS2==valueS && valueT >= rT2)) && bFlag==0 ) {
        binVal=! predMps
        if ( sFlag==0 )
            tRlps=lgPmps >> 2
        else
            tRlps=rT1+ (lgPmps >> 2)
        if ( rS2==valueS )
            valueT=valueT-rT2
        else
            valueT=256+ ((valueT << 1) | read_bits(1))-rT2
        while ( tRlps < 0x100 ) {
            tRlps=tRlps << 1
            valueT=(valueT << 1) | read_bits(1)
        }
        rT1=tRlps & 0xFF
    }
    else {
        binVal=predMps
        rS1=rS2
        rT1=rT2
    }

    if ( binVal != predMps || ( binVal==predMps &&bFlag==1 && rS2==boundS ) ) {
        rS1=0
    }
}

```

```
valueS=0
while ( valueT < 0x100 && valueS < boundS ) {
    valueS++
    valueT=(valueT << 1) | read_bits(1)
}
if ( valueT < 0x100 )
    bFlag=1
else
    bFlag=0
valueT=valueT & 0xFF
}

return (binVal)
}
```
